

计算机网络 Computer Network

南方科技大学 计算机科学与工程系 11812804 董正

计算机网络 Computer Network

第一章 计算机网络概述

1.1 网络边缘 Network Edge

1.1.1 接入网

1.1.1.1 家庭接入 Home Network

1.1.1.2 企业接入 Institutional Network (Ethernet)

1.1.1.3 广域无线接入 Mobile Network

1.1.2 物理媒体 Physical Media

1.2 网络核心 Network Core

1.2.1 分组交换 Packet Switching

1.2.2 电路交换 Circuit Switching

1.2.3 因特网结构: 网络的网络

1.3 分组交换网中的时延、丢包和吞吐量

1.3.1 时延

1.3.2 排队时延和丢包

1.3.4 吞吐量 Throughput

1.4 协议层次 Protocol Layers

1.4.1 分层的体系结构

1.4.2 封装 Encapsulation

1.5 网络安全 Network Security

第二章 应用层 Application Layer

2.1 应用层协议原理 Principles of Network Applications

2.1.1 网络应用程序体系结构 Application Architectures

2.1.2 进程通信

2.1.3 可供应用程序使用的运输服务

2.1.4 因特网提供的运输服务

2.1.5 应用层协议 Application-layer Protocol

2.2 Web and HTTP

2.2.1 HTTP 概况

2.2.2 非持续连接和持续连接

2.2.3 HTTP 报文格式

2.2.4 Cookie

2.2.5 Web 缓存

2.2.6 条件 GET

2.3 电子邮件 E-Mail

2.3.1 电子邮件系统的组成

2.3.2 SMTP

2.3.3 邮件访问协议 Mail Access Protocol

2.4 DNS: 因特网的目录服务

2.4.1 DNS 提供的服务

2.4.2 DNS 的工作原理

- 2.4.2.1 分布式、层次数据库
 - 2.4.2.2 DNS 缓存
 - 2.4.3 DNS 记录和报文
 - 2.5 P2P
 - 2.5.1 P2P 体系结构的扩展性
 - 2.5.2 BitTorrent
 - 2.6 视频流与 CDN
 - 2.6.1 因特网视频 Video Streaming
 - 2.6.2 DASH
 - 2.6.3 内容分发网 CDN
 - 2.7 Socket 编程
- 第三章 运输层 Transport Layer
- 3.1 运输层服务 Transport Layer Services
 - 3.2 多路复用与多路分解 Multiplexing & Demultiplexing
 - 3.2.1 无连接的多路复用与多路分解
 - 3.2.2 面向连接的多路复用与多路分解
 - 3.3 UDP
 - 3.3.1 UDP 的特性
 - 3.3.2 UDP 检验和
 - 3.4 可靠数据传输 Reliable Data Transfer
 - 3.4.1 构造可靠数据传输协议
 - 3.4.1.1 经完全可靠信道的可靠数据传输: rdt 1.0
 - 3.4.1.2 经具有比特差错信道的可靠传输策略: rdt 2.0
 - 3.4.1.3 rdt 2.1
 - 3.4.1.4 rdt 2.2
 - 3.4.1.5 经具有比特差错的丢包信道的可靠数据传输: rdt 3.0
 - 3.4.2 流水线可靠数据传输协议
 - 3.4.3 回退 N 步 Go-Back-N
 - 3.4.4 选择重传 Selective Repeat
 - 3.5 面向连接的运输: TCP
 - 3.5.1 TCP 连接
 - 3.5.2 TCP 报文段
 - 3.5.3 TCP 序号和确认号
 - 3.5.4 往返时间的估计与超时
 - 3.5.5 可靠数据传输
 - 3.5.6 流量控制 Flow Control
 - 3.5.7 TCP 连接管理 Connection Management
 - 3.6 拥塞控制原理 Principles of Congestion Control
 - 3.6.1 情况 1: 两个发送方和一台具有无限大缓存的路由器
 - 3.6.2 情况 2: 两个发送方和一台具有有限缓存的路由器
 - 3.6.3 情况 3: 4 个发送方和具有有限缓存的多台路由器和多跳路径
 - 3.7 TCP 拥塞控制
 - 3.7.1 慢启动 Slow Start
 - 3.7.2 拥塞避免 Congestion Avoidance
 - 3.7.3 快速恢复 Fast Recovery
 - 3.7.4 TCP 吞吐量
 - 3.7.5 TCP 公平性

3.7.6 明确拥塞通告：网络辅助拥塞控制

第四章 网络层: 数据平面 Network Layer: The Data Plane

4.1 网络层概述

4.1.1 转发和路由选择：数据平面和控制平面

4.1.2 网络服务模型 Network Service Model

4.2 路由器工作原理 Router

4.2.1 路由器结构概述 Router Architecture

4.2.2 输入端口处理和基于目的地转发

4.2.3 交换

4.2.4 输出端口处理

4.2.5 排队

4.2.6 分组调度

4.3 网际协议 Internet Protocol

4.3.1 IPv4 数据报格式

4.3.2 IPv4 数据报分片

4.3.3 IPv4 编址

4.3.4 网络地址转换 NAT

4.3.5 IPv6

4.4 通用转发和 SDN

第五章 网络层: 控制平面 Network Layer: The Control Plane

5.1 概述

5.2 路由选择算法 Routing Algorithm

5.2.1 链路状态路由选择算法 LS

5.2.2 距离路由向量算法 DV

5.3 因特网中自治系统内部的路由选择: OSPF

5.4 ISP 之间的路由选择: BGP

5.4.1 BGP 的作用

5.4.2 通告 BGP 路由信息

5.4.3 确定最好的路由

5.4.4 BGP 通告路由选择

5.4.5 Intra-AS 与 Inter-AS Routing 的比较

5.5 SDN 控制平面

5.5.1 SDN 结构

5.5.2 SDN 控制器

5.5.3 OpenFlow 协议

5.5.4 数据平面与控制平面交互的例子

5.6 ICMP: 因特网控制报文协议

5.7 网络管理和 SNMP

5.7.1 网络管理框架

5.7.2 简单网络管理协议 SNMP

第六章 链路层和局域网 Link Layer and LANs

6.1 链路层概述

6.1.1 链路层提供的服务

6.1.2 链路层在何处实现

6.2 差错检测和纠正技术 Error Detection and Correction

6.2.1 奇偶校验

6.2.2 循环冗余检测 Cyclic Redundancy Check

- 6.3 多路访问链路和协议 Multiple Access
 - 6.3.1 信道划分协议 Channel Partitioning Protocol
 - 6.3.2 随机接入协议 Random Access Protocol
 - 6.3.2.1 时隙 ALOHA
 - 6.3.2.2 ALOHA
 - 6.3.2.3 载波侦听多路访问 CSMA
 - 6.3.2.4 具有碰撞检测的 CSMA (CSMA/CD)
 - 6.3.3 轮流协议 Taking-Turns Protocol
 - 6.3.4 DOCSIS: 用于电缆因特网接入的链路层协议
 - 6.4 交换局域网 LANs
 - 6.4.1 链路层寻址和 ARP
 - 6.4.2 以太网
 - 6.4.3 链路层交换机
 - 6.4.3.1 交换机转发和过滤
 - 6.4.3.2 自学习 Self-Learning
 - 6.4.3.3 链路层交换机的性质
 - 6.4.3.4 交换机和路由器比较
 - 6.4.4 虚拟局域网
 - 6.5 链路虚拟化：网络作为链路层
 - 6.6 数据中心网络 Data Center Networking
 - 6.7 回顾：Web 页面请求的历程
 - 6.7.1 准备：DHCP、UDP、IP 和以太网
 - 6.7.2 仍在准备：DNS 和 ARP
 - 6.7.3 还是在准备：域内路由选择到 DNS 服务器
 - 6.7.4 Web 客户-服务器交互：TCP 和 HTTP
- 第七章 无线网络和移动网络 Wireless and Mobile Networks
- 7.1 概述
 - 7.1.1 无线网络的组成
 - 7.1.2 无线网络的分类
 - 7.2 无线链路和网络特征
 - 7.2.1 无线和有线的差别
 - 7.2.2 信噪比和比特差错率
 - 7.2.3 码分多址 CDMA
 - 7.3 WiFi: 802.11 无线 LAN
 - 7.3.1 802.11 体系结构
 - 7.3.2 802.11 MAC 协议
 - 7.3.3 IEEE 802.11 帧
 - 7.3.4 在相同的 IP 子网中的移动性
 - 7.3.5 802.11 中的高级特色
 - 7.3.6 个人域网络：蓝牙和 ZigBee
 - 7.4 蜂窝因特网接入 Cellular Internet Access
 - 7.4.1 2G 蜂窝网体系结构：语音与电话网连接
 - 7.4.2 3G 蜂窝数据网：将因特网扩展到蜂窝用户
 - 7.4.3 走向 4G: LTE
 - 7.5 移动管理原理
 - 7.5.1 寻址
 - 7.5.2 路由选择到移动节点

7.5.2.1 移动节点的间接路由选择 Indirect Routing

7.5.2.2 移动节点的直接路由选择 Direct Routing

END

第一章 计算机网络概述

1.1 网络边缘 Network Edge

- 端系统 End System = 主机 Host

- 客户端 Client
- 服务器 Server

Often in data centers

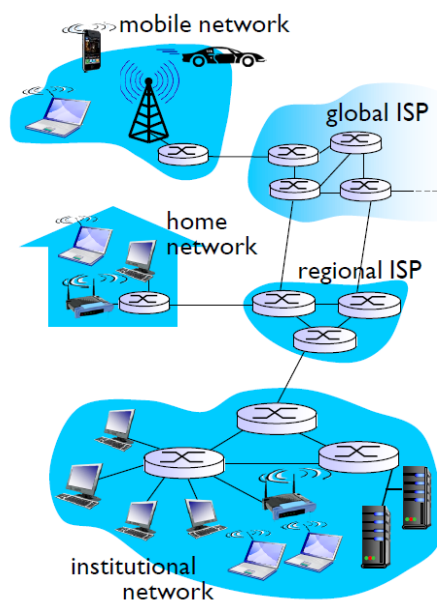
- 组成

端系统 End System

接入网 Access Network

物理媒体 Physical Media

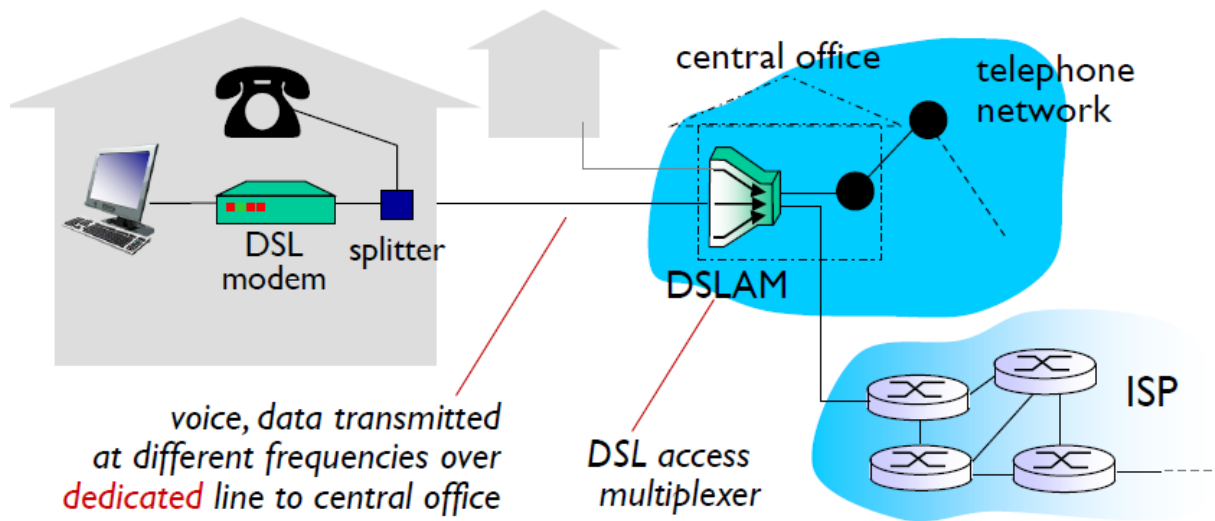
1.1.1 接入网



1.1.1.1 家庭接入 Home Network

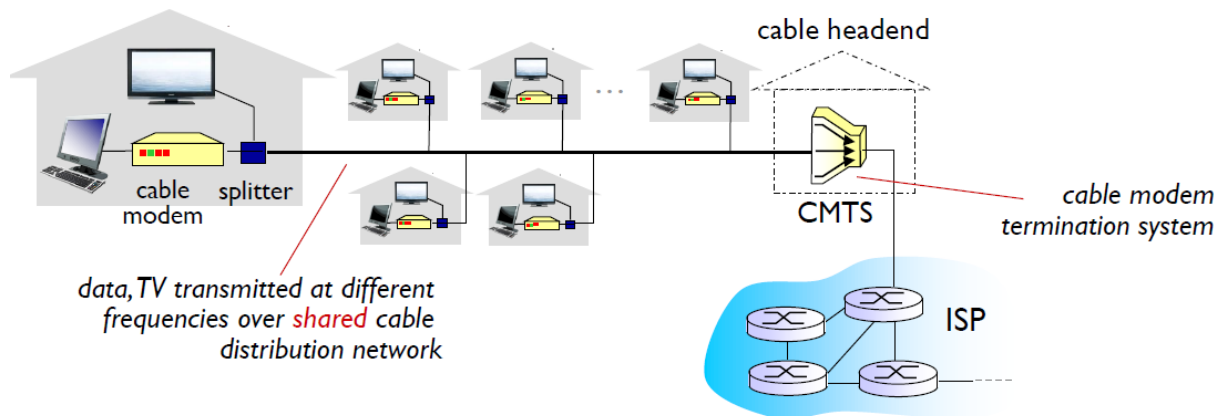
1. 数字用户线 Digital Subscriber Line (DSL)

- 调制解调器 Modem
- ISP: 电话公司
- 数字用户线接入复用器 DSLAM: 分流数据信号与电话信号
- 上行速率 < 2.5 Mbps 下行速率 < 24 Mbps

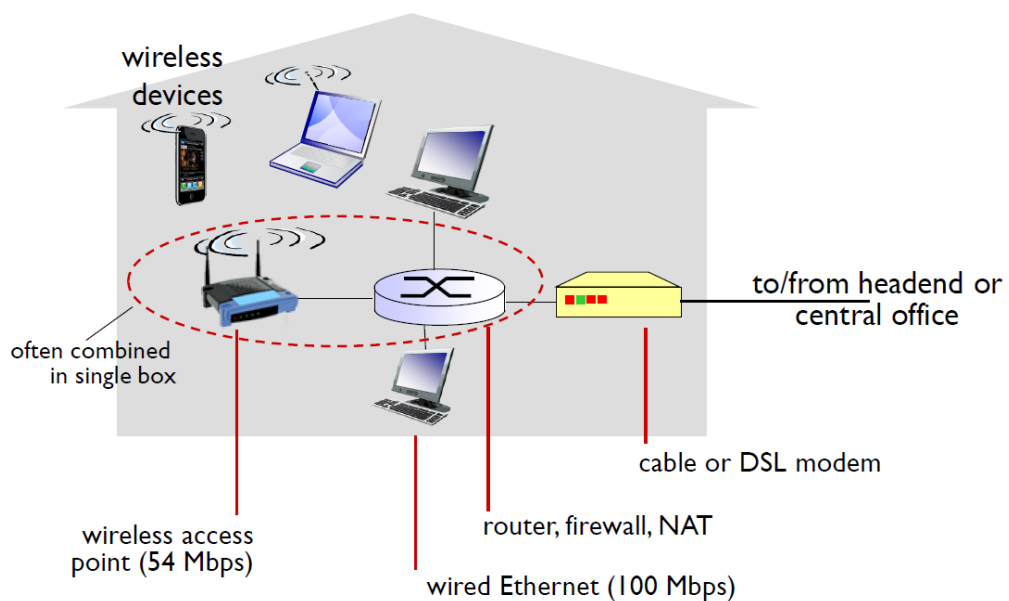


2. 电缆 Cable

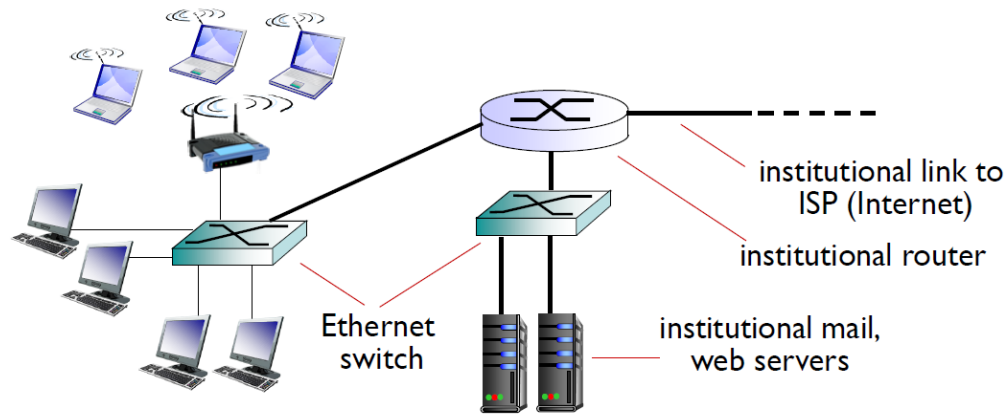
- 混合光纤同轴(Hybrid Fiber Coax, HFC)系统:
- 电缆调制解调器端接系统 Cable Modem Termination System (CMTS)
模拟信号 -> 数字信号



3. WIFI



1.1.1.2 企业接入 Institutional Network (Ethernet)



- ❖ typically used in companies, universities, etc
- ❖ 10 Mbps, 100Mbps, 1 Gbps, 10Gbps transmission rates
- ❖ today, end systems typically connect into Ethernet switch

1.1.1.3 广域无线接入 Mobile Network

1. WLAN

- 范围：建筑内
- 802.11 b/g/n/ac: 11, 54, 800, 1733 Mbps transmission rate

2. Wide-area wireless access

- 范围: 10km 蜂窝网络 (cellular)
- 2G, 3G, 4G, 5G

1.1.2 物理媒体 Physical Media

1. 导引型媒体 Guided Media

- 双绞铜线
无屏蔽双绞线 Unshielded Twisted Pair (UTP)
就是平常用的网线
- 同轴电缆 Coax
- 光纤 Fiber
长途导引型传输媒体

2. 非导引型媒体 Unguided Media

- 陆地无线电信道
LAN: 局域无线电信道
Cellular: 广域无线电信道
 - 卫星无线电信道
-

1.2 网络核心 Network Core

Router

1.2.1 分组交换 Packet Switching

- 分组 Packet (包)
源端将报文 (message) 划分成的小数据块
- 分组交换机 Packet Switch
 - 路由器 Router
 - 链路层交换机 Link-layer Switch

1. 存储转发传输 Store-and-forward Transmission

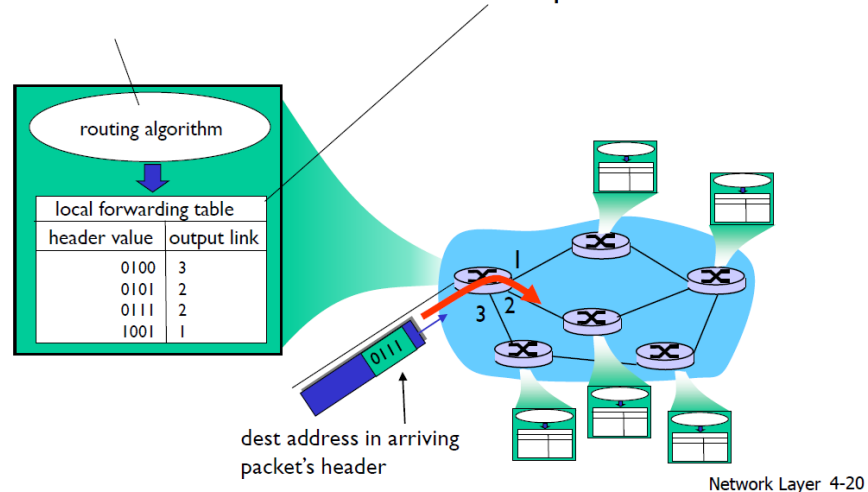
- 定义: 交换机能够开始向输出链路传输该分组的第一个 bit 之前, 必须接收到整个分组
- 传输时延: 通过由 N 条速率均为 R 的链路组成的路径, 从源到目的地的分组传输时延为

$$d_{\text{端到端}} = N \frac{L}{R}$$

2. 转发表和路由选择协议

routing: determines source-destination route taken by packets
▪ routing algorithms

forwarding: move packets from router's input to appropriate router output



1.2.2 电路交换 Circuit Switching

- 定义
在端系统间通信会话期间, 预留端系统之间沿路径通信所需要的资源 (缓存, 链路传输速率)
当两台主机要通信时, 该网络在两台主机之间创建一条专用的端对端连接 (end-to-end connection)

- 例子：电话网络

- 与分组交换对比

缺点：并行的时候带宽平分，而分组交换可以支持更多用户

优点：稳定，保证 performance，而分组交换在拥塞的时候会延迟和丢包

- 电路交换中的复用
 - 频分复用 Frequency-Division Multiplexing (FDM)
 - 时分复用 Time-Division Multiplexing (TDM)

1.2.3 因特网结构: 网络的网络

- Internet Service Provider (ISP)

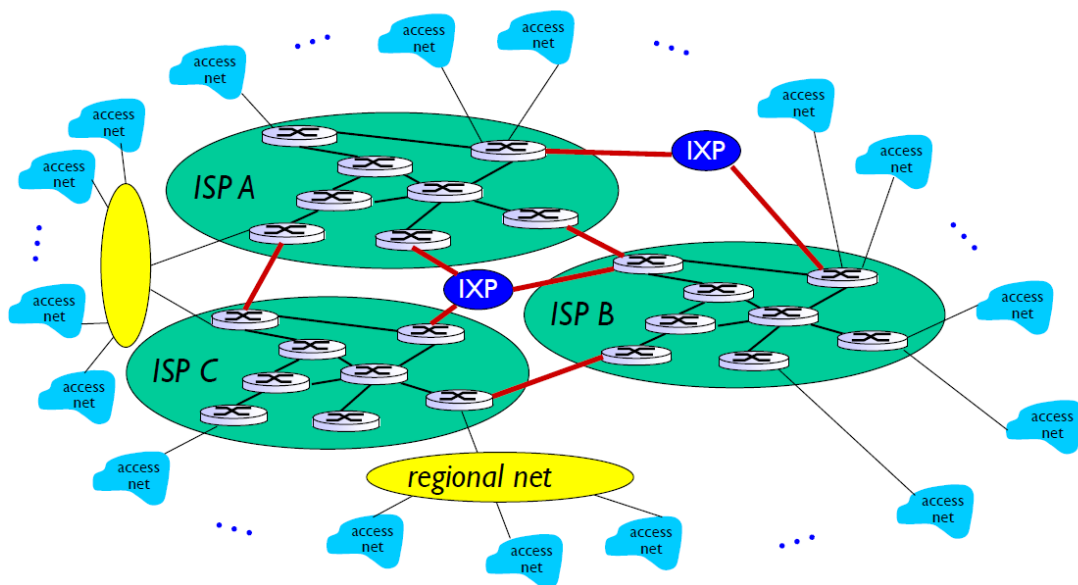
端系统通过 ISP 接入 Internet

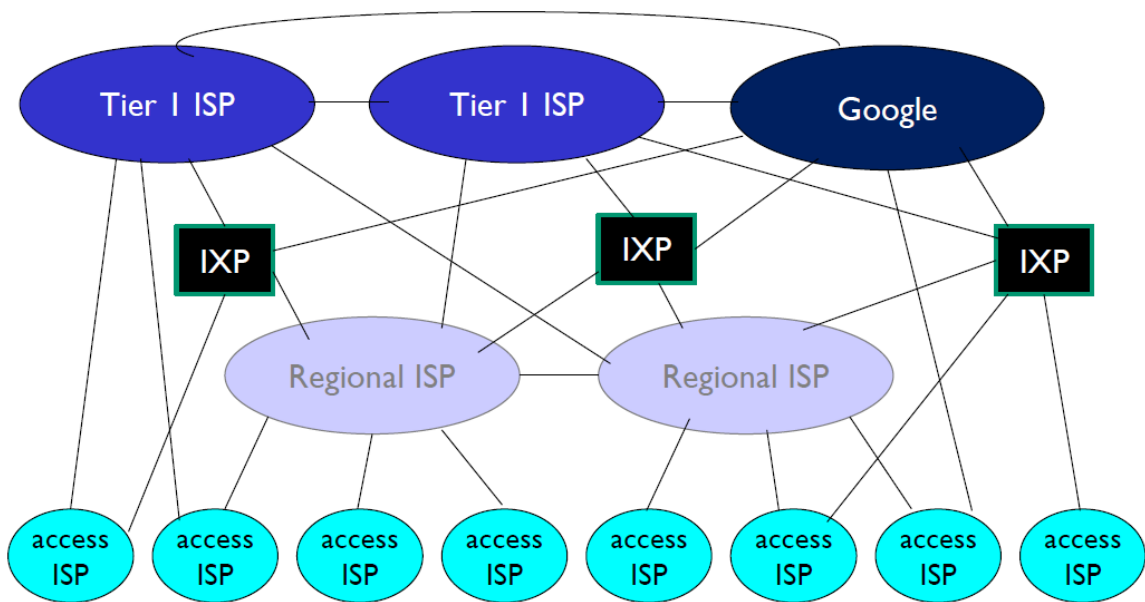
第一层 (tier-1) ISP 是最高层 ISP

- 因特网交换点 Internet Exchange Point (IXP)

汇合 ISP

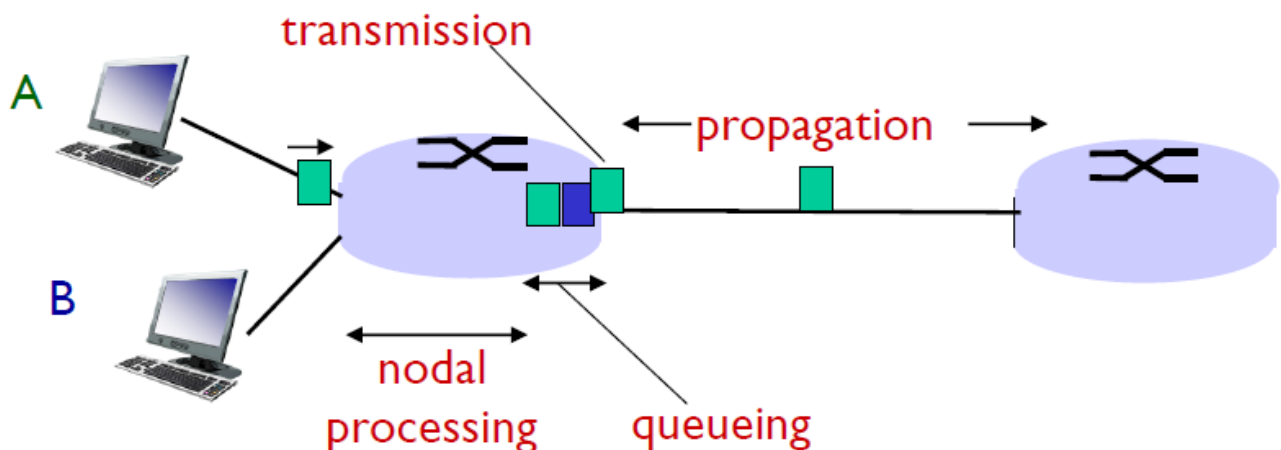
- 内容提供商网络 Content Provider Network





1.3 分组交换网中的时延、丢包和吞吐量

1.3.1 时延



$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

1. 节点处理时延 Nodal Processing Delay

分组到达路由器的時候，检查分组首部、决定分组去向、检查比特位错误等操作需要的时间

2. 排队时延 Queuing Delay

每台分组交换机与许多链路相连，对于每条链路有一个**输出缓存 (Output Buffer)** 或称**输出队列 (Output Queue)**，它用于存储路由器准备发往那条链路的分组。

排队时延即为分组在输出缓存中等待传输需要的时间

3. 传输时延 Transmission Delay

路由器将分组的所有比特推向链路所需要的时间

L : packet length (bits)

R : link bandwidth (bps)

$$d_{trans} = \frac{L}{R}$$

4. 传播时延 Propagation Delay

分组在链路上传播所需的时间

d : length of physical link

s : propagation speed in medium ($\sim 2 \times 10^8 m/s$)

$$d_{prop} = \frac{d}{s}$$

1.3.2 排队时延和丢包

- 流量强度 Traffic Intensity

a : 分组到达队列的平均速率 (pkt/s)

L : packet length (bits)

R : link bandwidth (bps)

$$I = \frac{La}{R}$$

若 $I > 1$, 则比特到达队列的平均速率大于从该队列传输出去的速率, 队列长度持续增加, 然后炸了

- 丢包 Packet Loss

输出队列已满, 当有新分组到达时, 该分组或已经排队的分组之一将被丢弃 (drop)

1.3.4 吞吐量 Throughput

- 定义

Rate (bits/time unit) at which bits transferred between sender/receiver.

指目的地接收的速率

- 瞬时吞吐量 Instantaneous Throughput

平均吞吐量 Average Throughput

- 瓶颈链路 Bottleneck Link

一段链路中传输速率最慢的, 决定了吞吐量

1.4 协议层次 Protocol Layers

- 协议 Protocol

Protocols define format, order of messages sent and received among network entities, and actions taken on msg transmission, receipt

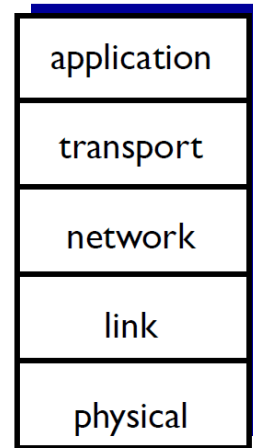
协议定义了两个或多个通信实体之间交换的报文的**格式和顺序**，以及报文发送、接收一条报文或其他事件所**采取的动作**

1.4.1 分层的体系结构

1. 因特网协议栈 Internet Protocol Stack

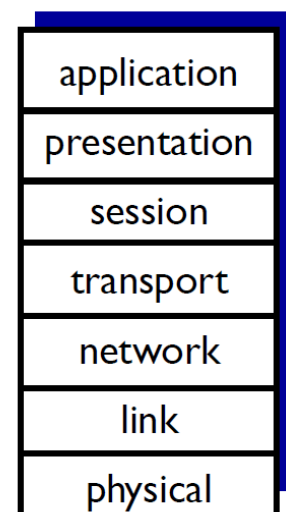
- ❖ **application**: supporting network applications
 - FTP, SMTP, HTTP
- ❖ **transport**: process-process data transfer
 - TCP, UDP
- ❖ **network**: routing of datagrams from source to destination
 - IP, routing protocols
- ❖ **link**: data transfer between neighboring network elements
 - Ethernet, 802.111 (WiFi), PPP
- ❖ **physical**: bits “on the wire”

- 应用层：报文 message
- 运输层：报文段 segment
- 网络层：数据报 datagram
- 链路层：帧 frame
- 物理层：比特 bit

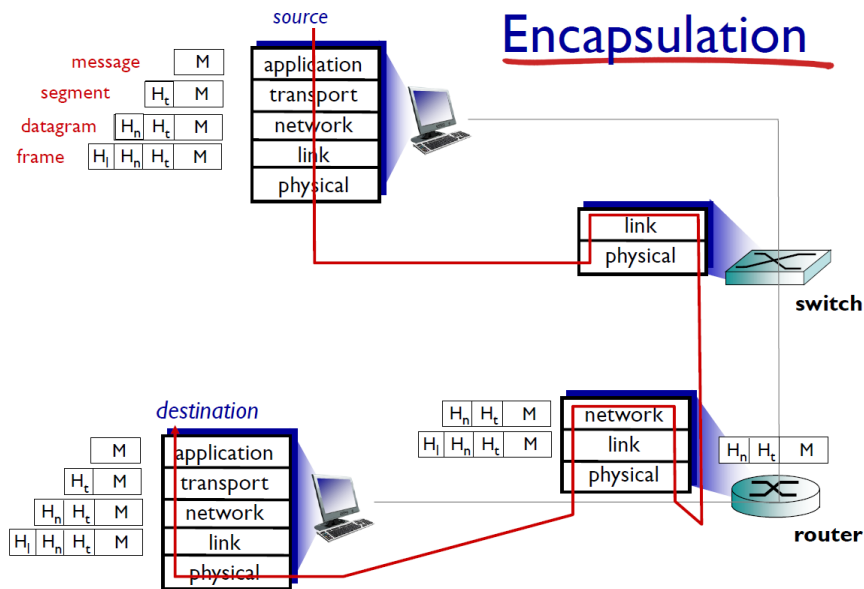


2. OSI 模型 ISO/OSI Reference Model

- ❖ **presentation**: allow applications to interpret meaning of data, e.g., encryption, compression, machine-specific conventions
- ❖ **session**: synchronization, checkpointing, recovery of data exchange
- ❖ Internet stack “missing” these layers!
 - these services, *if needed*, must be implemented in application
 - needed?



1.4.2 封装 Encapsulation



对于每一层，一个分组有两种类型的字段：

- 首部字段 Header
- 有效载荷字段 Payload Field

通常是来自上一层的分组

1.5 网络安全 Network Security

1. 恶意软件 Malware

- 病毒 Virus
需要某种形式的用户交互来感染用户设备的恶意软件
- 蠕虫 Worm
无需任何明显用户交互就能进入设备的恶意软件
- 自我复制 Self-replicating

2. 拒绝服务攻击 Denial-of-Service (DoS) Attack

分布式拒绝服务攻击 Distributed DoS (DDoS) Attack

3. 分组嗅探器 Packet Sniffer

4. IP 哄骗 IP Spoofing

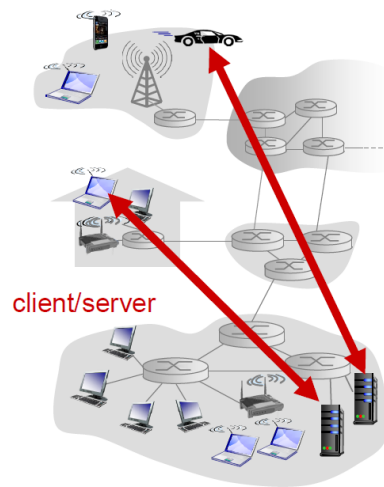
第二章 应用层 Application Layer

2.1 应用层协议原理 Principles of Network Applications

2.1.1 网络应用程序体系结构 Application Architectures

1. 客户-服务器体系结构 Client-server Architecture

Client-server architecture



server:

- always-on host
- permanent IP address
- data centers for scaling

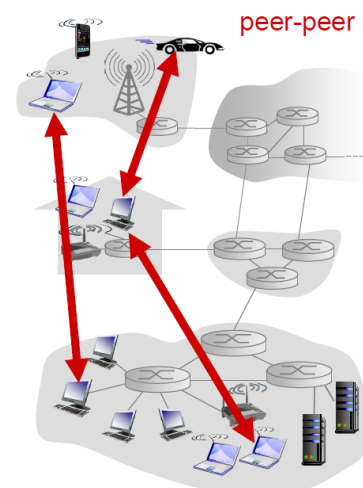
clients:

- communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do not communicate directly with each other

2. P2P 体系结构

P2P architecture

- no always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - self scalability – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
 - complex management



- 自扩展性 Self-scalability

2.1.2 进程通信

- 进程 Process

运行在端系统的一种程序，用于通信

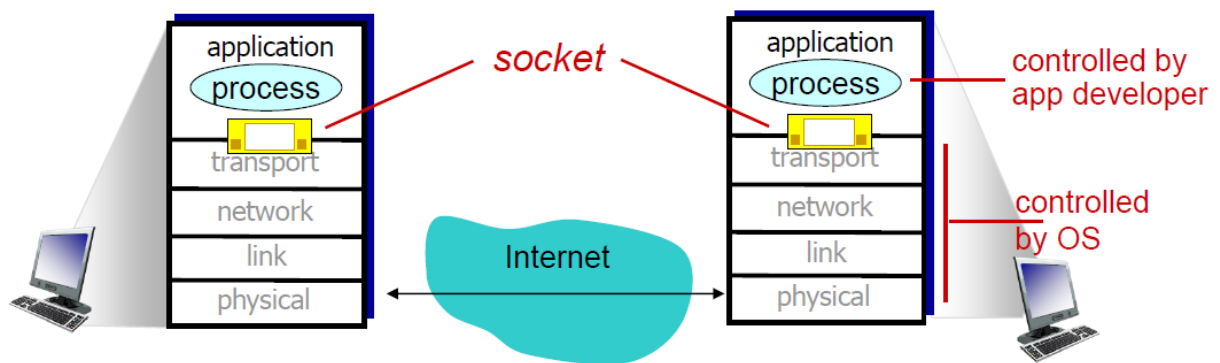
1. 客户和服务进程

在一对进程之间的通信会话场景中，发起通信的进程被标识为客户，在会话开始时等待联系的进程是服务器

先动手的是客户

2. 进程与计算机网络之间的接口: 套接字 Socket

- 进程通过一个称为套接字的软件接口向网络发送报文和从网络接收报文
- 又称应用程序编程接口 **Application Programming Interface (API)**



3. 进程寻址

- 端口号 Port Number

除了知道报文发送目的地的 IP 地址外，发送进程还必须指定运行在接收主机上的接收进程（具体说是接收套接字），因为一般而言一台主机能运行许多网络应用。目的地的**端口号 (Port Number)**用于这个目的。

- 常用端口号

Web 服务器: 80

邮件服务器 (使用 SMTP 协议): 25

2.1.3 可供应用程序使用的运输服务

1. 可靠数据传输与 Data Integrity

- 如果一个协议提供了确保数据完整以及正确性的数据交付服务，就认为提供了可靠数据传输 (reliable data transfer)
- 容忍丢失的应用 Loss-tolerant Application
能够承受一定量数据丢失的应用，如音视频应用
邮件，网页等就不能容忍

2. 吞吐量 Throughput

- 带宽敏感的应用 Bandwidth-sensitive Application
具有吞吐量要求的应用

- 弹性应用 Elastic Application

能根据当时可用的带宽弹性地利用可供使用的吞吐量

例：邮件，文件传输，Web 传送

3. 定时 Timing

有些应用要求低延迟，如网络游戏

	application	data loss	throughput	time sensitive
	file transfer	no loss	elastic	no
	e-mail	no loss	elastic	no
	Web documents	no loss	elastic	no
real-time	audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes, 100' s msec
stored	audio/video	loss-tolerant	same as above	
	interactive games	loss-tolerant	few kbps up	yes, few secs
	text messaging	no loss	elastic	yes, 100' s msec yes and no

4. 安全性 Security

运输协议能加密/解密发送进程传输的数据

2.1.4 因特网提供的运输服务

1. TCP 服务

- 面向连接的服务 Connection Oriented

在应用层数据报文开始流动之前，TCP要求客户和服务端互相交换运输层控制信息，称为握手。在握手阶段后，一个 **TCP 连接 (TCP Connection)** 就在两个进程的套接字之间建立了，这条连接是全双工的，即双方的进程可以在此连接上同时进行报文收发。

- 可靠的数据传送服务 Reliable Transportation
- 拥塞控制 Congestion Control

当发送方与接收方之间的网络出现拥塞时，TCP 会抑制发送进程

2. UDP 服务

- 无连接
 - 没有握手过程
- 不可靠的数据传送
- 没有拥塞控制
- 轻量级

	application	application layer protocol	underlying transport protocol
remote terminal access	e-mail	SMTP [RFC 2821]	TCP
	Web	HTTP [RFC 2616]	TCP
	file transfer	FTP [RFC 959]	TCP
streaming multimedia		HTTP (e.g., YouTube), RTP [RFC 1889]	TCP or UDP
Internet telephony		SIP, RTP, proprietary (e.g., Skype)	TCP or UDP

2.1.5 应用层协议 Application-layer Protocol

应用层协议定义了:

- 交换的报文类型，例如请求报文和响应报文
- 各种报文类型的语法
- 字段的语义
- 确定一个进程何时及如何发送报文，对报文进行响应的规则

2.2 Web and HTTP

2.2.1 HTTP 概况

- HTTP
超文本传输协议 Hypertext Transfer Protocol

- 使用 TCP 作为传输层协议
- 无状态协议 Stateless Protocol
HTTP 服务器不保存关于客户的任何信息

- Uniform Resource Locator (URL)

`protocol://hostname[:port]/path/[/;parameters][?query]#fragment`

<https://www.computerhope.com/jargon/u/url.htm>

Protocol Subdomain Domain and domain suffix Directories Web page

ComputerHope.com

2.2.2 非持续连接和持续连接

1. 非持续连接 Non-persistent Connection

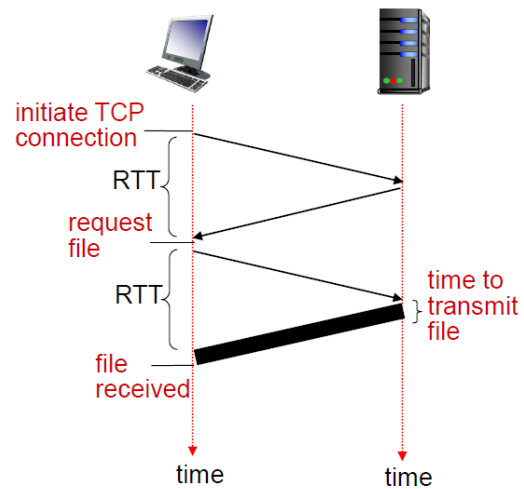
每个请求/响应对经一个单独的 TCP 连接发送

例: HTTP 1.0

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time:

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- file transmission time
- non-persistent HTTP response time = $2RTT + \text{file transmission time}$



2. 持续连接 Persistent Connection

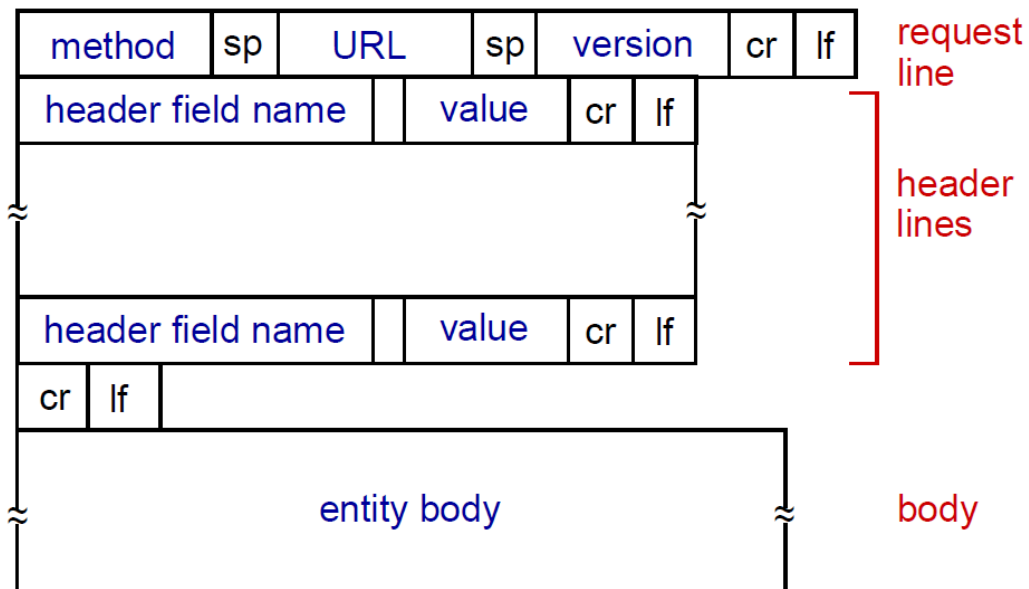
所有的请求及相应经相同的 TCP 连接发送

例: HTTP 1.1

2.2.3 HTTP 报文格式

1. HTTP 请求报文 GET, POST

- 请求行 Request Line
- 首部行 Header Line
- 空行
- 实体体 Entity Body



2. HTTP 响应报文

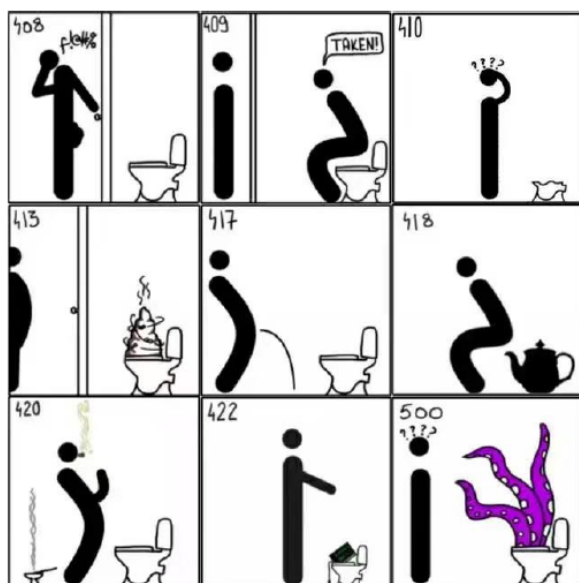
- 状态行
版本 sp 状态码 sp 短语 cr lf
- 首部行
- 空行

- 实体体

3. 状态码



- 301 Moved Permanently
- 302 Found
- 305 Use Proxy
- 307 Temporary Redirect
- 400 Bad Request
- 402 Payment Required
- 403 Forbidden
- 404 Not Found
- 406 Not Acceptable



- 408 Request Timeout
- 409 Conflict
- 410 Gone
- 413 Payload Too Large
- 417 Expectation Failed
- 418 I'm a teapot
- 420 Enhance Your Calm
- 422 Unprocessable Entity
- 500 Internal Server Error

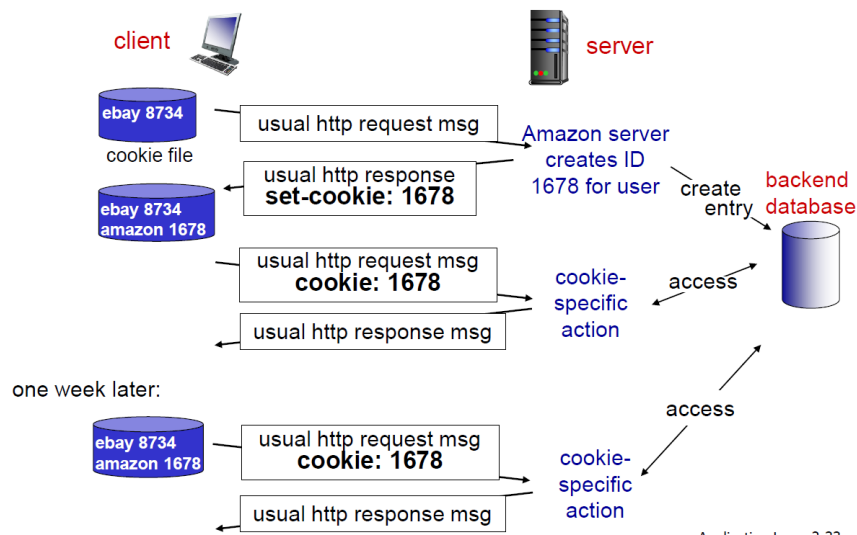
2.2.4 Cookie

1. 组成

- HTTP 响应报文中的一个 cookie 首部行
- HTTP 请求报文中的一个 cookie 首部行
- HTTP 在用户端系统保留有一个 cookie 文件，并由用户的浏览器进行管理
- 位于 Web 站点的一个后端数据库

2. 流程

Cookies: keeping “state” (cont.)

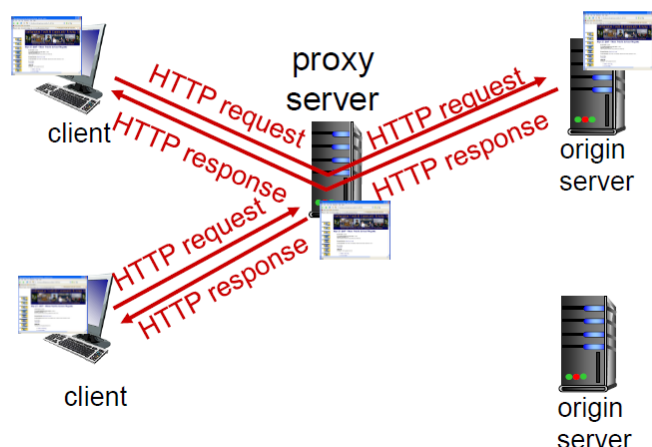


2.2.5 Web 缓存

- Web 缓存器 (Web Cache) 也叫代理服务器 (Proxy Server), 它是能够代表初始 Web 服务器来满足 HTTP 请求的网络实体
- Web Cache 的工作流程

goal: satisfy client request without involving origin server

- user sets browser: Web accesses via cache
- browser sends all HTTP requests to cache
 - object in cache: cache returns object
 - else cache requests object from origin server, then returns object to client

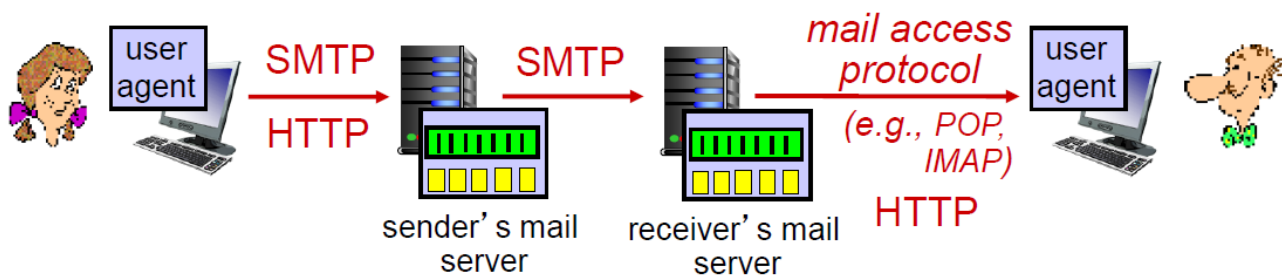


- Web Cache 的特点
 - 既是服务器又是客户
 - 减少客户请求的回应时间
 - 减少企业接入网的流量
 - 内容分发网络 Content Distribution Network (CDN), 方便 content provider 发布内容
 - 例: 一个 Local Web Cache 对企业接入网流量的影响
- 不想写了, 反正就是很有效

2.2.6 条件 GET

- 在 GET 请求中包含一个 `If-Modified-Since: <date>` 首部行
- 如果网页在这个时间之后更新过，服务器才会传回新网页，否则返回 `304 Not Modified`

2.3 电子邮件 E-Mail



2.3.1 电子邮件系统的组成

1. 用户代理 User Agent

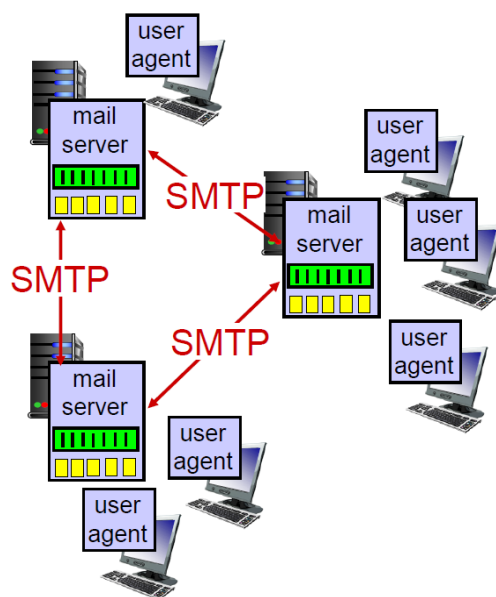
允许用户阅读、回复、转发、保存和撰写报文

2. 邮件服务器 Mail Server

每个接收方在邮件服务器上有一个邮箱 (Mailbox)

邮件服务器还有一个向外发送报文的邮件队列 (Message Queue), 如果邮件发不到目的地, 则暂时保存在邮件队列中

3. 简单邮件传输协议 Simple Mail Transfer Protocol (SMTP)



2.3.2 SMTP

- Uses TCP to reliably transfer email message from client to server, port 25
- Uses **persistent connections**
- Direct Transfer
sending server to receiving server
- Three phases of transfer
 - handshaking (greeting)
 - transfer of messages
 - closure
- Command/response interaction (like HTTP)
 - commands: ASCII text
 - response: status code and phrase
- Messages must be in 7 bit ASCII
- Example

```
1 S: 220 hamburger.edu
2 C: HELO crepes.fr
3 S: 250 Hello crepes.fr, pleased to meet you
4 C: MAIL FROM: <alice@crepes.fr>
5 S: 250 alice@crepes.fr... Sender ok
6 C: RCPT TO: <bob@hamburger.edu>
7 S: 250 bob@hamburger.edu ... Recipient ok
8 C: DATA
9 S: 354 Enter mail, end with "." on a line by itself
10 C: Do you like ketchup?
11 C: How about pickles?
12 C: .
13 S: 250 Message accepted for delivery
14 C: QUIT
15 S: 221 hamburger.edu closing connection
```

CRLF.CRLF 表示报文结束

2.3.3 邮件访问协议 Mail Access Protocol

1. POP3 Post Office Protocol

工作流程: download-and-keep

- 特许 Authorization
用户代理发送 (明文) 用户名和口令以鉴别用户
- 事务处理
用户代理取回报文, 还可以对报文做删除标记以及取消标记、获取邮件的统计信息
- 更新

客户发出 quit 命令之后，结束会话，邮件服务器删除被标记删除的报文

POP3 protocol

authorization phase

- client commands:
 - user: declare username
 - pass: password
- server responses
 - +OK
 - ERR

transaction phase, client:

- list: list message numbers
- retr: retrieve message by number
- dele: delete
- quit

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

2. IMAP Internet Mail Access Protocol

- 所有报文都存在服务器上

IMAP更高级一些

IMAP : 143 ,

POP3:110

POP3 是 download-and-delete 和 download-and-keep 模式

- 可以建文件夹
- 保持用户状态

POP3 是 stateless 的

2.4 DNS: 因特网的目录服务

2.4.1 DNS 提供的服务

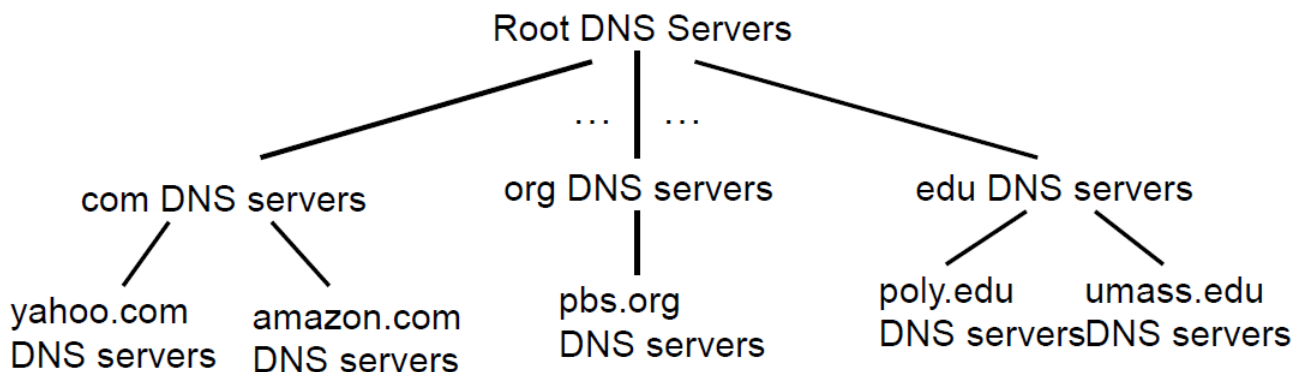
- 域名系统 Domain Name System (DNS) 提供的服务
 - 进行主机名到 IP 地址的转换
 - 主机别名 Host aliasing, 与之相对的是规范主机名 (Canonical hostname)
 - 邮件服务器别名 Mail server aliasing
 - 负载分配 Load distribution
- DNS 的构成
 - 一个由分层的 DNS 服务器实现的分布式数据库
 - 一个使得主机能够查询分布式数据库的应用层协议

2.4.2 DNS 的工作原理

2.4.2.1 分布式、层次数据库

- 结构
 - 根 DNS 服务器 Root DNS Server
 - 顶级域服务器 TLD (Top Level Domain) DNS Server
 - 权威 DNS 服务器 Authoritative DNS Server

- 本地 DNS 服务器 Local DNS Server (或称 Default name server)



- 两种访问方式

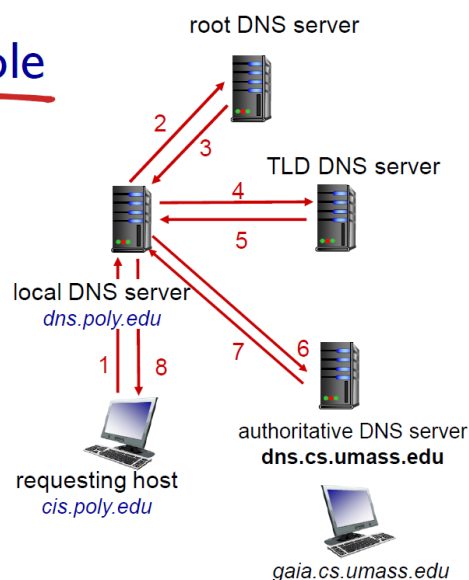
1. 迭代查询 Iterated Query

DNS name resolution example

- host at cis.poly.edu wants IP address for gaia.cs.umass.edu

iterated query:

- contacted server replies with name of server to contact
- "I don't know this name, but ask this server"

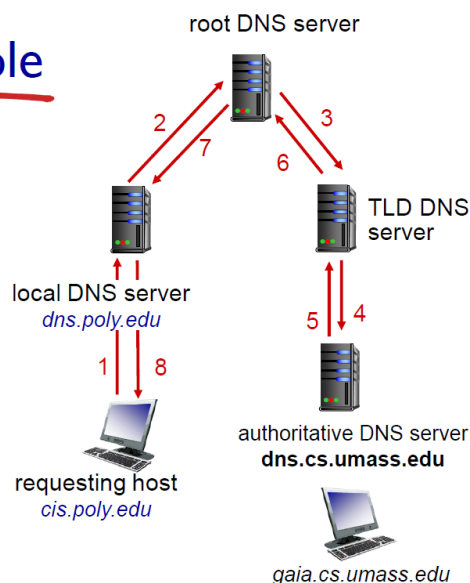


2. 递归查询 Recursive Query

DNS name resolution example

recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



所以 Local DNS 炸了就凉了

2.4.2.2 DNS 缓存

- Cache 的条目每隔一段时间就会被丢弃, 因为主机名和 IP 的对应是可变的
- 大部分情况下都能利用 cache 免去访问根 DNS 服务器

2.4.3 DNS 记录和报文

1. 资源记录 Resource Record (RR)

(Name, Value, Type, TTL)

◦ Type=A

Name 是主机名, Value 是对应 IP

例: (relay1.bar.foo.com, 145.37.93.126, A)

◦ Type=NS

Name 是个域, Value 是知道如何获取该域的 IP 的权威 DNS 服务器的主机名

例: (foo.com, dns.foo.com, NS)

◦ Type=CNAME

Value 是个规范主机名, 别名为 Name

例: (foo.com, relay1.bar.foo.com, CNAME)

◦ Type=MX

Value 是个邮件服务器的规范主机名, 别名为 Name

例: (foo.com, mail.bar.foo.com, MX)

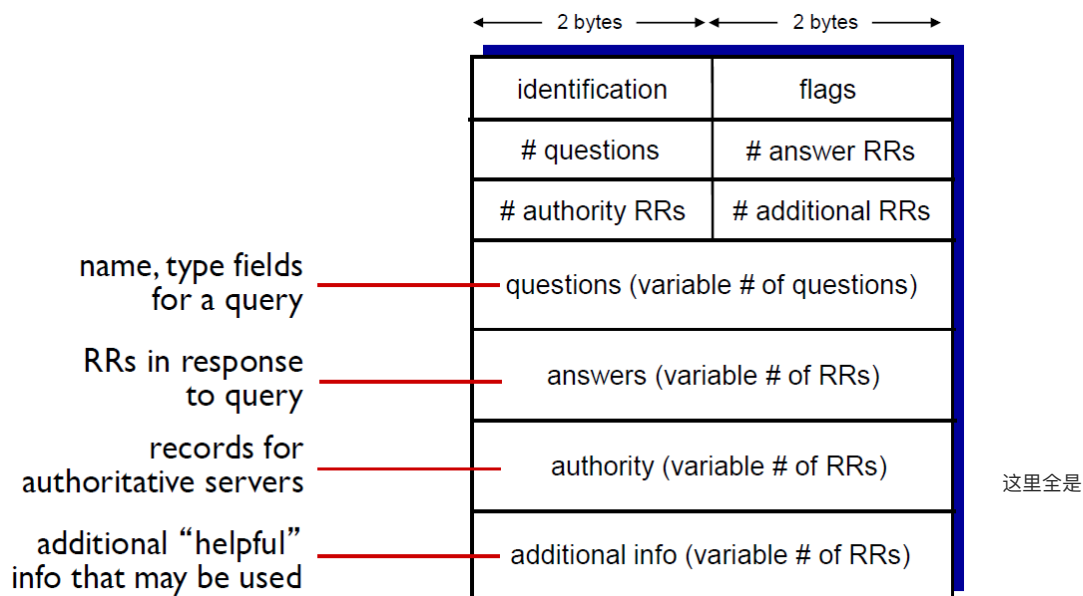
2. DNS 报文

- *query* and *reply* messages, both with same *message format*

message header

- **identification**: 16 bit # for query, reply to query uses same #
- **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative

← 2 bytes →		← 2 bytes →	
identification		flags	
# questions		# answer RRs	
# authority RRs		# additional RRs	
questions (variable # of questions)			
answers (variable # of RRs)			
authority (variable # of RRs)			
additional info (variable # of RRs)			



3. 在 DNS 数据库中插入记录

1. 找一个注册登记机构 (Registrar)
2. 让他向 TLD DNS 服务器插两条记录

```
1 | (xxx.com, dns.xxx.com, NS)
2 | (dns.xxx.com, ###.###.###.###, A)
```

3. 当你要访问 xxx.com 时

- 通过这条 NS 类型的记录查到管事的 DNS 服务器的主机名
- 通过查 A 类型的记录找到这个 DNS 服务器的 IP
- 根据这个 IP 找上门去问这个 DNS xxx.com 的 IP 是啥
- DNS 给你一个 CNAME 类型的记录，告诉你 xxx.com 的规范主机名
- 查 A 类型的记录找到这个规范主机名的 IP
- 马上就到你家门口

2.5 P2P

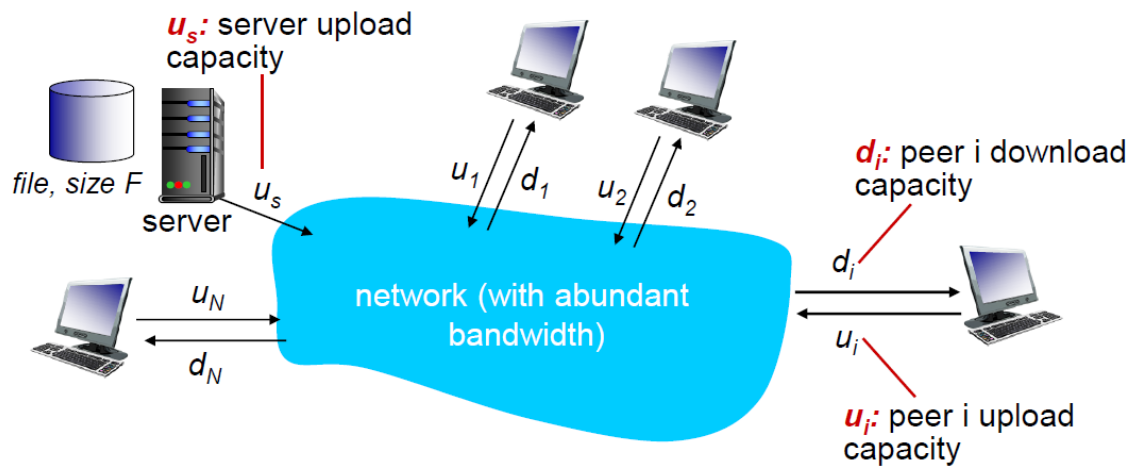
2.5.1 P2P 体系结构的扩展性

P2P 结构见 2.1.1

- 分发时间 Distribution Time

把大小为 F 的文件分发给 N 个对等方

- client - server



服务器: 连续上传 N 份文件, 上传速度 u_s , 花费时间 $\frac{NF}{u_s}$

d_{min} : 客户的最低下载速度, 花费时间 $\frac{F}{d_{min}}$

总时间 $D_{cs} \geq \max\{\frac{NF}{u_s}, \frac{F}{d_{min}}\}$

o P2P

每个 peer 都有上传能力

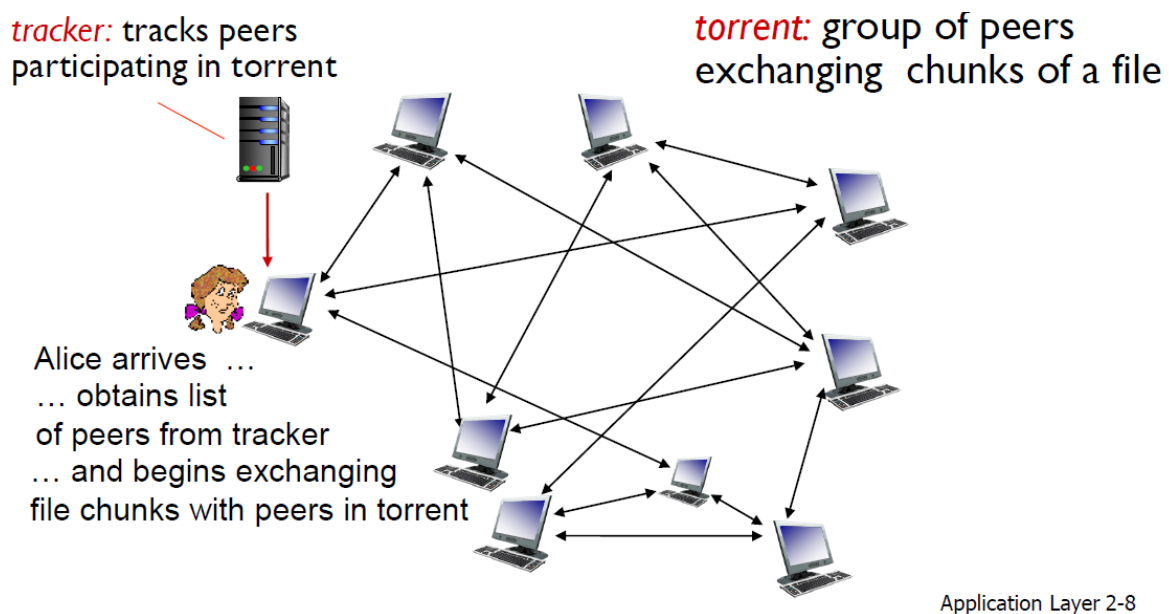
服务器: 只上传 1 份文件, 上传速度 u_s , 花费时间 $\frac{F}{u_s}$

d_{min} : 客户的最低下载速度, 花费时间 $\frac{F}{d_{min}}$

总上传速率 $u_s + u_1 + u_2 + \dots + u_N$

总时间 $D_{P2P} \geq \max\{\frac{F}{u_s}, \frac{F}{d_{min}}, \frac{NF}{u_s + \sum_{i=1}^N u_i}\}$

2.5.2 BitTorrent



• 定义

参与一个特定文件的分发的所有对等方的集合被称为一个**洪流 (torrent)**。在一个洪流中对等方彼此下载等长度的**块 (chunk)**, 典型的块长度为 256KB。

每个洪流有一个基础设施节点, 称为**追踪器 (tracker)**。当一个对等方加入洪流时, 它向追踪器注册自己, 并周期性地通知追踪器它仍在该洪流中。

- 最稀缺优先 Rarest First

对于一个 peer，针对他没有的块在他的邻居中决定最稀缺的块（在邻居中副本数量最少的块），并首先请求那些最稀缺的块

- 一报还一报 Tit-for-tat

对每个邻居都持续测量接收到比特的速率，并确定以最高速率流入的 4 个邻居。每过 10 秒，重新计算该速率并可能修改这 4 个对等方的集合，这 4 个对等方被称为**疏通 (unchoked)**。每过 30 秒，也要随机选择一位邻居并向其发送块。所以结果是速率匹配的 peer 成为互相的疏通。

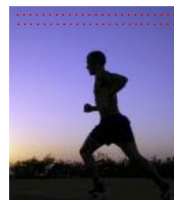
2.6 视频流与 CDN

2.6.1 因特网视频 Video Streaming

Multimedia: video

- video: sequence of images displayed at constant rate
 - e.g., 24 images/sec
- digital image: array of pixels
 - each pixel represented by bits
- coding: use redundancy *within* and *between* images to decrease # bits used to encode image
 - spatial (within image)
 - temporal (from one image to next)

spatial coding example: instead of sending N values of same color (all purple), send only two values: color value (purple) and number of repeated values (N)



frame i



frame $i+1$

temporal coding example: instead of sending complete frame at $i+1$, send only differences from frame i

- 视频编码
 - CBR Constant Bit Rate
 - VBR Variable Bit Rate
 - Example
 - MPEG 1 (CD-ROM): 1.5 Mbps
 - MPEG 2 (DVD): 3~6 Mbps
 - MPEG 4: <1 Mbps

2.6.2 DASH

- 经 HTTP 的动态适应流 Dynamic Adaptive Streaming over HTTP

视频编码为几个不同的版本，每个版本比特率不同，对应不同的质量水平。客户动态地请求来自不同版本且长度为几秒的视频段数据块。当可用带宽量较高时，用户请求来自高速率版本的块，可用带宽量较低时请求低速率版本的块。每次使用一个 HTTP GET 报文请求一个块。

HTTP 服务器提供一个告示文件 (**manifest file**), 为每个版本提供一个 URL 及其比特率。

- "Intelligence" at client: client determines
 - when to request chunk
 - what encoding rate to request
 - where to request chunk

2.6.3 内容分发网 CDN

- CDN 服务器的安置原则

1. 深入 Enter Deep

通过在遍及全球的接入 ISP 中部署服务器集群来深入到 ISP 的接入网中。其目标是靠近端用户, 减少端用户和 CDN 集群之间的链路和路由器的数量, 从而减小时延

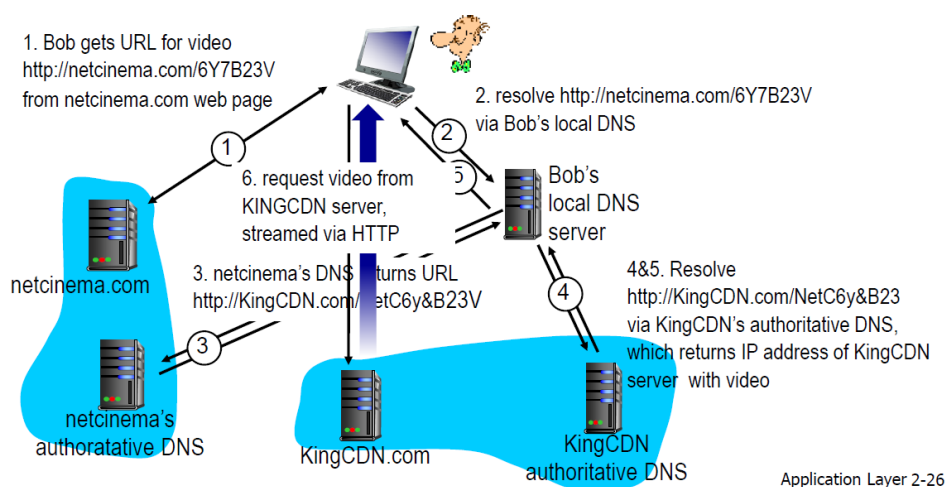
2. 邀请做客 Bring Home

通过在少量 (例如 10 个) 关键位置建造大集群来邀请到 ISP 做客。通常将集群放置在因特网交换点 (IXP)

- CDN 操作流程

Bob (client) requests video `http://netcinema.com/6Y7B23V`

- video stored in CDN at `http://KingCDN.com/NetC6y&B23V`



2.7 Socket 编程

1. UDP

server (running on serverIP)

create socket, port= x:
`serverSocket =
socket(AF_INET,SOCK_DGRAM)`

read datagram from
`serverSocket`

write reply to
`serverSocket`
specifying
client address,
port number

client

create socket:
`clientSocket =
socket(AF_INET,SOCK_DGRAM)`

Create datagram with server IP and
port=x; send datagram via
`clientSocket`

read datagram from
`clientSocket`

close
`clientSocket`

2. TCP

server (running on hostid)

create socket,
port=x, for incoming
request:
`serverSocket = socket()`

wait for incoming
connection request
`connectionSocket =
serverSocket.accept()`

read request from
`connectionSocket`

write reply to
`connectionSocket`

close
`connectionSocket`

client

create socket,
connect to hostid, port=x
`clientSocket = socket()`

send request using
`clientSocket`

read reply from
`clientSocket`

close
`clientSocket`

TCP
connection setup

第三章 运输层 Transport Layer

3.1 运输层服务 Transport Layer Services

1. 服务

运输层协议为运行在不同主机上的应用进程之间提供逻辑通信 (logic communication) 功能

2. 报文段 Segment

3.2 多路复用与多路分解 Multiplexing & Demultiplexing

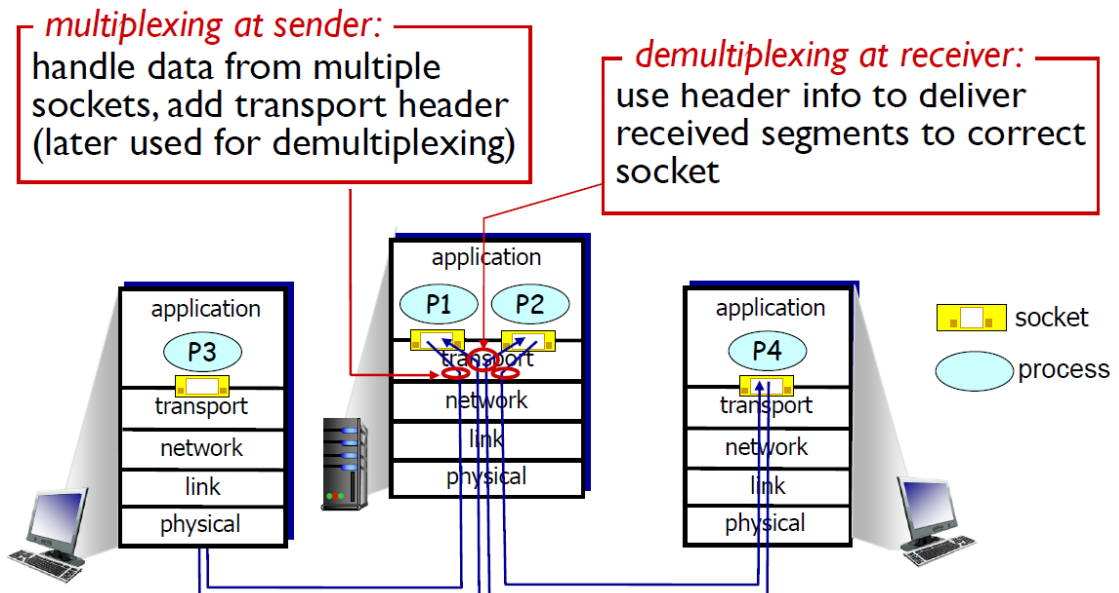
1. 定义

◦ 多路分解 Demultiplexing

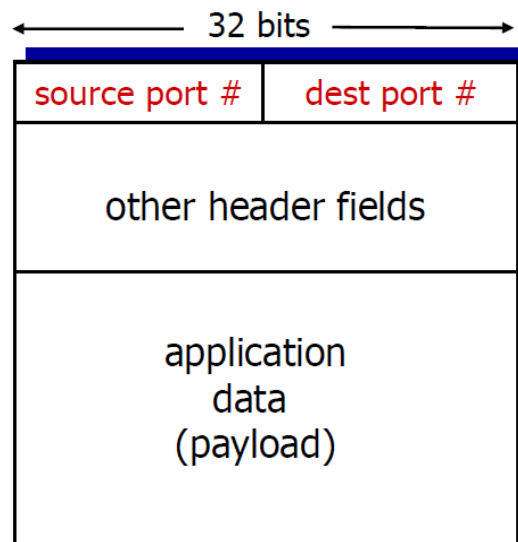
将运输层报文段中的数据交付到正确的套接字的过程

◦ 多路复用 Multiplexing

在源主机从不同套接字中收集数据块，并为每个数据块封装上首部信息从而生成 **报文段 (segment)**，然后将报文段传递到网络层，这些过程称为多路复用



2. 报文段结构



TCP/UDP segment format

3. 周知端口号 Well-known port number

0 ~ 1023 范围内的端口号是受限制的，保留给某些应用层协议
端口号范围是 0 ~ 65535 (16 bit)

3.2.1 无连接的多路复用与多路分解

- 多路复用

客户的运输层创建报文段，在首部加入源端口号和目的端口号。网络层将报文段封装到一个 IP 数据报中，交付给接收主机

- 多路分解

接收主机运输层检查目的端口号，交付给对应套接字

- 缺点

一个 UDP 套接字是由一个二元组 (`dst IP`, `dst port`) 标识的，所以来自不同 IP、不同端口的 UDP 报文段会被定向到相同的进程

3.2.2 面向连接的多路复用与多路分解

- TCP 报文段的多路分解

TCP 套接字是由一个四元组 (`src IP`, `src port`, `dst IP`, `dst port`) 标识的。因此，接收端运输层使用全部 4 个值来将报文段定向到相应的套接字

3.3 UDP

3.3.1 UDP 的特性

- 无连接

发送方与接收方之间没有握手阶段 (handshaking)

每个 UDP 报文段被单独处理

- 优点
 - 无连接建立
 - 无连接状态
 - 关于发送什么数据以及何时发送的应用层控制更加精细
 - 分组首部开销小
 - 没有拥塞控制，发送快
- 缺点
 - 可能丢失
 - 可能不按顺序到达
- 应用
 - 多媒体流
 - DNS
 - SNMP 简单网络管理协议

3.3.2 UDP 检验和

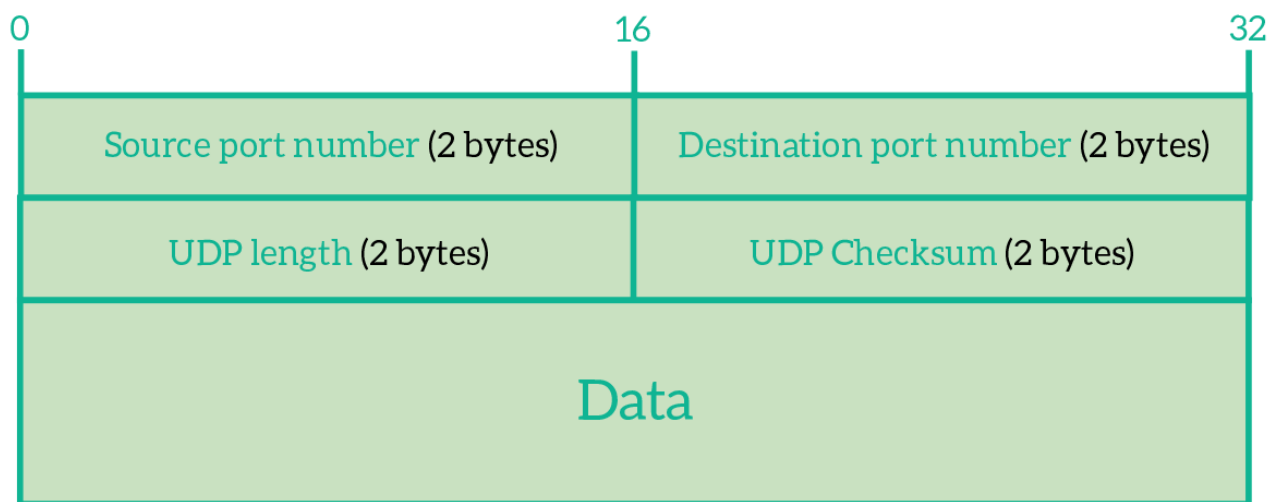
example: add two 16-bit integers

		1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
		1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum		1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum		0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

例如发送方要发送两个 16 bit 字，在发送之前，把这俩数相加，注意进位的 1 要加回到右边。加完之后取反码。反码在 checksum 字段一起发给接收端，接收端收到之后把这三个数加在一起，结果应该是全 1，哪里是 0 哪一位就有错。 n 个数同理。

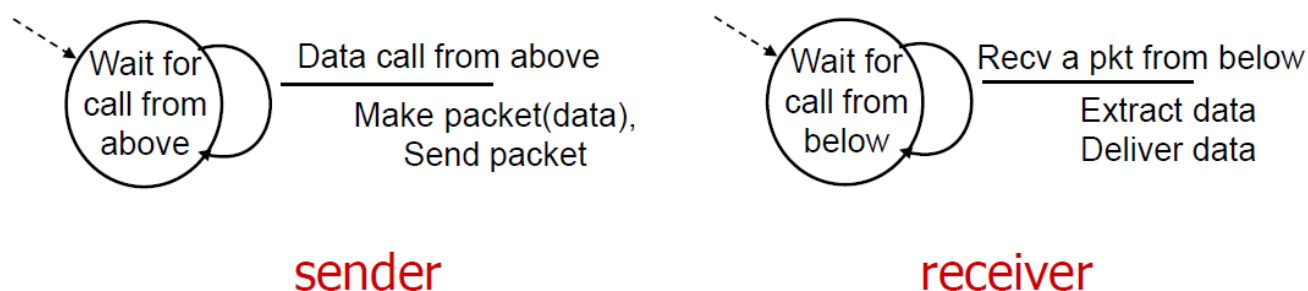
UDP 报文段结构：



3.4 可靠数据传输 Reliable Data Transfer

3.4.1 构造可靠数据传输协议

3.4.1.1 经完全可靠信道的可靠数据传输：rdt 1.0

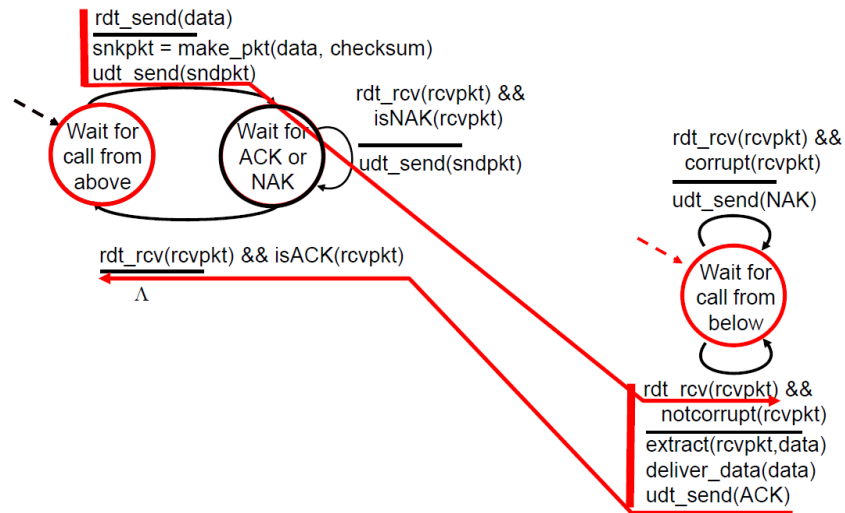


3.4.1.2 经具有比特差错信道的可靠传输策略：rdt 2.0

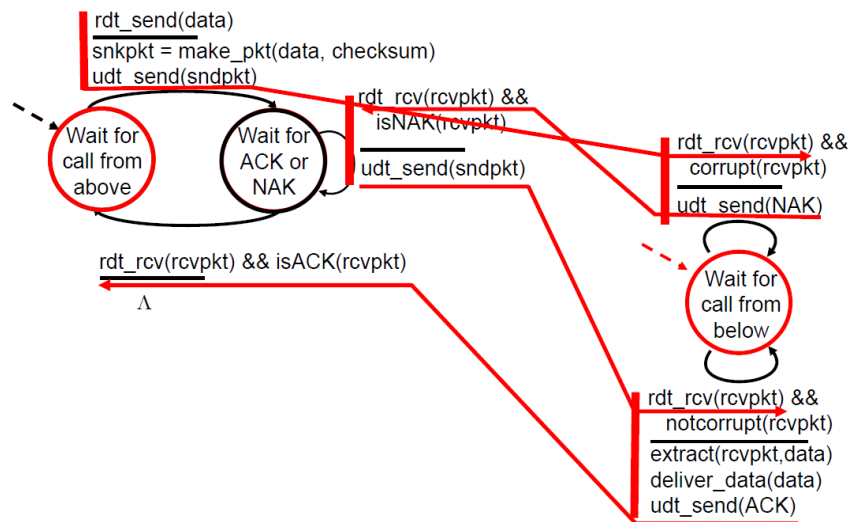
- 自动重传协议 ARQ (Automatic Repeat Request)
 - 差错检测
 - 接收方反馈
 - 肯定确认 ACK positive acknowledgement
 - 否定确认 NAK negative acknowledgement
 - 只需 1 个 bit, ACK=1, NAK=0
 - 重传
- 停等协议 Stop-and-wait Protocol
 - 当发送方处于等待 ACK 或 NAK 的状态时，它不能从上层获得更多数据
 - 发送方将不会发送一块新数据，除非发送方确认接收方已正确接收当前分组
- 缺点

没有考虑 ACK 和 NAK 受损的情况

rdt2.0: operation with no errors



rdt2.0: error scenario

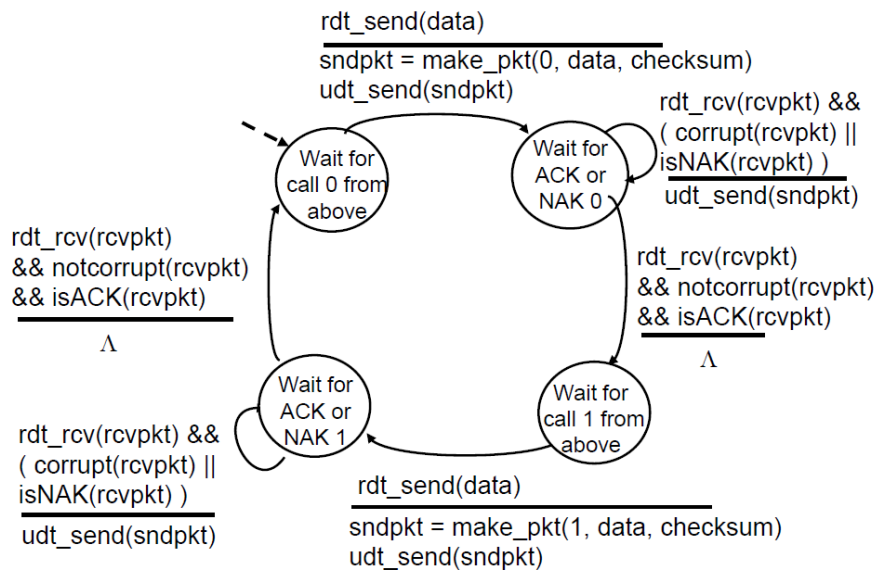


3.4.1.3 rdt 2.1

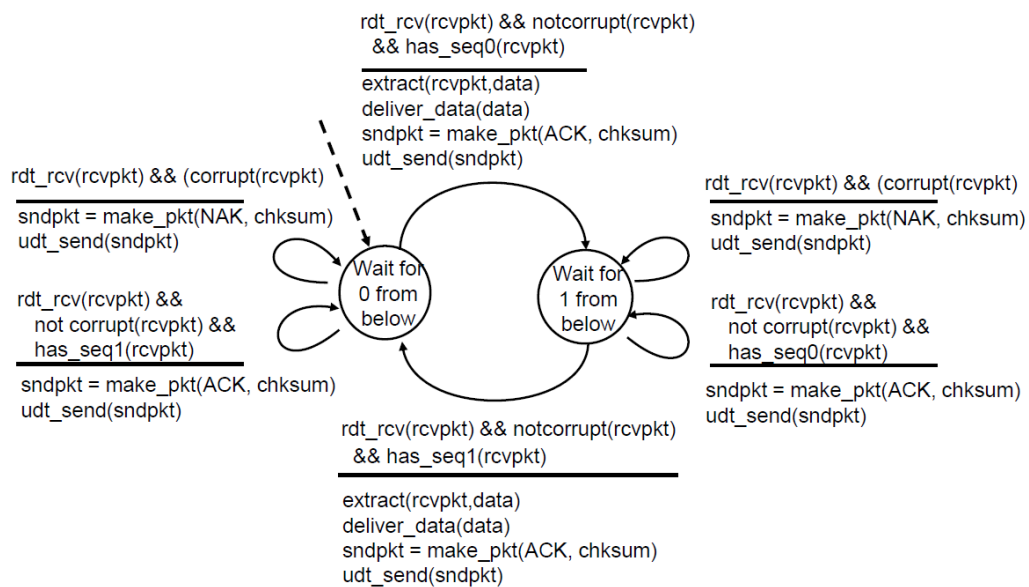
- 序号 Sequence Number

在发送的包上加一个 1 bit 的序号，用模 2 运算前向移动，这样接收方就可以知道是否为一次重传

Sender



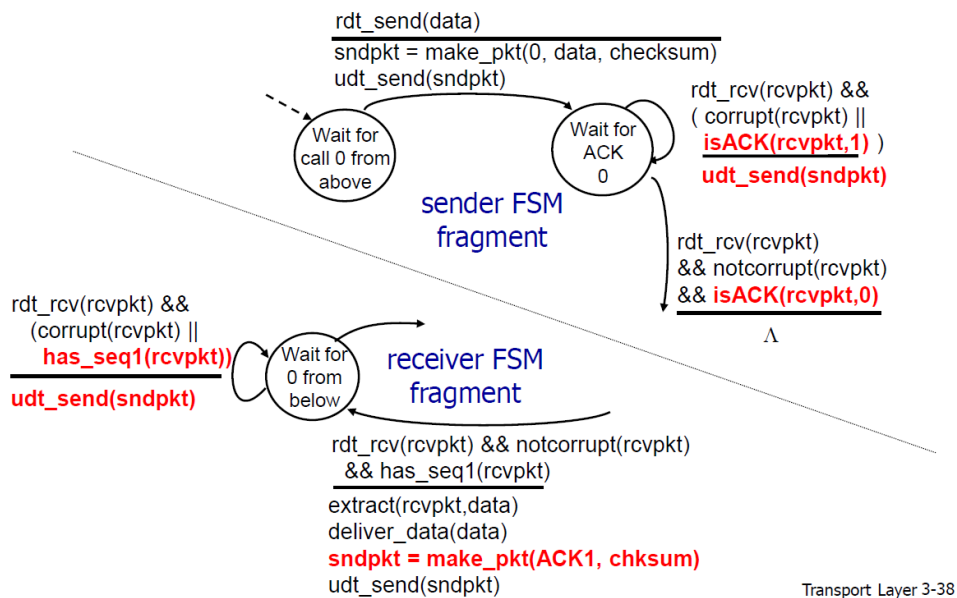
Receiver



3.4.1.4 rdt 2.2

没有 NAK 了

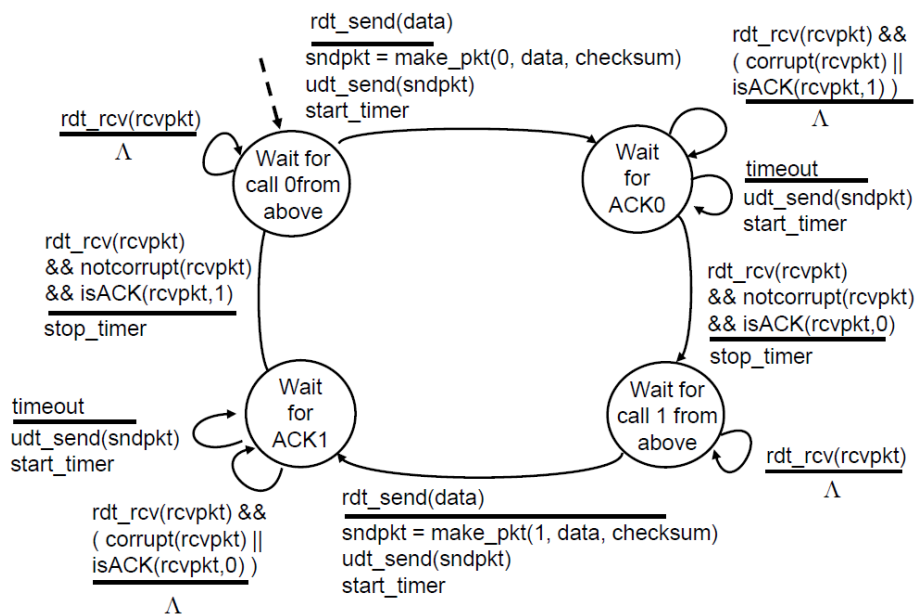
发送方收到冗余 ACK 就知道后面的没被收到



3.4.1.5 经具有比特差错的丢包信道的可靠数据传输：rdt 3.0

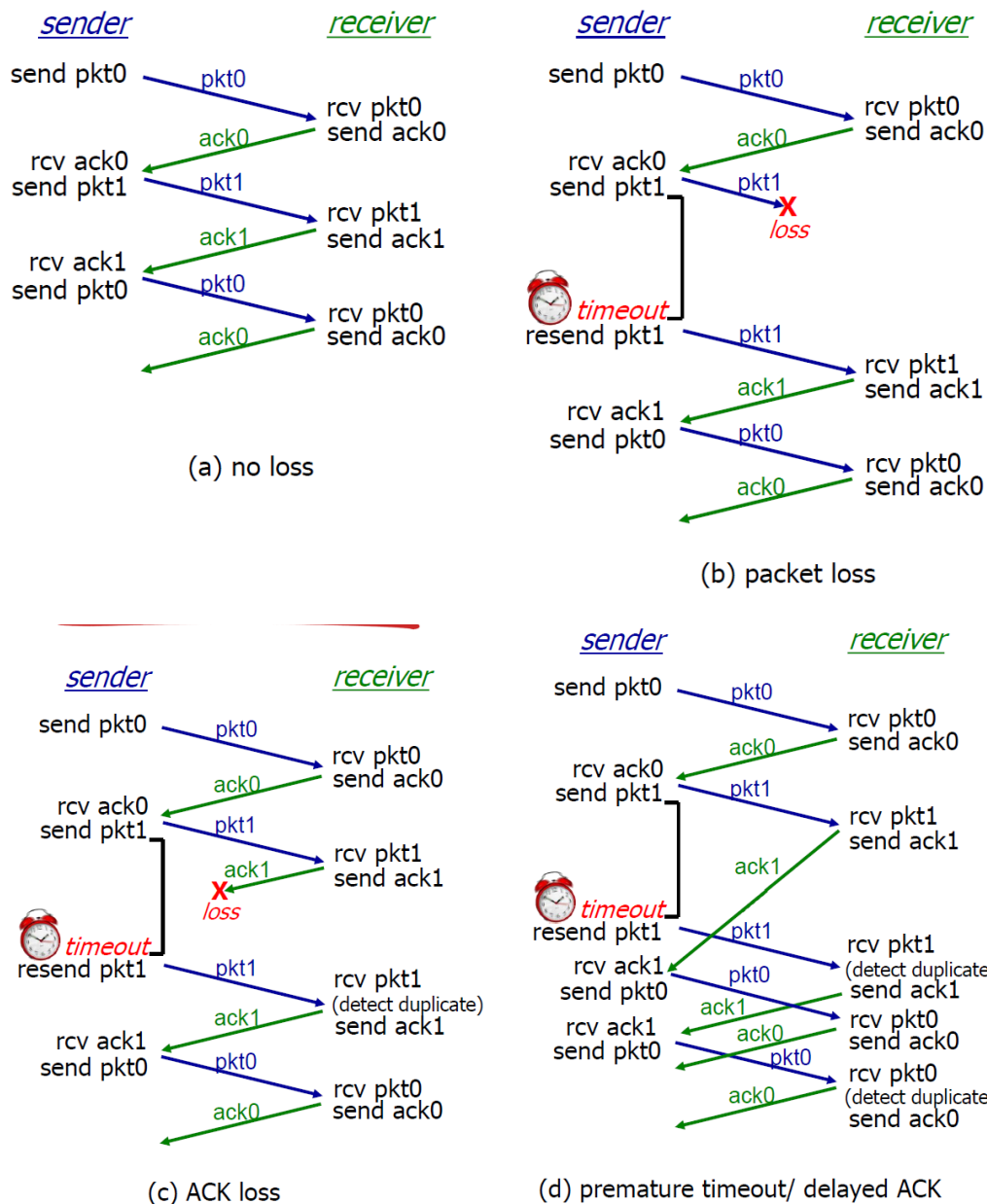
- 倒计时定时器 Countdown Timer（发送方）
 - 每次发送一个分组（包括重传的）就启动一个定时器
 - 响应定时器中断
 - 终止定时器

Sender



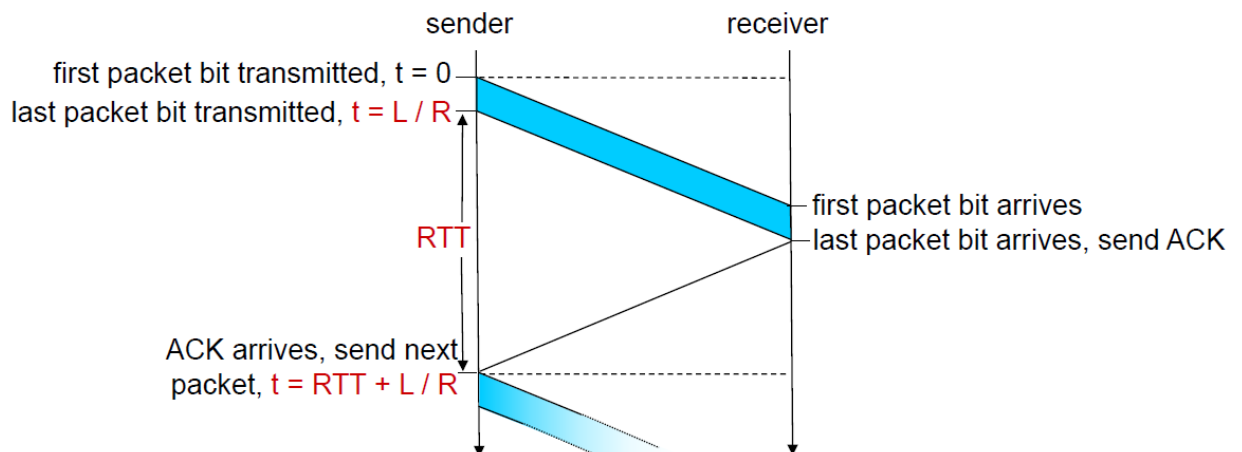
注意：只在 timeout 的时候才重发

- 比特交替协议 Alternating-bit Protocol
 - 分组序号在 0 和 1 之间交替



3.4.2 流水线可靠数据传输协议

- 利用率 Utilization



$$U = \frac{\text{传输用时}}{\text{总用时}} = \frac{\frac{L}{R}}{RTT + \frac{L}{R}}$$

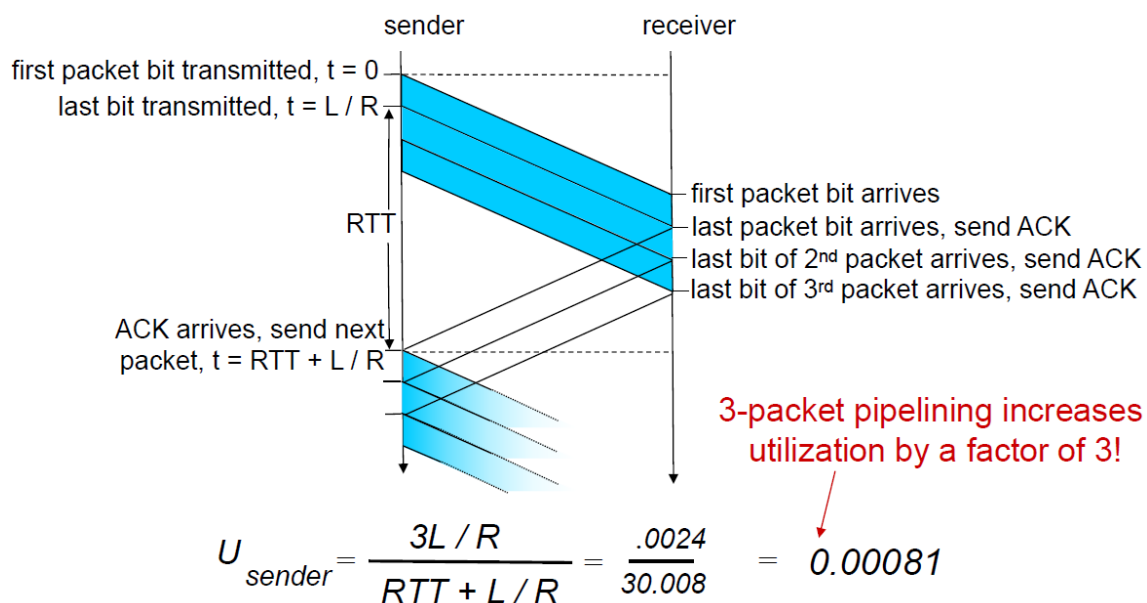
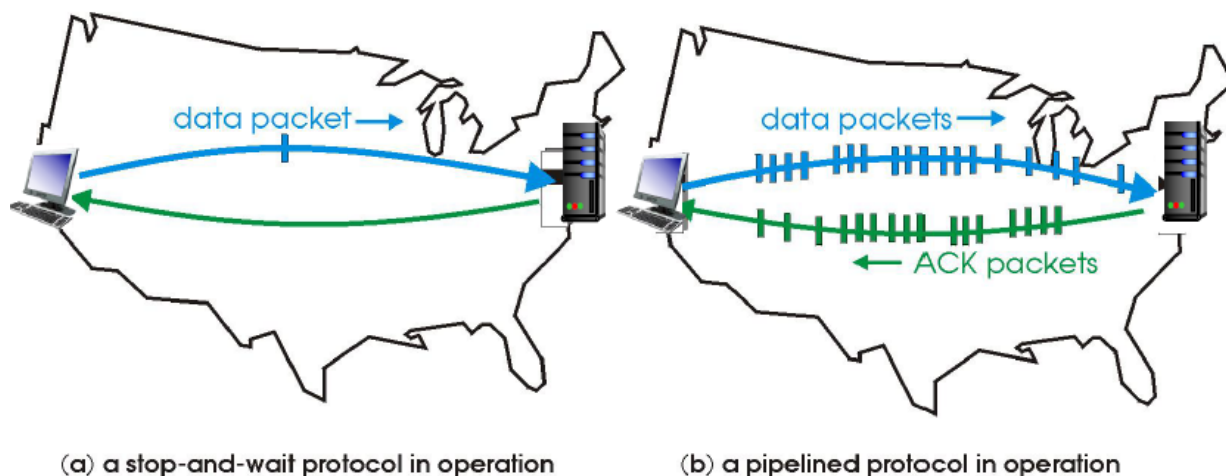
当 RTT 远大于发送时间时，利用率将会非常低（真正干活的时间很少，其他时间都在等链路传播）

注意 RTT 的定义：发送方最后一个 bit 发到链路上到收到 ACK 之间的时间

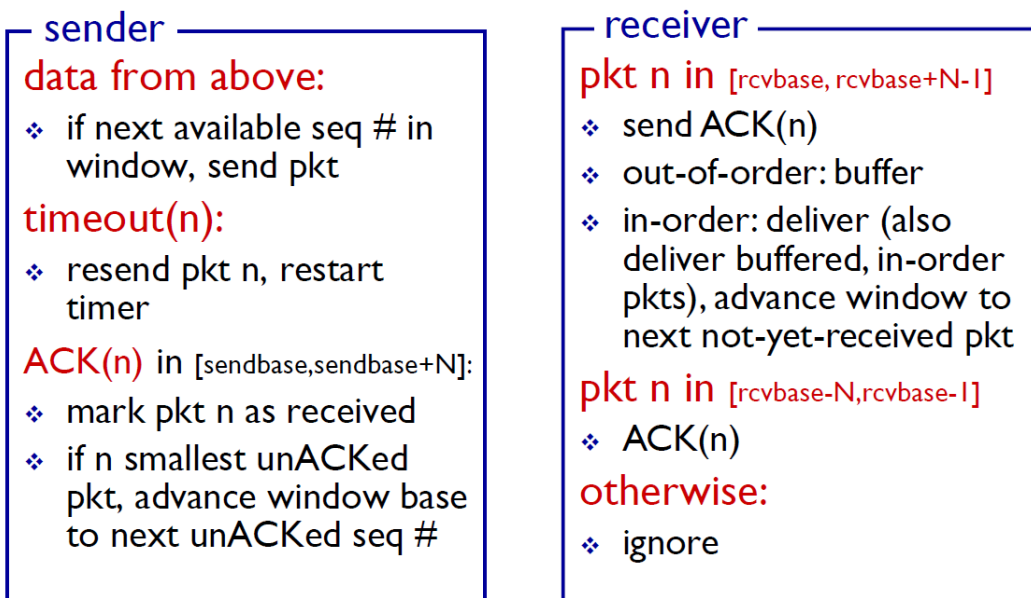
- 流水线 Pipelining

同时有多个包在链路上传播

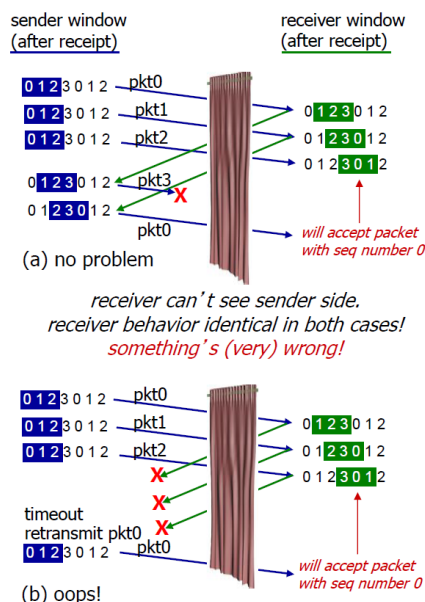
n 级流水线会让利用率提高 n 倍



3.4.3 回退 N 步 Go-Back-N



- 接收方可能会 ACK 一个窗口之前的包 (已经正确收到的包)，是因为之前的 ACK loss 了
- 接收方窗口的 base 还是第一个已发但没收到 ACK 的包，但后面可能有收到 ACK 的，所以重传补齐之后 base 可能向右滑好几个
- 接收方窗口的 `rcv_base` 表示第一个还没收到的包，同理，后面可能有已经收到并缓存的
- Sequence Number 都是 **重复使用的**，比如之前那个 01 比特交替
所以窗口长度必须小于等于序号空间大小的一半

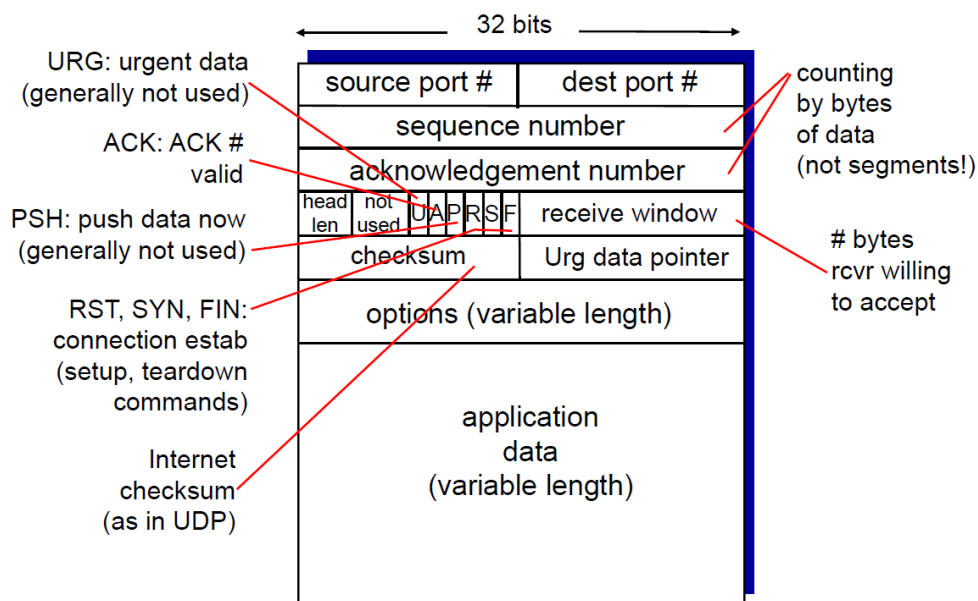


3.5 面向连接的运输: TCP

3.5.1 TCP 连接

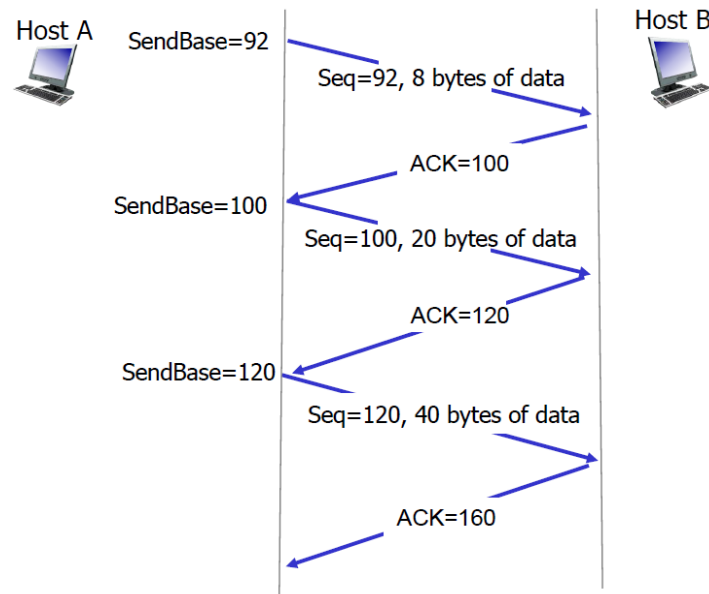
- 面向连接的 Connection-oriented
三次握手 Three-way Handshake
- 全双工服务 Full-duplex Service
一个连接实现双向数据传输
- 点对点 Point-to-point
- 发送缓存 Send Buffer
- 最大报文段长度 MSS (Maximum Segment Size)
TCP 可从缓存中取出并放入报文段中的数据数量
- 最大传输单元 MTU (Maximum Transmission Unit)
由本地发送主机发送的最大链路层帧长度
- TCP 报文段 TCP Segment
TCP 为每块客户数据封装上 TCP 首部，下传到网络层

3.5.2 TCP 报文段



- 16 bit 的源端口号和目的端口号
- 32 bit 的序号字段 Sequence Number Field
- 32 bit 的确认号字段 Acknowledgment Number Field
- 16 bit 的接收窗口字段 Receive Window Field
- 4 bit 的首部长度字段 Header Length Field
- 可选与变长的选项字段 Option Field
- 6 bit 的标志字段 Flag Field

3.5.3 TCP 序号和确认号



- Seq: 从哪一位开始发
- Len: 数据长度 (bytes)
- ACK=n: 0 ~ n-1 都收到了, 你该从第 n 个开始发了 (累积确认 Cumulative ACK)
- ACK=Seq+Len
- Syn 和 Fin 会占用一个编号, 虽然发这两条的时候 Len=0, 但是接收方的 ACK 会 +1

3.5.4 往返时间的估计与超时

- 指数加权移动平均 EWMA (Exponential Weighted Moving Average)

$$EstimatedRTT = (1 - \alpha)EstimatedRTT + \alpha SampleRTT$$

一般取 $\alpha = \frac{1}{8}$

- RTT 偏差

SampleRTT 一般会偏离 EstimatedRTT 的程度

$$DevRTT = (1 - \beta)DevRTT + \beta |SampleRTT - EstimatedRTT|$$

一般取 $\beta = \frac{1}{4}$

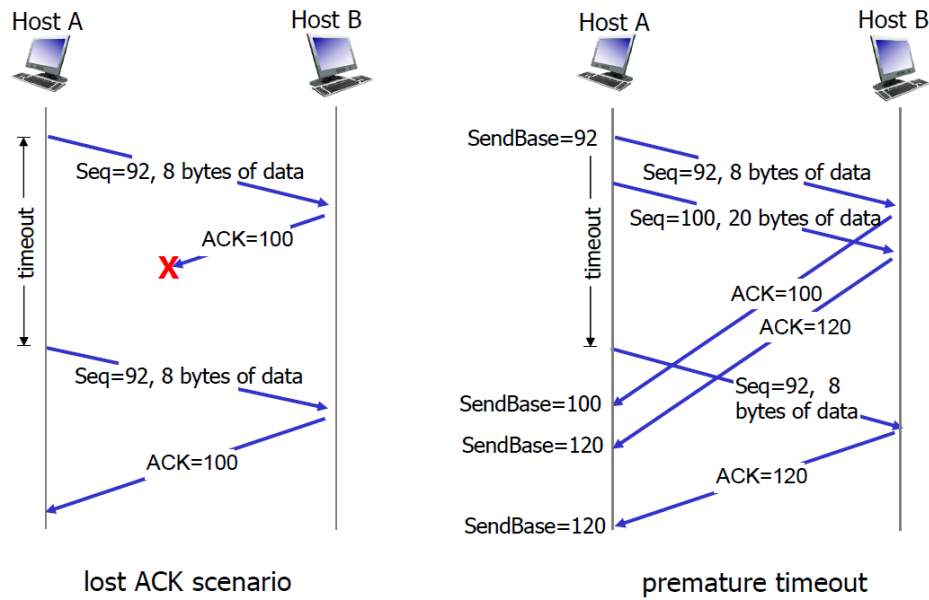
- 超时时间间隔 RTO (Retransmission Timeout)

$$TimeoutInterval = EstimatedRTT + 4DevRTT$$

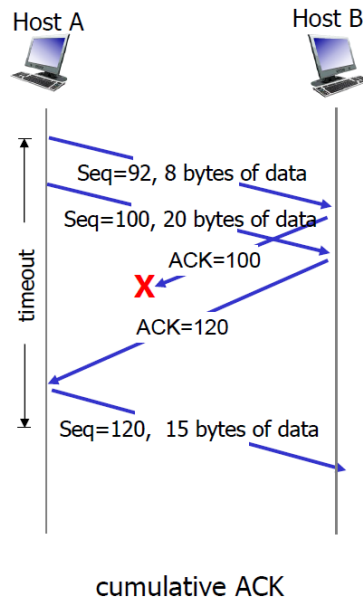
3.5.5 可靠数据传输

- Premature Timeout

第二个报文段的 ACK 在新的超时发生之前到达, 不会重传第二个报文段



- 累积 ACK 的作用



- 超时间隔加倍
对于一个包，每次重传的时候会把超时间隔加倍（指数增长）
- 快速重传 Fast Retransmit
接收方功能：

<i>event at receiver</i>	<i>TCP receiver action</i>
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK . Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	cumulative ACK . immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send duplicate ACK , indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send updated ACK , provided that segment starts at lower end of gap

如果发送方收到了 3 个冗余 ACK，则马上重传

3 个是冗余，一共收到了 4 个！！

- TCP 的流水线差错恢复机制

既不是 GBN 也不是 SR

有一种实现方式是选择确认 (Selective Acknowledgment)

3.5.6 流量控制 Flow Control

- 接收窗口 Receive Window (*rwnd*)

用于给发送方一个指示：接收方还有多少可用的缓存空间

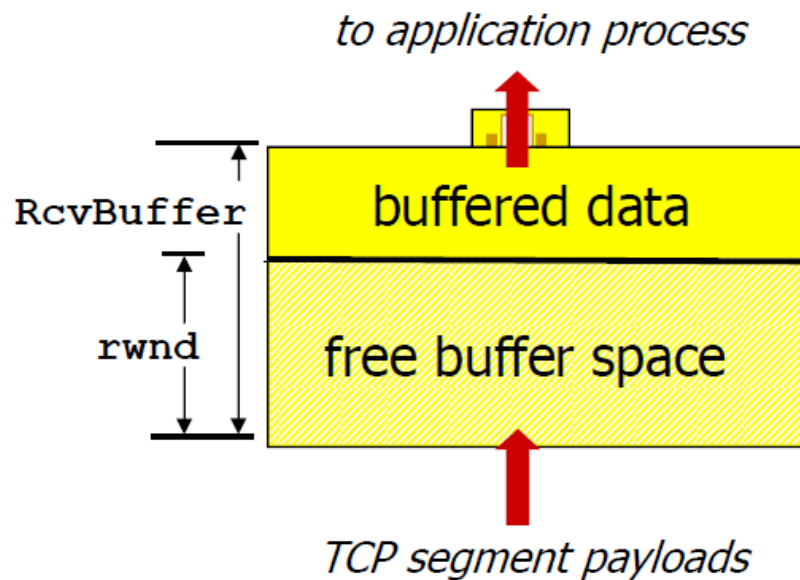
- *LastByteRead*: 接收方的应用进程从缓存读出的数据流的最后一个字节编号

LastByteRcvd: 从网络中到达并已放入接收方接收缓存中的数据流的最后一个字节编号

- 接收窗口与接收缓存之间的关系

$$LastByteRcvd - LastByteRead \leq RcvBuffer$$

$$rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]$$



- 发送方操作

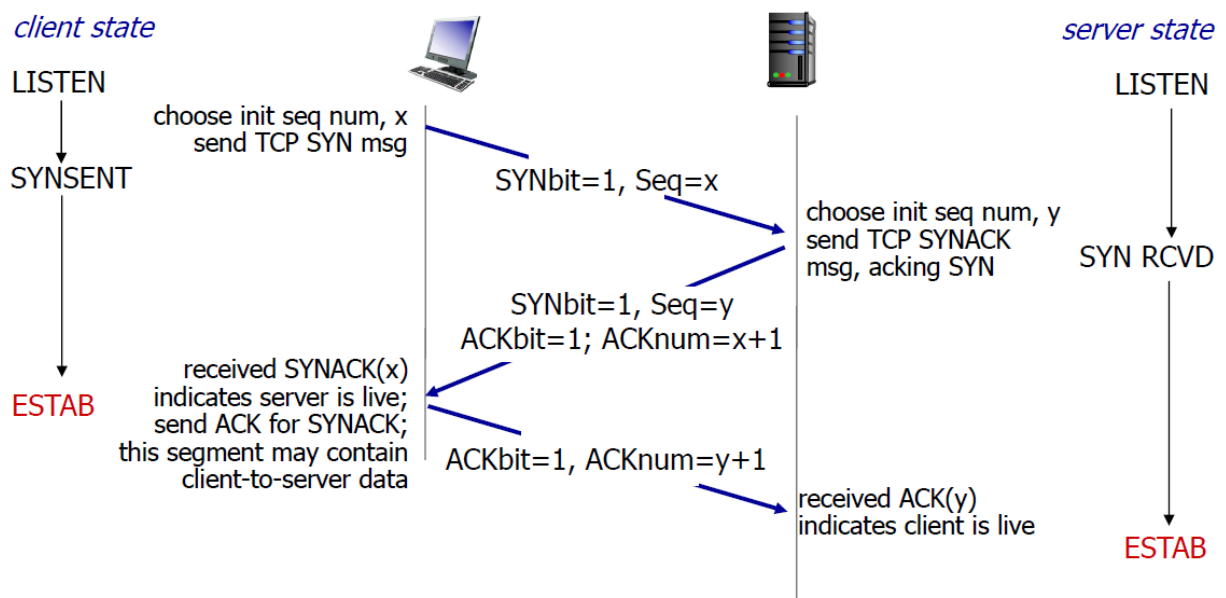
发送方要把发送到连接中但还未收到确认的数据量控制在 $rwnd$ 以内，从而保证不会使接收方缓存溢出

$$LastByteSent - LastByteAcked \leq rwnd$$

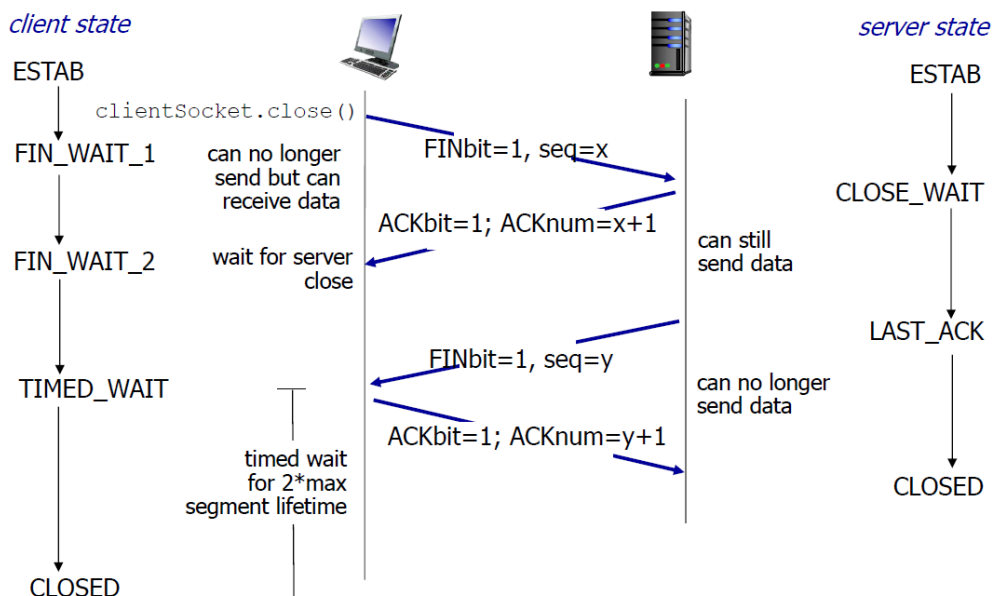
3.5.7 TCP 连接管理 Connection Management

- 三次握手 Three-way Handshaking

确认两方的发送、接收功能都正常

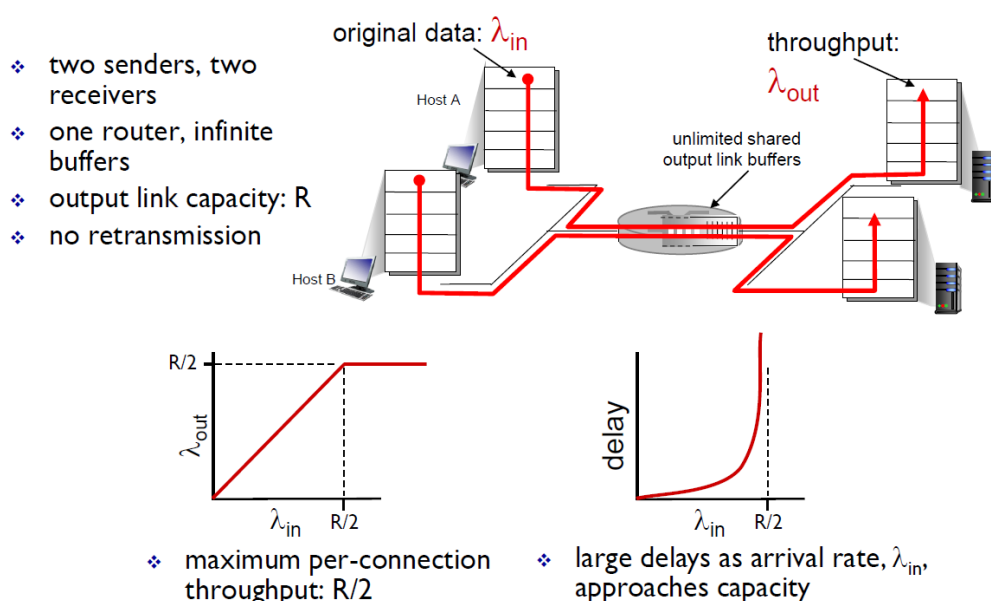


- 关闭连接：四次握手



3.6 拥塞控制原理 Principles of Congestion Control

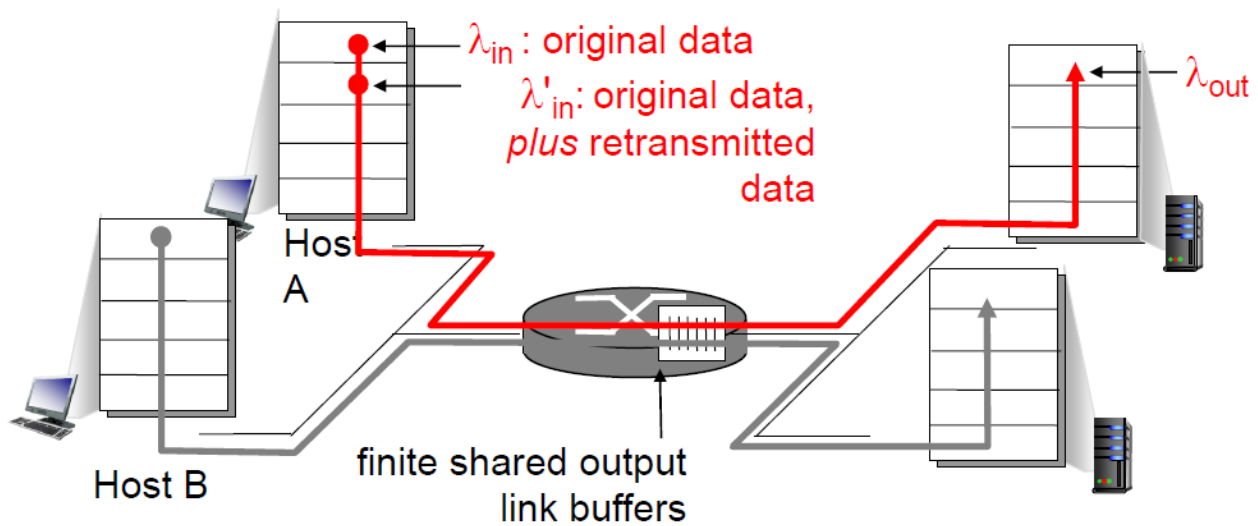
3.6.1 情况 1：两个发送方和一台具有无限大缓存的路由器



- 输出链路的吞吐量是 R

所以如果发送速率太大，输出小于输入，队列无限延长

3.6.2 情况 2：两个发送方和一台具有有限缓存的路由器



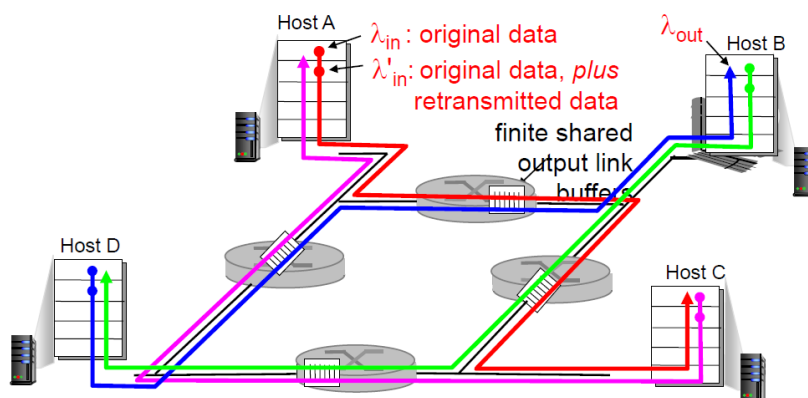
- 供给载荷 Offered Payload: λ'_{in} bytes/second
- 理想的情况：发送方只有在路由器缓存有空间的时候才发包
- 还行的情况：发送方仅当确定一个分组丢失时才重传
- 最坏的情况：发送方的 timer 超时了，重发，而实际上没丢包，只是还在路由器里排队 (premature timeout)。所以重传的分组是没必要的，浪费了带宽资源

3.6.3 情况 3：4 个发送方和具有有限缓存的多台路由器和多跳路径

- ❖ four senders
- ❖ multihop paths
- ❖ timeout/retransmit

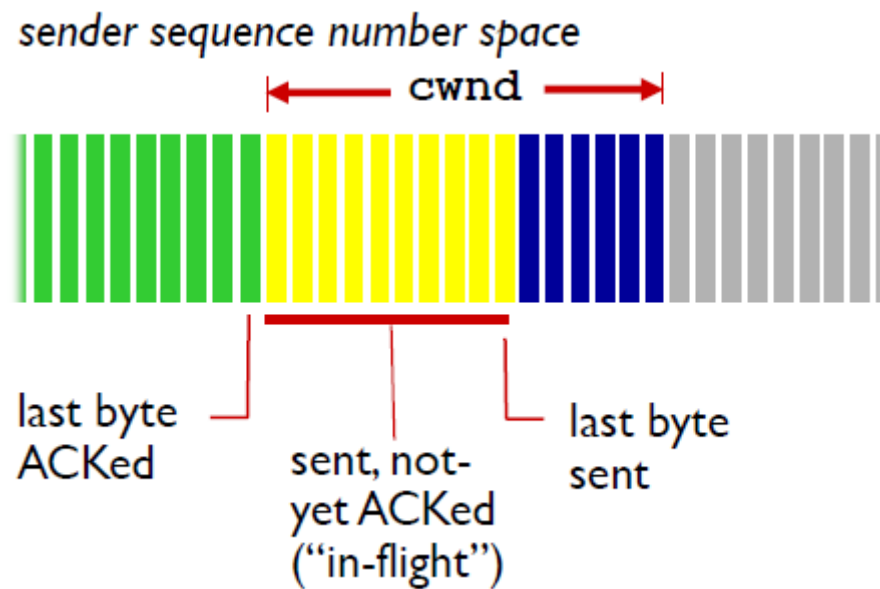
Q: what happens as λ_{in} and λ'_{in} increase ?

A: as red λ'_{in} increases, all arriving blue pkts at upper queue are dropped, blue throughput $\rightarrow 0$



3.7 TCP 拥塞控制

- 拥塞窗口 Congestion Window ($cwnd$)



$$LastByteSent - LastByteAcked \leq \min\{cwnd, rwnd\}$$

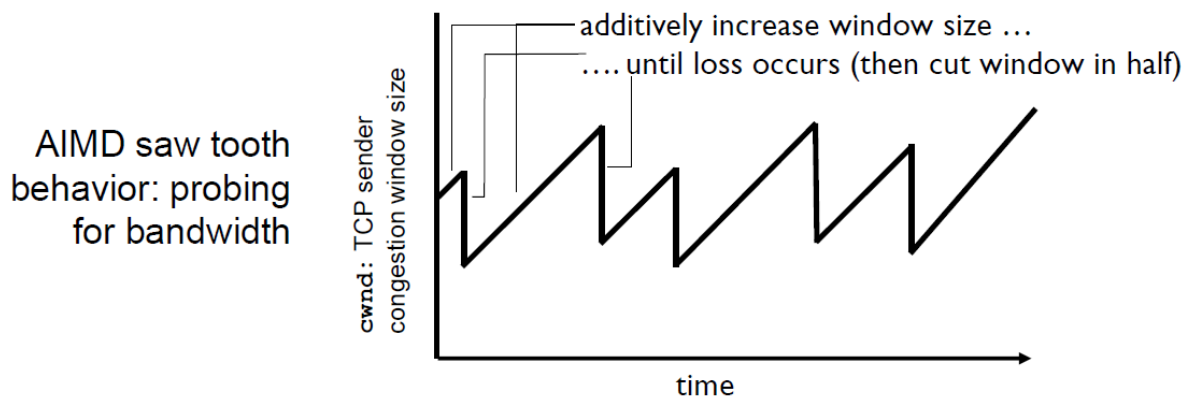
发送速率 $rate = \frac{cwnd}{RTT} \text{ byte/s}$ (不考虑丢包和发送时延)

动态大小控制：自计时 Self-clocking

- TCP 拥塞控制的原则：
 - 一个丢失的报文段意味着拥塞，因此当丢失报文段时应降低 TCP 发送方的速率
 - 一个 ACK 报文段指示该网络正在向接收方交付发送方的报文段，因此当未确认报文段的确认到达时，能增加发送方的速率 **$cwnd$ 在每次收到 ACK 时增加**
 - 带宽探测。TCP 逐步尝试逼近拥塞时的速率，并在到达后快速后退，进而再次开始探测
 - 注：怎么降低/增加发送方速率，看上面公式，RTT 不好改，所以改 $cwnd$
- 特点
 - 加性增 Additive Increase
 - 乘性减 Multiplicative Decrease

■ **additive increase**: increase $cwnd$ by 1 MSS every RTT until loss detected

■ **multiplicative decrease**: cut $cwnd$ in half after loss



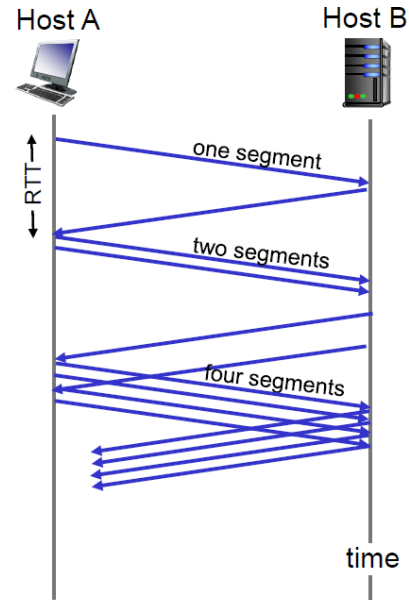
3.7.1 慢启动 Slow Start

- 在开始阶段, $cwnd$ 以指数增长

$$cwnd+ = MSS$$

比如现在的 $cwnd = 2$, 说明链路上有两个包, 收到一次 ACK 就加 1 个 MSS, 所以最后 $cwnd = 4$, 然后 $+1+1+1+1$ 变成 8...

- ❖ when connection begins, increase rate exponentially until first loss event:
 - initially $cwnd = 1\text{ MSS}$
 - double $cwnd$ every RTT
 - done by incrementing $cwnd$ for every ACK received
- ❖ summary: initial rate is slow but ramps up exponentially fast



- 慢启动阈值 $ssthresh$

检测到拥塞时, $ssthresh = cwnd/2$

当慢启动到达 $ssthresh$ 时, 结束慢启动, 转入拥塞避免模式

3.7.2 拥塞避免 Congestion Avoidance

- $cwnd$ 以每 RTT 1 MSS 的速率增长

$$cwnd+ = MSS \cdot \frac{MSS}{cwnd}$$

- 出现拥塞=丢包了, 发送方检测到丢包, 要么是 timeout, 要么是三次冗余 ACK 触发快速重传

- 超时: 问题很大, 重新从 1 开始

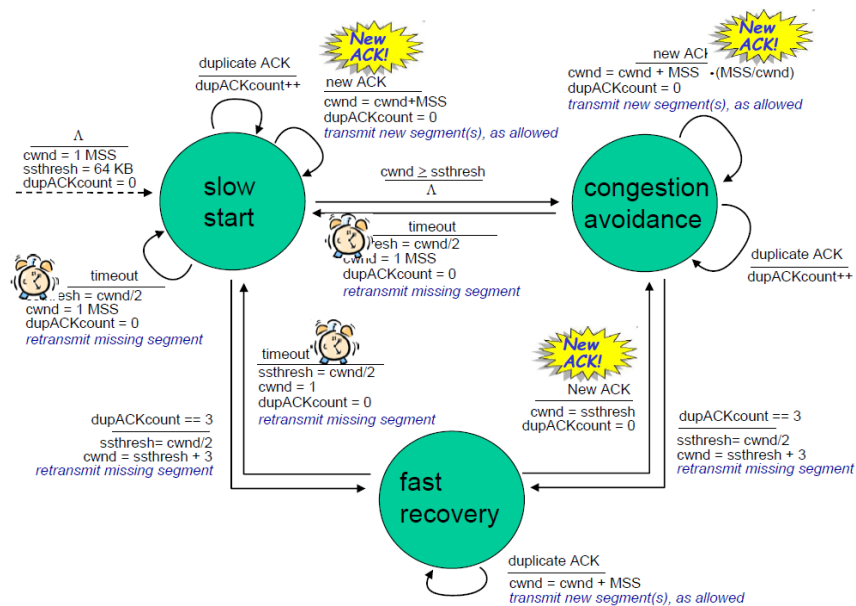
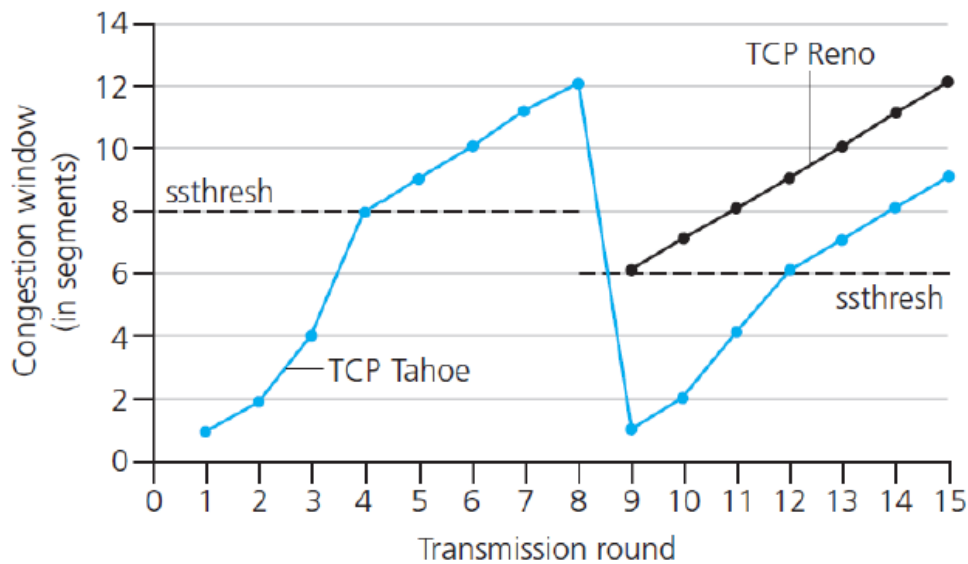
$ssthresh = cwnd/2, cwnd = 1MSS$, 进入慢启动状态

- 收到三次冗余 ACK: 最起码路还是通的

$ssthresh = cwnd/2, cwnd = \frac{1}{2}cwnd + 3MSS$, 进入快速恢复状态

注意这三个冗余 ACK 会让 $cwnd+ = 3MSS$, 具体看下面的状态图

3.7.3 快速恢复 Fast Recovery



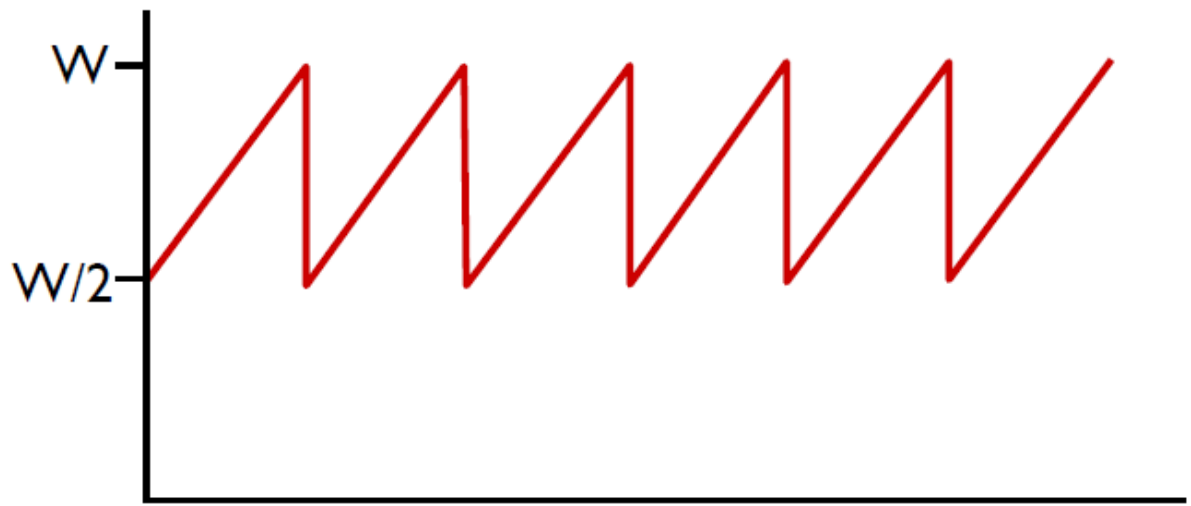
3.7.4 TCP 吞吐量

- 一条连接的吞吐量

忽略慢启动阶段

W : 发生丢包时的窗口长度

$$avgThroughput = \frac{3}{4} \frac{W}{RTT}$$



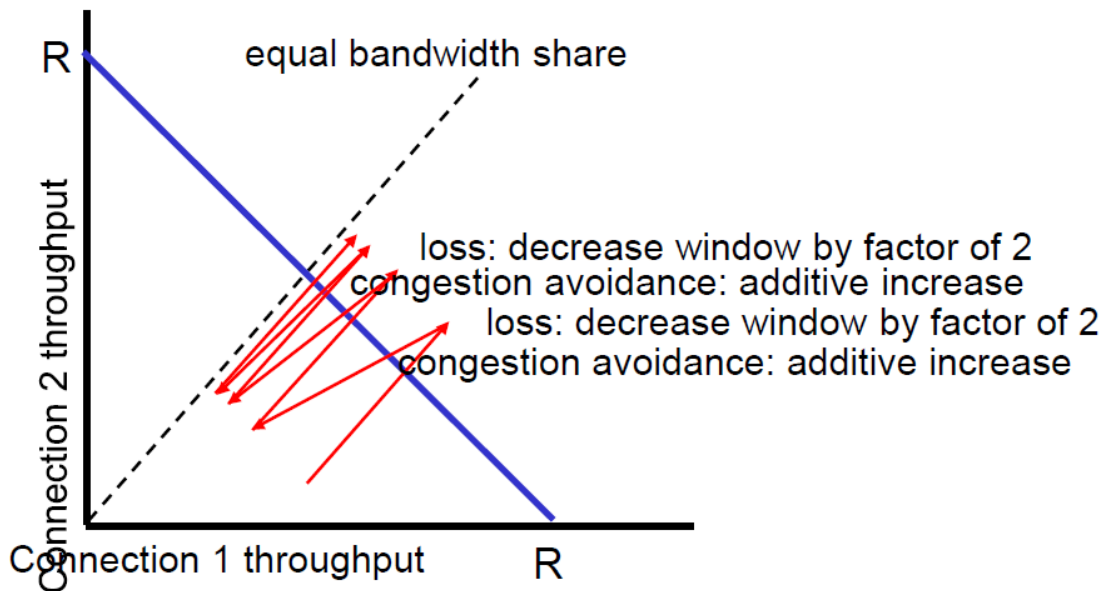
- 吞吐量和丢包率的关系

L : 丢包率 Loss Probability

$$avgThroughput = \frac{1.22MSS}{RTT\sqrt{L}}$$

3.7.5 TCP 公平性

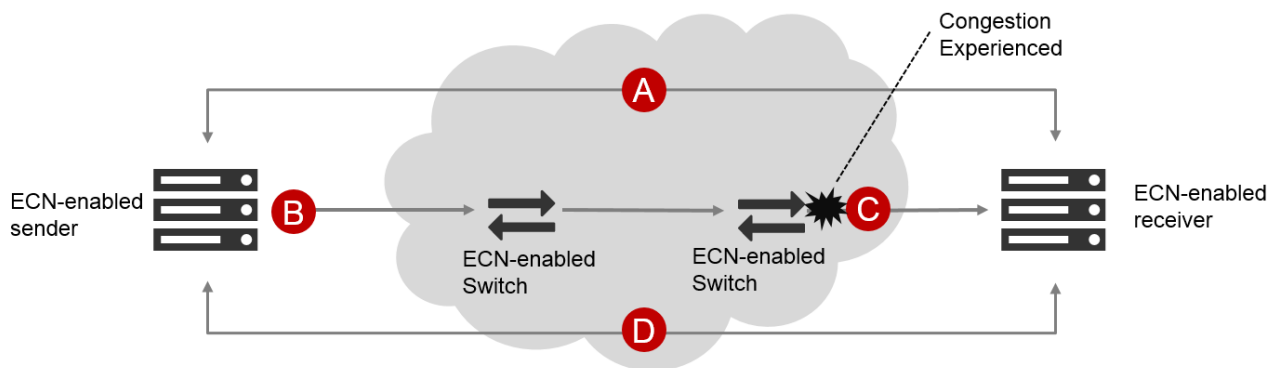
- K 个 TCP 连接共用一段传输速率为 R 的瓶颈链路，则每个连接的平均传输速率应为 $\frac{R}{K}$
- 例：两个 TCP 连接的带宽平衡



- 当然也可以不公平
一个应用可以建立多个平行 TCP 连接
开的越多，抢的带宽越多

3.7.6 明确拥塞通告：网络辅助拥塞控制

明确拥塞通告 ECN (Explicit Congestion Notification)



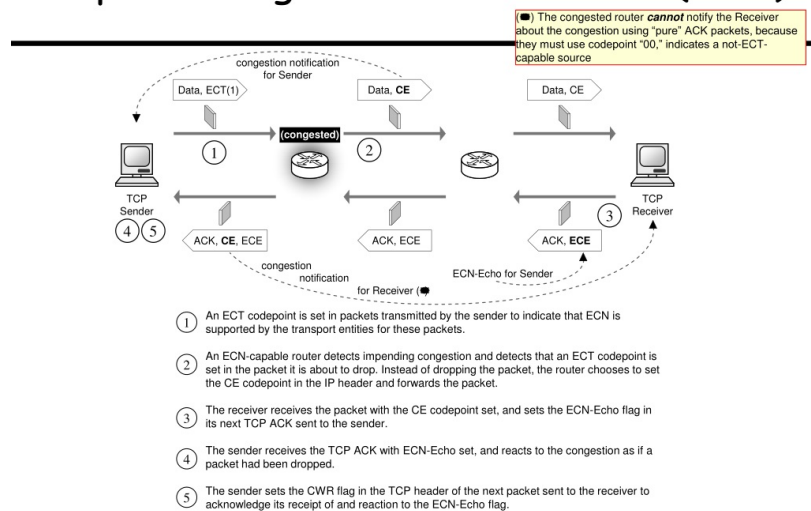
A ECN (Explicit Congestion Notification) negotiated during connection establishment

C CE set (Congestion Experienced)

B ECT set (ECN Capable Transport)

D Receiver echoes congestion experienced CE to sender

Explicit Congestion Notification (ECN)



第四章 网络层：数据平面 Network Layer: The Data Plane

4.1 网络层概述

4.1.1 转发和路由选择：数据平面和控制平面

- 转发 Forwarding (数据平面)

将分组从一个输入链路接口转移到适当的输出链路接口的路由器本地动作。

转发表 (forwarding table)

- 路由选择 Routing (控制平面)

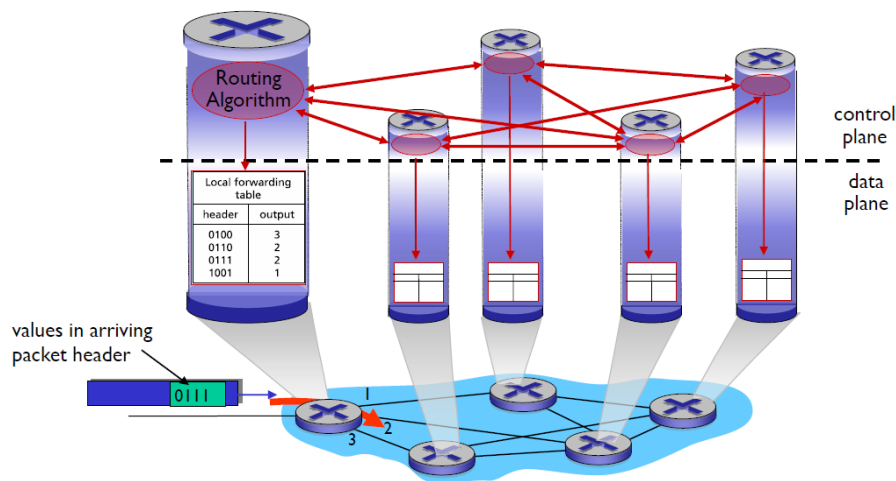
确定分组从源到目的地所采取的端到端路径的网络范围处理过程。

当分组从发送方流向接收方时，网络层必须决定这些分组所采用的路由或路径。计算这些路径的算法被称为**路由选择算法 (Routing Algorithm)**。

- 两种控制平面

传统

Individual routing algorithm components *in each and every router* interact in the control plane



4.1.2 网络服务模型 Network Service Model

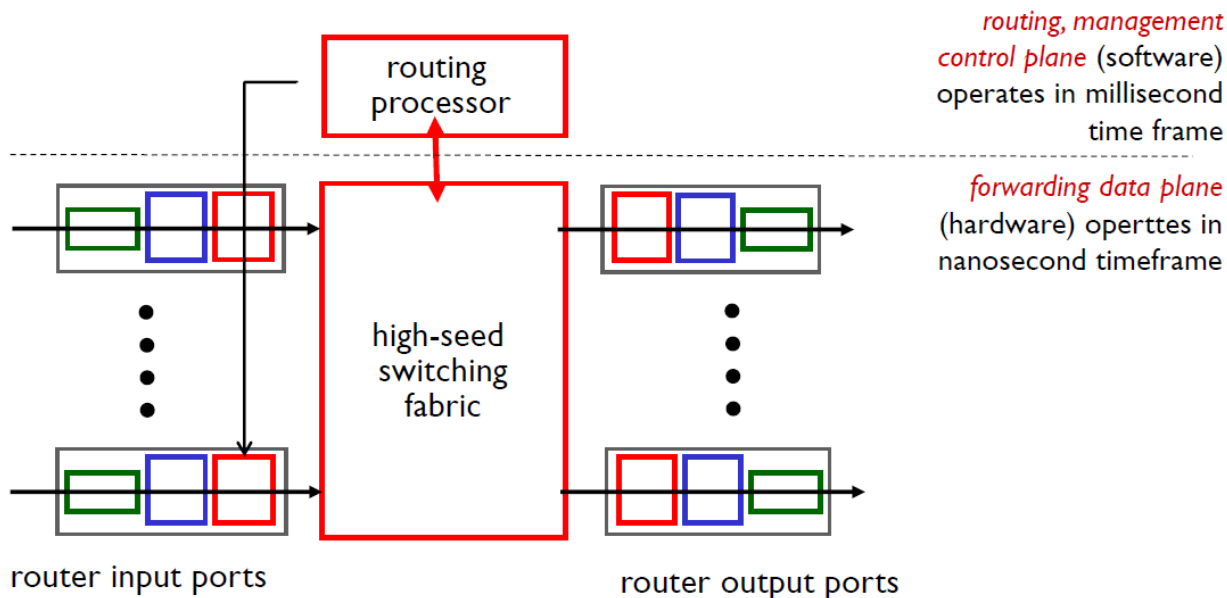
- 可能的服务
 - 确保交付
 - 具有时延上限的确保交付
 - 有序分组交付
 - 确保最小带宽
 - 安全性

- 因特网网络层的服务

尽力而为服务 best-effort service

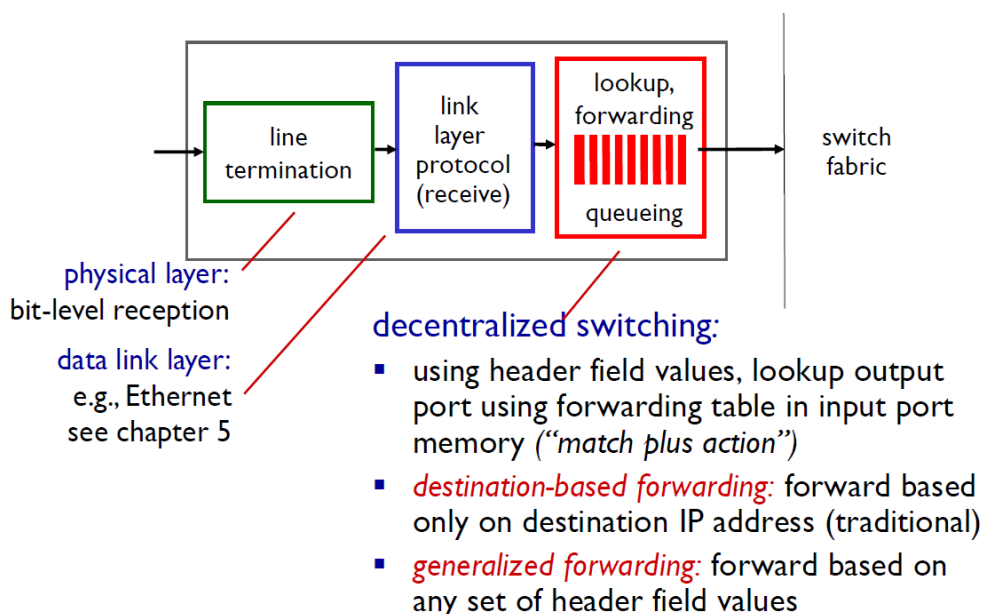
4.2 路由器工作原理 Router

4.2.1 路由器结构概述 Router Architecture



- 输入端口 Input Port
- 交换结构
- 输出端口
- 路由选择处理器

4.2.2 输入端口处理和基于目的地转发



- 基于目的地转发 Destination Based Forwarding
 - 最长前缀匹配原则 Longest Prefix Matching Rule
 - 转发表中找目的地址的最长前缀匹配项

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

examples:

DA: 11001000 00010111 00010110 10100001

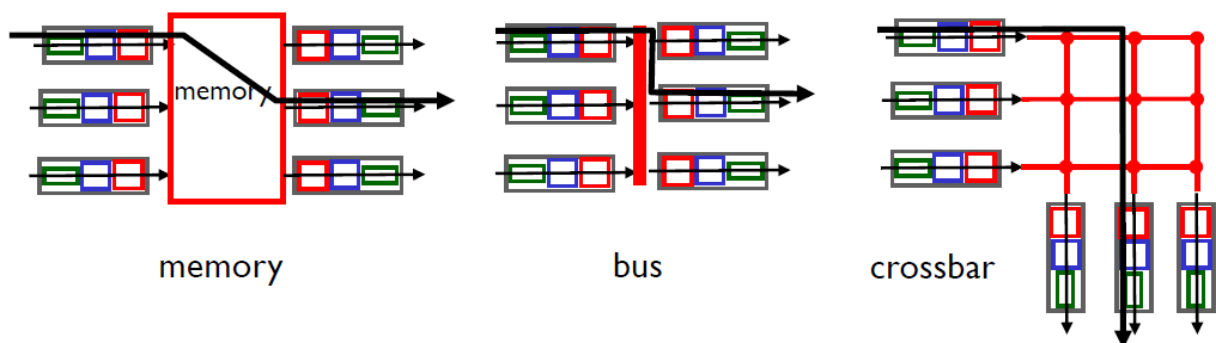
which interface?

DA: 11001000 00010111 00011000 10101010

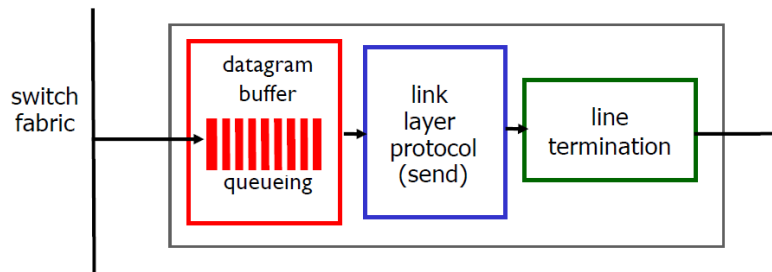
which interface?

4.2.3 交换

- 功能
 - 把分组从输入缓存转移到合适的输出缓存
- 三种交换技术
 - 经内存交换
 - 经总线交换
 - 经互连网络交换 (非阻塞)



4.2.4 输出端口处理



- **buffering** required from fabric faster rate Datagram (packets) can be lost due to congestion, lack of buffers
- **scheduling** datagrams Priority scheduling – who gets best performance, network neutrality

4.2.5 排队

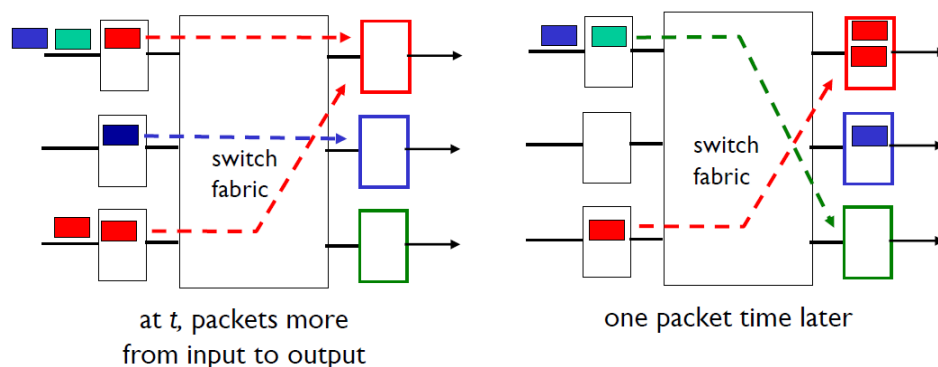
1. 输入排队

- 线路前部 (HOL, Head-Of-the-Line) 阻塞

在一个输入队列中排队的分组必须等待通过交换结构发送，因为它被位于线路前部的分组阻塞

2. 输出排队

- 主动队列管理 AQM Active Queue Management
- 随机早期检测 RED Randomly Early Detection: 一种 AQM 算法
- 排队时延和丢包都是在这发生的
- 分组丢弃策略
 - 弃尾 tail-drop
 - 优先权 priority
 - 随机 random



- buffering when arrival rate via switch exceeds output line speed
- **queueing (delay) and loss due to output port buffer overflow!**

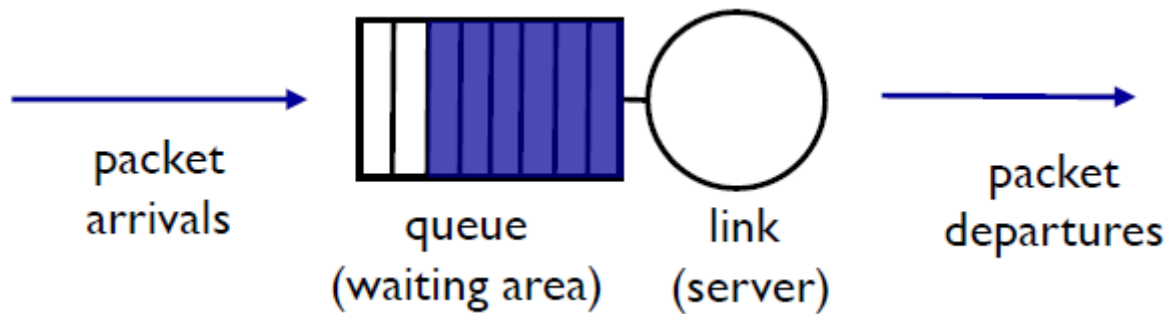
3. 缓存长度

当有 N 条流流过一个容量为 C 的链路时，缓存所需要的数量是

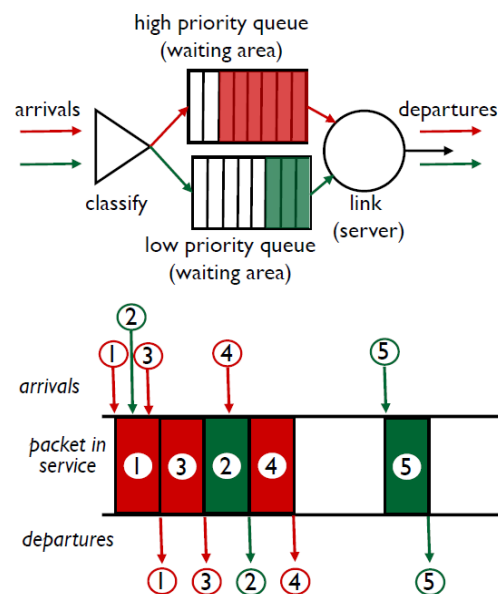
$$B = \frac{RTT \cdot C}{N}$$

4.2.6 分组调度

1. 先进先出 FIFO

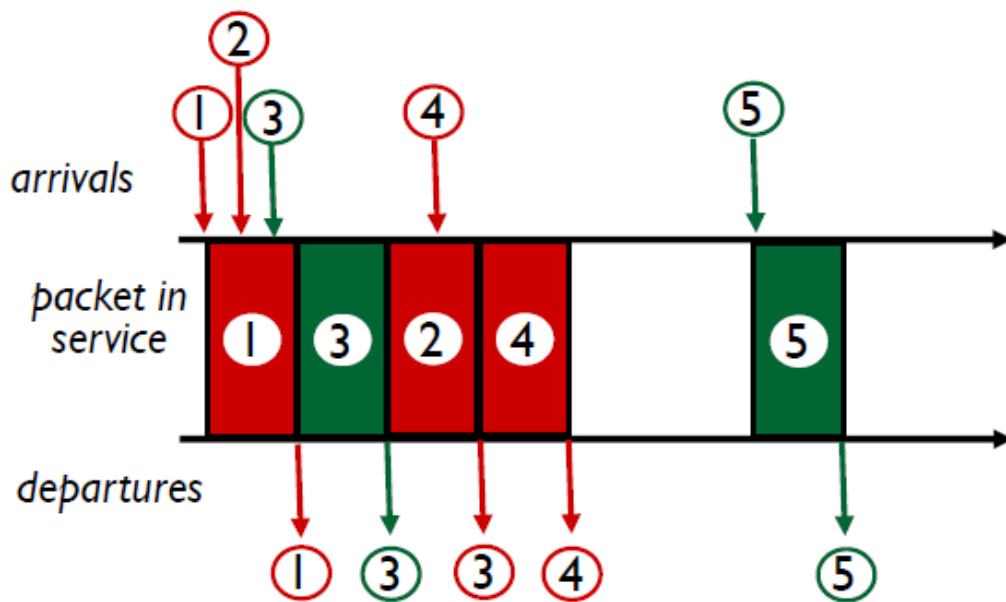


2. 优先权排队 Priority Scheduling



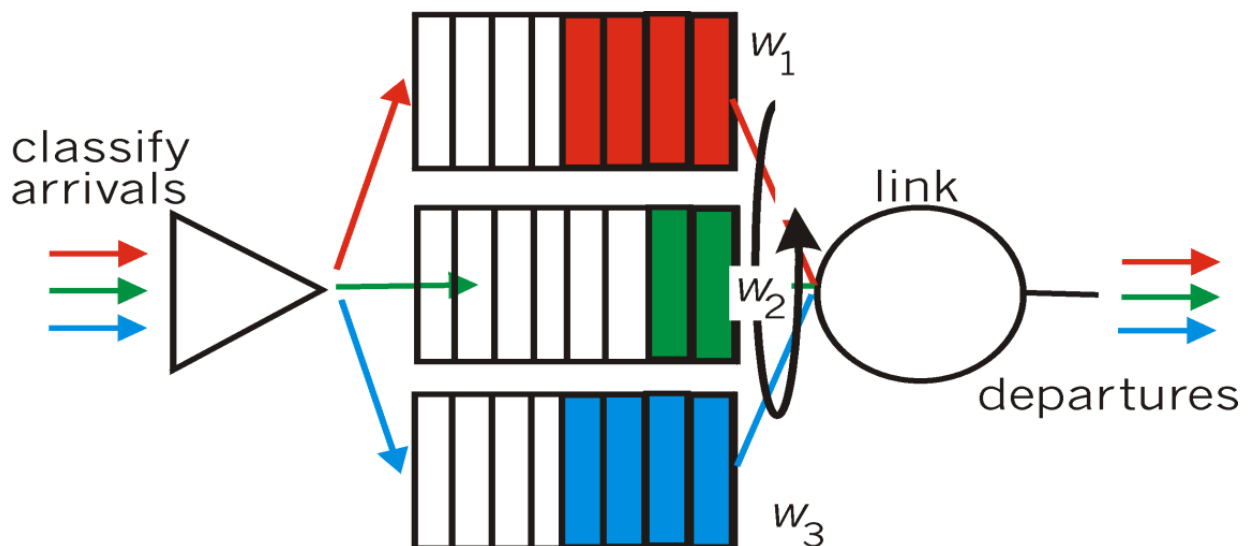
3. 循环排队 Round Robin Scheduling

把分组分类，按顺序一个类发一个



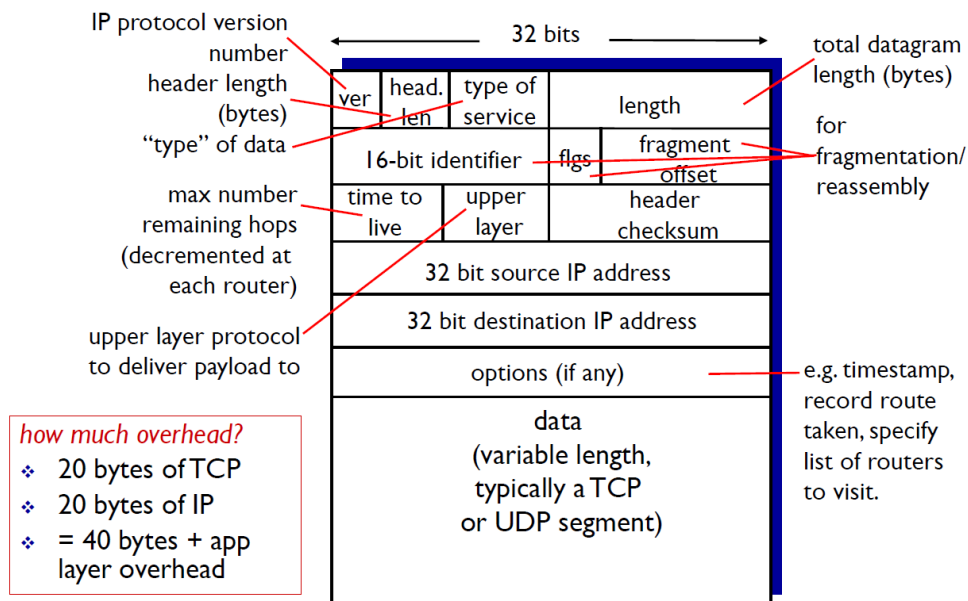
4. 加权公平排队 WFQ Weighted Fair Queuing

每一类按权重分配带宽



4.3 网际协议 Internet Protocol

4.3.1 IPv4 数据报格式



- 版本号
- 首部长度
- 服务类型 TOS
- 数据报长度 (header+payload)
- 标识、标志、片偏移
- 寿命 TTL
- 协议 (运输层)
- 首部检验和
- 源和目的 IP 地址
- 选项
- 数据 (有效载荷)

4.3.2 IPv4 数据报分片

- 最大传送单元 MTU Maximum Transmission Unit
链路层帧能承载的最大数据量
如果 IP 数据报较长，大于 MTU，就需要分片
- 片 Fragment
片在到达目的地后才重新组装，依据是标识、标志、片偏移
 - 标识：每发一个片 +1
 - 标志：最后一个片的是 0，其他片是 1
 - 片偏移：表示这个片是从总体的哪一位开始的
- 片长度
一个长度 L 的数据报分成 n 片，总长度是 $L + 20(n - 1)$ ，20 指 header 的长度

example:

- ❖ 4000 byte datagram
- ❖ MTU = 1500 bytes

1480 bytes in
data field

offset =
 $1480/8$

	length =4000	ID =x	fragflag =0	offset =0
--	-----------------	----------	----------------	--------------

*one large datagram becomes
several smaller datagrams*

	length =1500	ID =x	fragflag =1	offset =0
--	-----------------	----------	----------------	--------------

	length =1500	ID =x	fragflag =1	offset =185
--	-----------------	----------	----------------	----------------

	length =1040	ID =x	fragflag =0	offset =370
--	-----------------	----------	----------------	----------------

4.3.3 IPv4 编址

- 接口 Interface

主机/路由器与物理链路之间的边界

一个 IP 与一个接口关联，而不是与包括该接口的主机或路由器

- 点分十进制记法 Dotted-decimal Notation

每个 IP 地址长度为 32 bit，即 4 字节，每个字节按十进制书写，字节之间以点隔开

- 一个 IP 地址分为 subnet part 和 host part

- 子网 Subnet

对于网络的逻辑划分

一个子网内部的设备不通过路由器互相连接

- 子网掩码 Subnet Mask

IP & Mask=Subnet Part 按位与

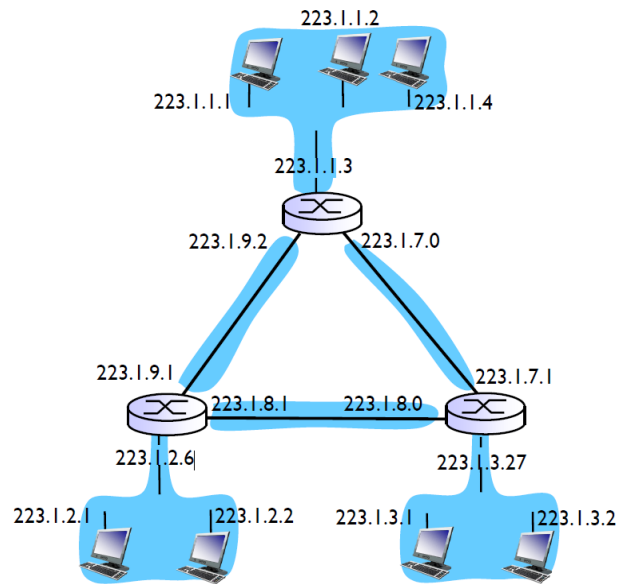
- 例: IP=192.168.2.1, mask=255.255.255.0

子网: 192.168.2.0

或写成 192.168.2.1/24

- 计算子网个数: 把路由器拿走，剩下几个区域就有几个子网

例: 6个

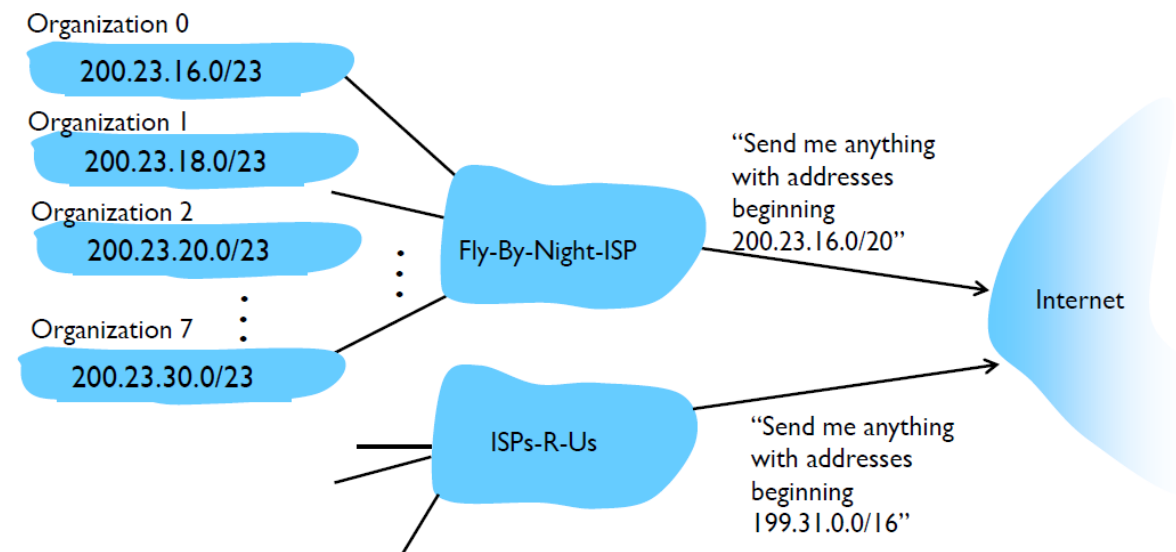


- 无类别域间路由选择 CIDR Classless Interdomain Routing

$a.b.c.d/x$

最高 x 位被称作前缀或网络前缀，剩下的部分用于区分子网内部设备

ISP 在处理消息的时候，把子网地址相同的一起处理。这种使用单个网络前缀通告多个网络的能力被称为地址聚合 (address aggregation) 或路由聚合 (route aggregation) 或路由摘要 (route summarization)



- 广播地址 255.255.255.255
向广播地址发数据报时，该报文会交付给同一网络中的所有主机
- 动态主机配置协议 DHCP Dynamic Host Configuration Protocol
 - 即插即用协议 Plug-and-play Protocol / 零配置协议 Zeroconf Protocol
 - 应用层
传输层是 UDP
 - 步骤
 - DHCP 服务器发现
DHCP 发现报文 (DHCP Discover Message)

发给广播地址

- DHCP 服务器提供

DHCP 提供报文 (DHCP Offer Message)

发给广播地址

- DHCP 请求

DHCP 请求报文 (DHCP Request Message)

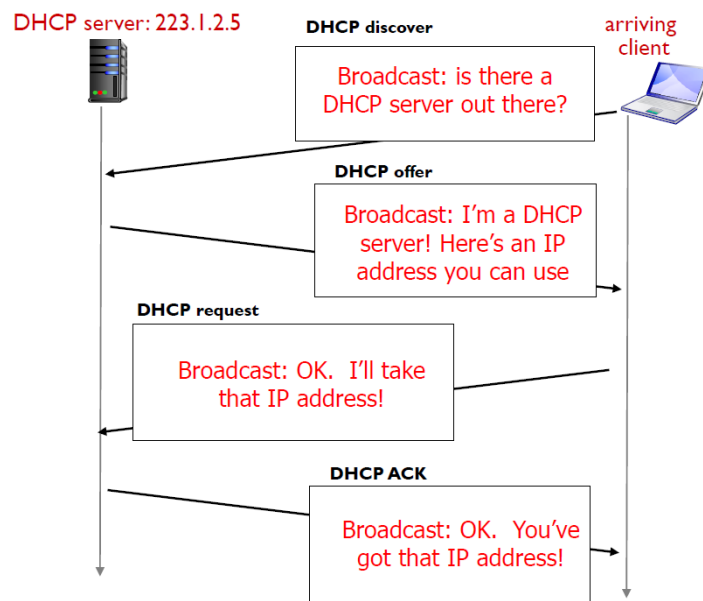
发给广播地址，在报文内容里指明请求的 DHCP 服务器地址

- DHCP ACK

DHCP ACK 报文 (DHCP ACK Message)

发给广播地址

整个过程 client 的 IP 地址都是 0.0.0.0

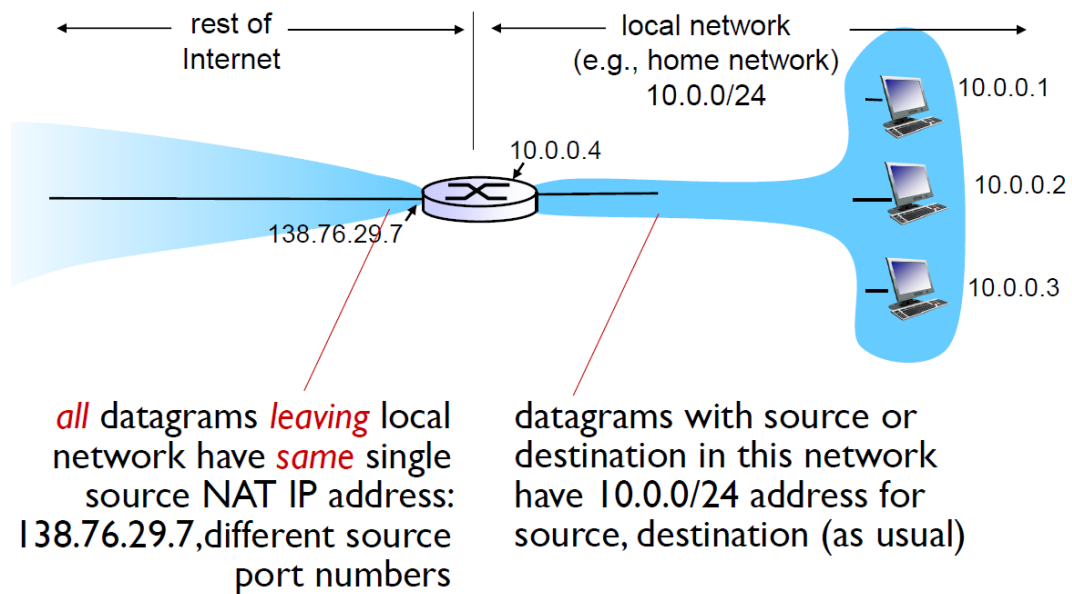


○ DHCP 还能提供以下信息

- 第一跳的路由器地址
- DNS 服务器的名称和 IP 地址
- 子网掩码

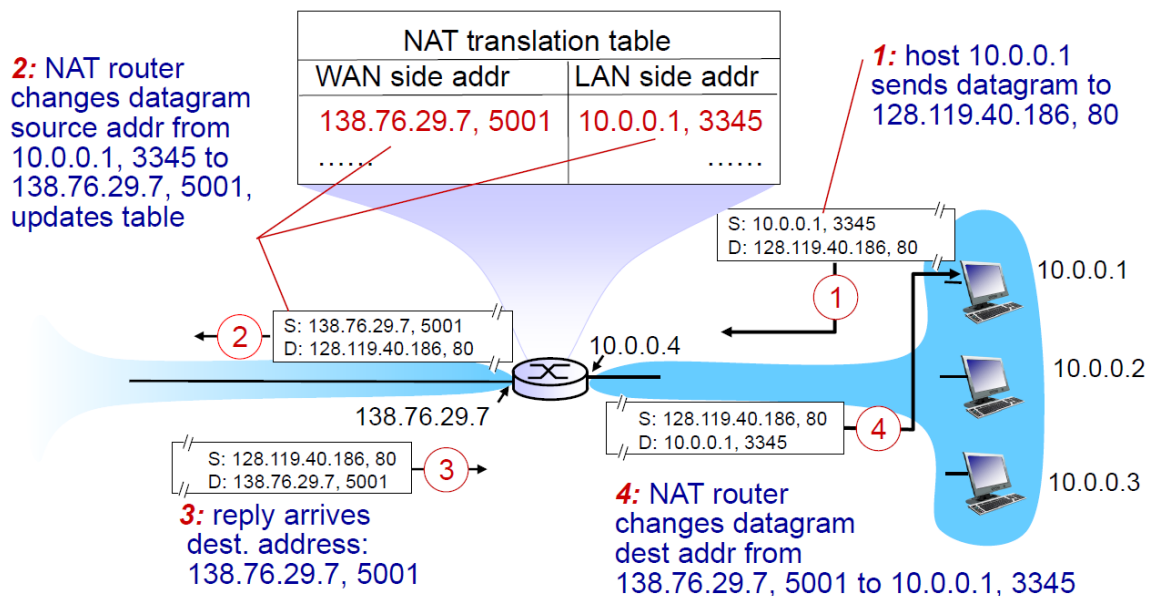
4.3.4 网络地址转换 NAT

- Network Address Translation (NAT)



- NAT 转换表

- 区分内部的主机：看端口号



- 外部客户端访问 NAT 内部服务器：NAT 穿越 (NAT Traversal)

4.3.5 IPv6

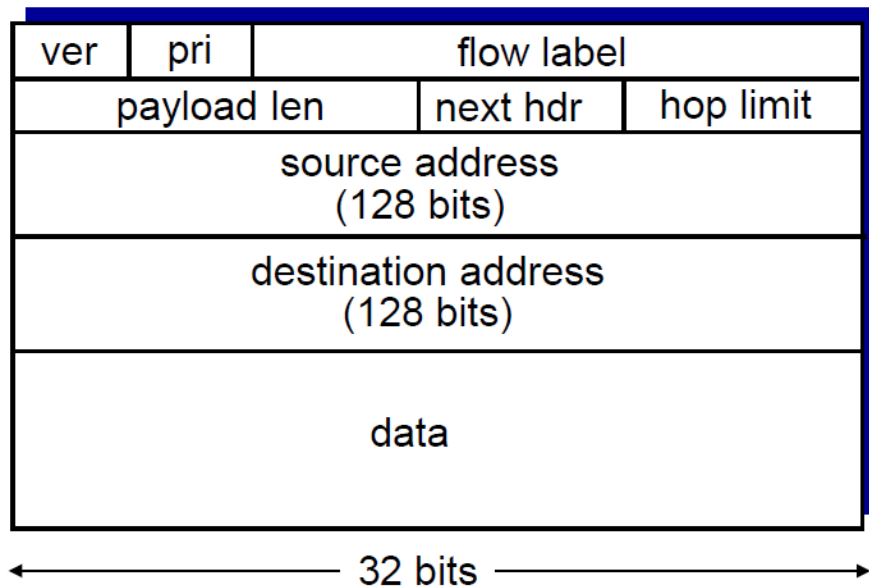
- IPv6 数据报变化

- 扩大的地址容量: 32 bit -> 128 bit
- 简化高效的 40 字节定长首部
- 流标签
- 没有分片 (fragment)
- 没有首部 checksum
- 没有选项字段

- IPv6 数据报字段

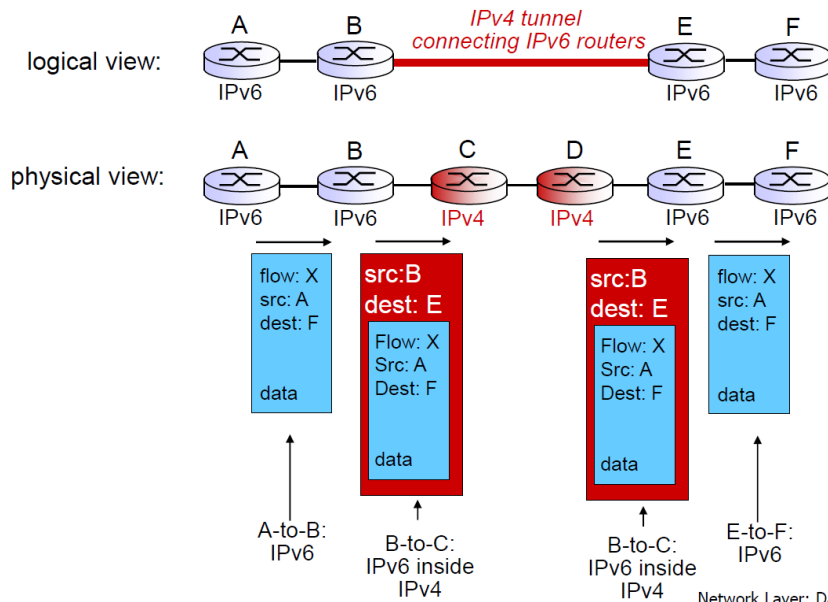
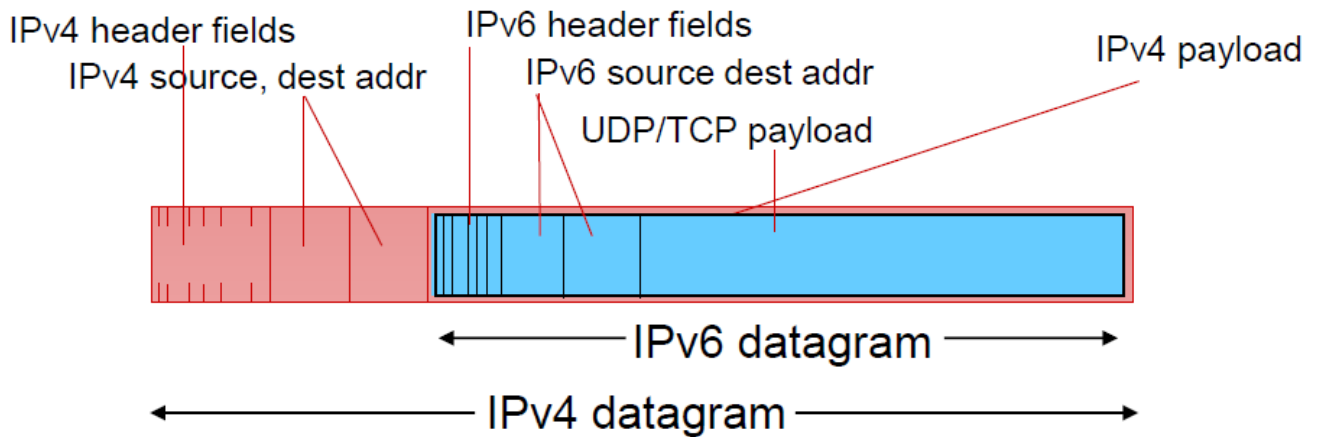
- 版本
- 流量类型
- 流标签
- 有效载荷长度

- 下一个首部: payload 交给什么运输层协议
- 跳限制: TTL
- 源地址和目的地址
- 数据

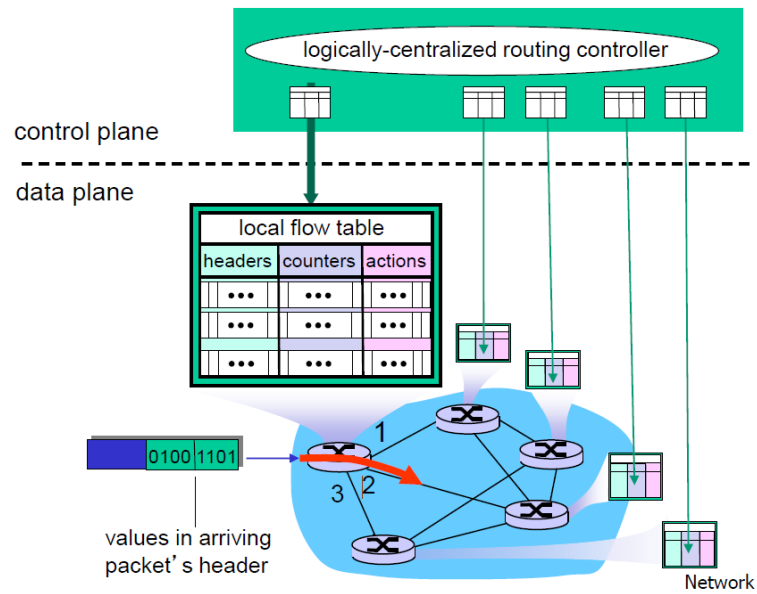


- 隧道 Tunnel

实现 IPv4 路由器和 IPv6 路由器之间通信



4.4 通用转发和 SDN



1. 流 Flow

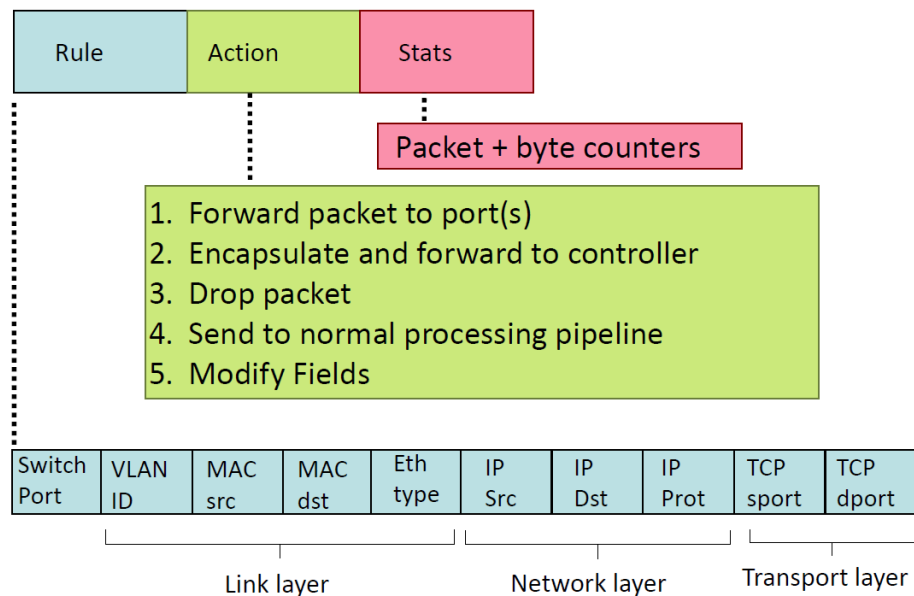
在首部中定义

2. OpenFlow 通用转发

- 匹配
 - 转发
 - 丢弃
 - 修改字段
- 优先级
- 计数器

3. 流表 Flow Table

- 首部字段值的集合
- 计数器集合
- 所采取的动作集合



4. 流表例子 Examples

Destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	*	51.6.0.8	*	*	*	port6

IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Firewall:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	*	*	*	*	22	drop

do not forward (block) all datagrams destined to TCP port 22

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP sport	TCP dport	Action
*	*	*	*	*	128.119.1.1	*	*	*	*	drop

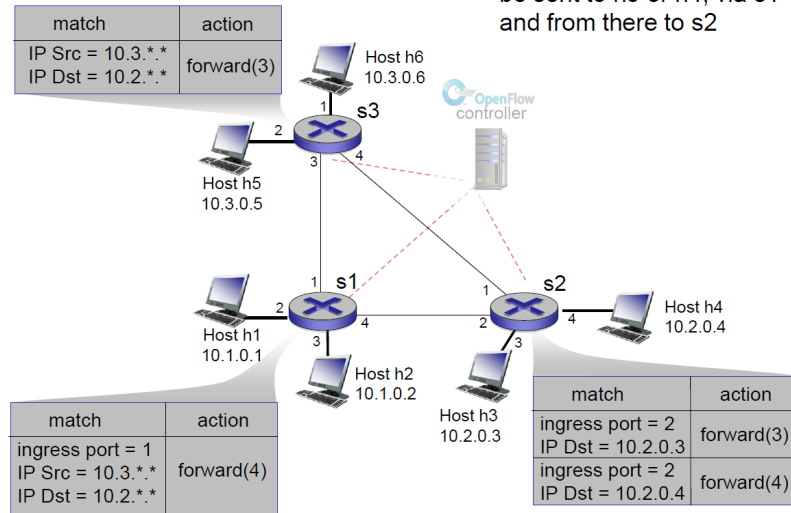
do not forward (block) all datagrams sent by host 128.119.1.1

5. OpenFlow 功能

- 路由 Router
 - 匹配: 最长目的 IP 前缀
 - 动作: 转发
- 交换 Switch
 - 匹配: 目的 MAC 地址
 - 动作: 转发或者 flood
- 防火墙 Firewall
 - 匹配: IP 地址和端口号
 - 动作: 允许或者拒绝
- NAT
 - 匹配: IP 地址和端口号
 - 动作: 重写 IP 地址和端口号

OpenFlow example

Example: datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

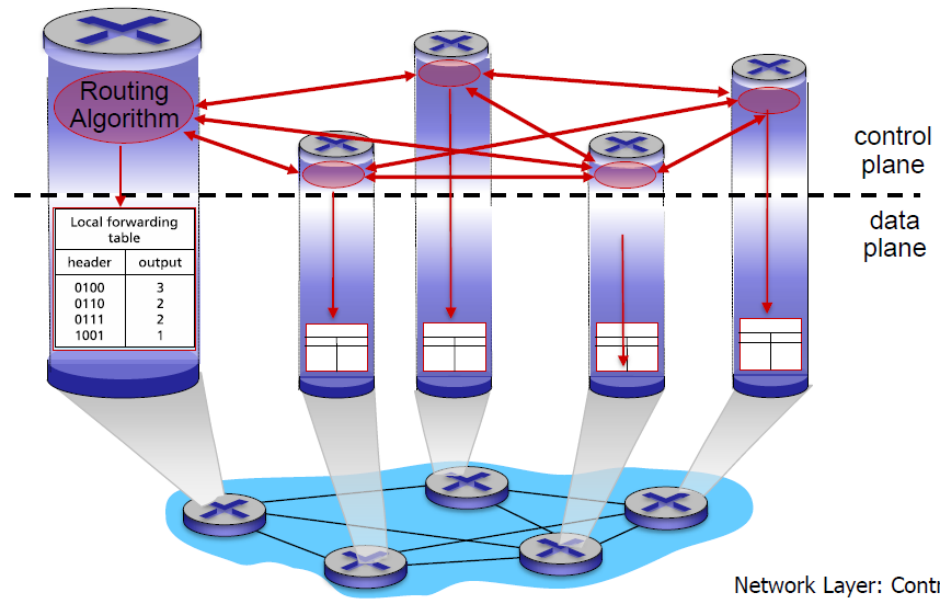


第五章 网络层：控制平面 Network Layer: The Control Plane

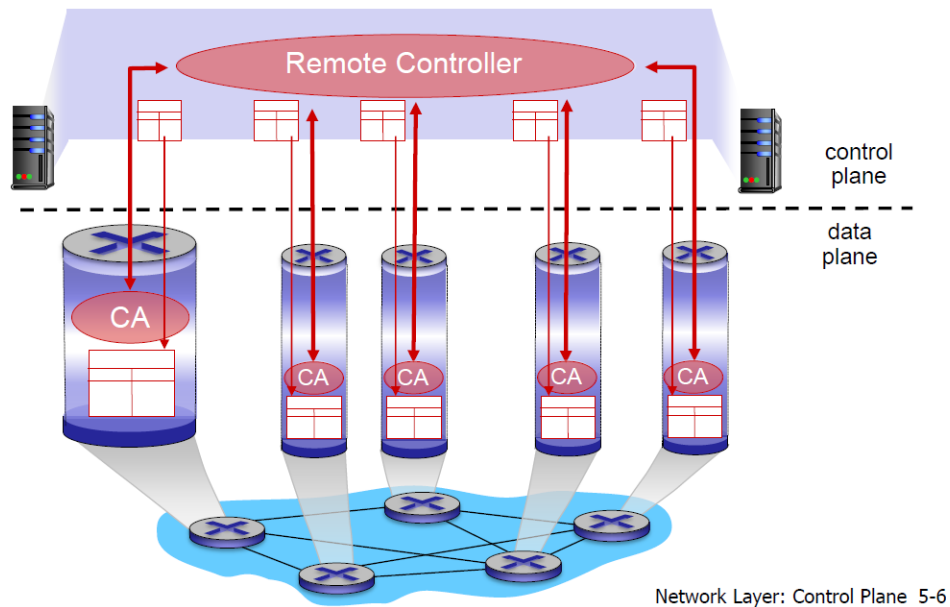
5.1 概述

Routing:

1. 每路由器控制



2. 逻辑集中式控制



5.2 路由选择算法 Routing Algorithm

路由选择算法 Routing Algorithm:

从发送方到接收方的过程中确定一条通过路由器网络的好的路径。通常，一条好路径指具有最低开销的路径

1. 按集中还是分散分类

- 集中式路由选择算法 (Centralized Routing Algorithm)
具有全局信息
链路状态 (Link State, LS) 算法
- 分散式路由选择算法 (Decentralized Routing Algorithm)
距离向量 (Distance-Vector, DV) 算法

2. 按动态还是静态分类

- 静态路由选择算法 (Static Routing Algorithm)
路由随时间的变化非常慢
通常是人工调整
- 动态路由选择算法 (Dynamic Routing Algorithm)
随网络流量负载或拓扑发生变换而改变路由选择路径

3. 按负载敏感还是迟钝分类

- 负载敏感算法 (Load-sensitive Algorithm)
链路开销会动态地变化以反映出底层链路的当前拥塞水平
- 负载迟钝算法 (Load-insensitive Algorithm)
某条链路的开销不明确反映其当前 (或最近) 的拥塞水平

5.2.1 链路状态路由选择算法 LS

Dijkstra's algorithm

- net topology, link costs known to all nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- computes least cost paths from one node (‘source’) to all other nodes
 - gives *forwarding table* for that node
- iterative: after k iterations, know least cost path to k dest.’s

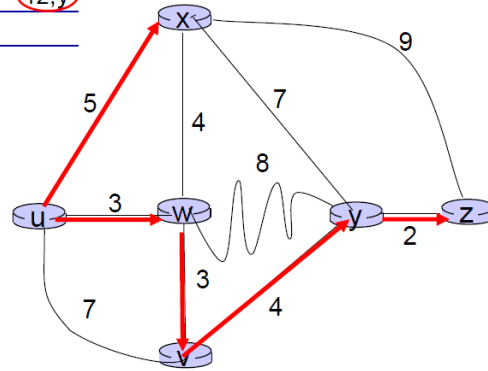
notation:

- $C(x,y)$: link cost from node x to y; $= \infty$ if not direct neighbors
- $D(v)$: current value of cost of path from source to dest. v
- $p(v)$: predecessor node along path from source to v
- N' : set of nodes whose least cost path definitively known

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwX	6,w			11,w	14,x
3	uwXv				10,v	14,x
4	uwXvy					12,y
5	uwXvyz					

notes:

- ❖ construct shortest path tree by tracing predecessor nodes
- ❖ ties can exist (can be broken arbitrarily)



- 例：转发表的写法（和上面不是一个 graph）

resulting forwarding table in u:

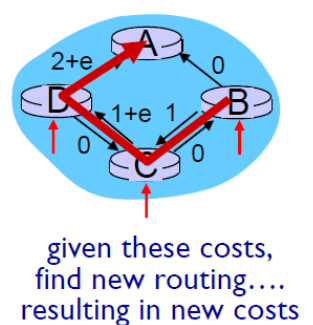
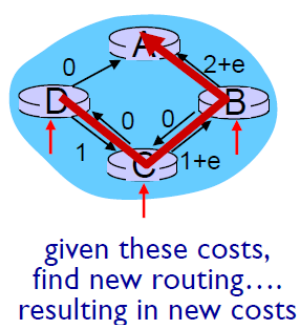
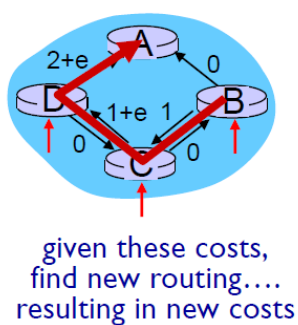
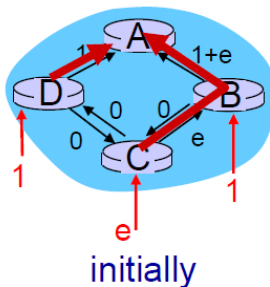
destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

- 受震荡影响

算法运行过程中边权变化了（负载敏感）

oscillations possible:

- e.g., support link cost equals amount of carried traffic:

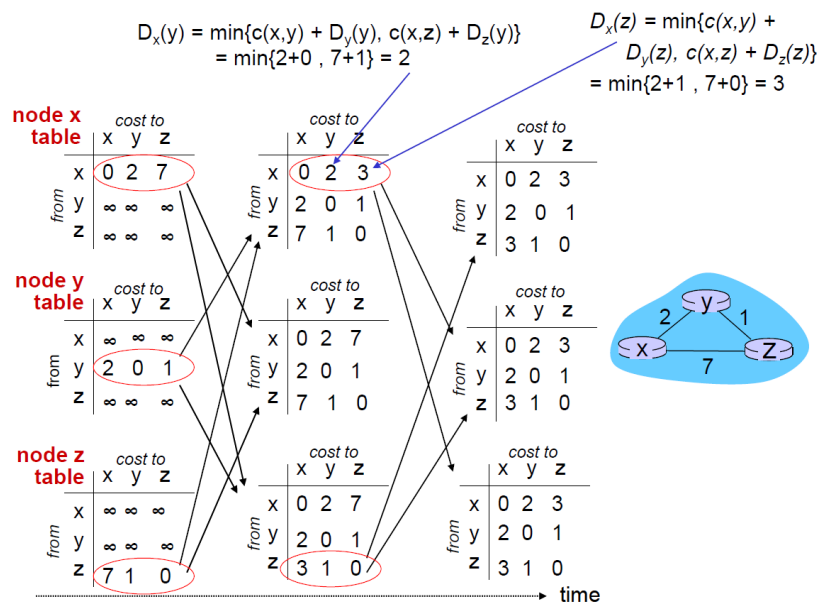


5.2.2 距离路由向量算法 DV

- Bellman-Ford 方程

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

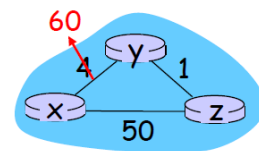
- 每个节点 x 维护以下信息
 - 对于每个邻居 v , 从 x 直接相连邻居 v 的开销为 $c(x, v)$
 - 节点 x 的距离向量, 即 $\mathbf{D}_x = [D_x(y) : y \in N]$, 包含了 x 到 N 中所有目的地 y 的开销估计值
 - 它的每个邻居的距离向量, 即对 x 的每个邻居 v , 有 $\mathbf{D}_v = [D_v(y) : y \in N]$
- 工作流程
 - 每个节点不时地向它的每个邻居发送它的距离向量副本
 - 节点 x 从它的邻居接收到距离向量后, 使用 B-F 方程更新自己的距离向量
 - 如果 x 的距离向量因这个更新步骤而改变, 节点 x 接下来向它的每个邻居发送更新后的距离向量, 这继而让所有邻居更新自己的距离向量



- 路由选择环路 Routing Loop

link cost changes:

- ❖ node detects local link cost change
- ❖ *bad news travels slow* - “count to infinity” problem!
- ❖ 44 iterations before algorithm stabilizes:
- ❖ $D_y(x) = \min\{c(y, x) + D_x(x), c(y, z) + D_z(x)\} = \min\{60+0, 1+5\} = 6$
- ❖ $D_z(x) = \min\{c(z, x) + D_x(x), c(z, y) + D_y(x)\} = \min\{50+0, 1+6\} = 7$
- ❖ $D_y(x) = 8, D_z(x) = 9, \dots$ totally 44 iteration!



poisoned reverse:

- ❖ If Z routes through Y to get to X :
 - Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- ❖ will this completely solve count to infinity problem?

第一步 $y \rightarrow x$ 的距离就出问题了, $1 + 5$ 是 $y \rightarrow z \rightarrow y \rightarrow x$ 的距离, 因为 z 里存的距离还是 $z \rightarrow y \rightarrow x = 1 + 4 = 5$ 还没更新。

- 毒性逆转 Poisoned Reverse

z 告诉 y : $D_z(x) = \infty$

- LS 与 DV 的对比

message complexity

- **LS:** with n nodes, E links, $O(nE)$ msgs sent
- **DV:** exchange between neighbors only
 - convergence time varies

speed of convergence

- **LS:** $O(n^2)$ algorithm requires $O(nE)$ msgs
 - may have oscillations
- **DV:** convergence time varies
 - may be routing loops
 - count-to-infinity problem

robustness: what happens if router malfunctions?

LS:

- node can advertise incorrect *link* cost
- each node computes only its own table

DV:

- DV node can advertise incorrect *path* cost
- each node's table used by others
 - error propagate thru network

5.3 因特网中自治系统内部的路由选择: OSPF

- 自治系统 Autonomous System (AS)

每个 AS 由一组通常处在相同管理控制下的路由器组成。通常在一个 ISP 中的路由器以及互联它们的链路组成一个 AS。一个自治系统由其全局唯一的 AS 号 (ASN) 所标识。AS 又称为 **domain**。

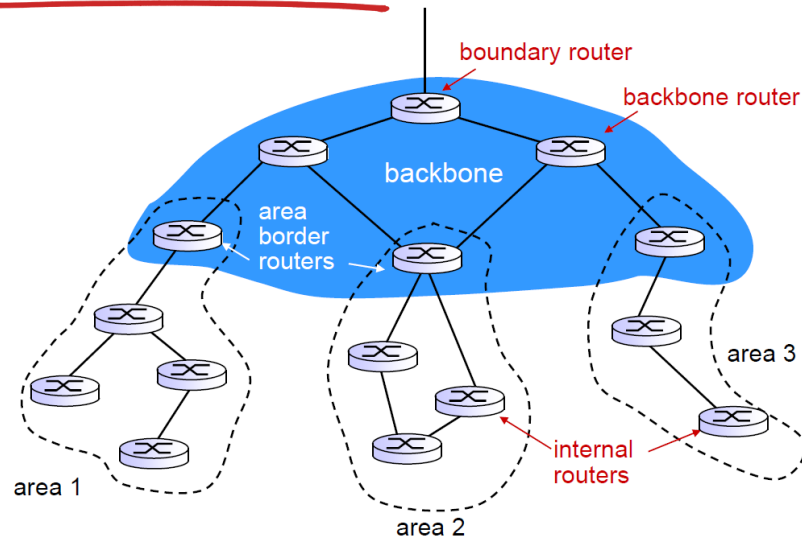
- 网关路由器 Gateway Router
连接其他 AS 的路由器

- 内部路由器 Interior Router

在一个 AS 内部运行的路由选择算法叫做自治系统内部路由选择协议 (intra-autonomous system routing protocol, IGP)

- 开放最短路优先 OSPF Open Shortest Path First
 - Public
 - 链路状态 (LS) - Dijkstra 算法
 - 向 AS 里所有其他路由器广播路由选择信息
- 优点
 - 安全
 - 多条相同开销的路径
 - 对单播与多播路由选择的综合支持
 - 支持在单个 AS 中的层级结构
- OSPF 分层

Hierarchical OSPF



- 两层
 - area: LS 算法只在 area 内部运行
 - backbone
- area border router
 - 总结内部的路由选择信息，通知其他 area border router
- backbone router
 - backbone router 之间运行 OSPF 算法
- boundary router
 - 连接其他 AS

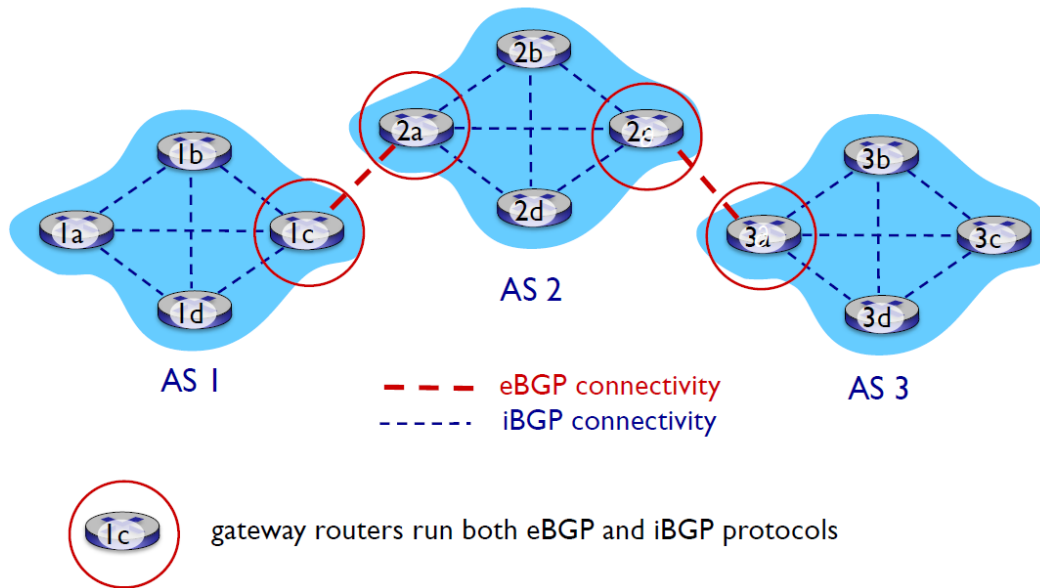
5.4 ISP 之间的路由选择: BGP

- 自治系统间路由选择协议 Inter-autonomous System Routing Protocol
- 边界网关协议 Border Gateway Protocol (BGP)
 - 非常重要，和 IP 协议一个级别
 - ISP 通过 BGP 才能互联

5.4.1 BGP 的作用

- 从邻居 AS 获得前缀的可达性信息 (eBGP)
- 将路由选择信息传播给所有 AS 的内部路由器 (iBGP)
- 确定到该前缀的“最好的”路由

5.4.2 通告 BGP 路由信息



- 运输层: TCP 端口: 179
- 距离: AS3 向 AS1和2 通告其中的一个路由器 x 的可达信息
 1. 3a 告诉 2c: AS3 x
 2. 2c 在 AS2 中广播: AS3 x
 3. 2a 告诉 1c: AS2 AS3 x
 4. 1c 在 AS1 中广播: AS2 AS3 x

5.4.3 确定最好的路由

- BGP 属性 (Attribute)

前缀+属性=路由
- 两个重要的 BGP 属性

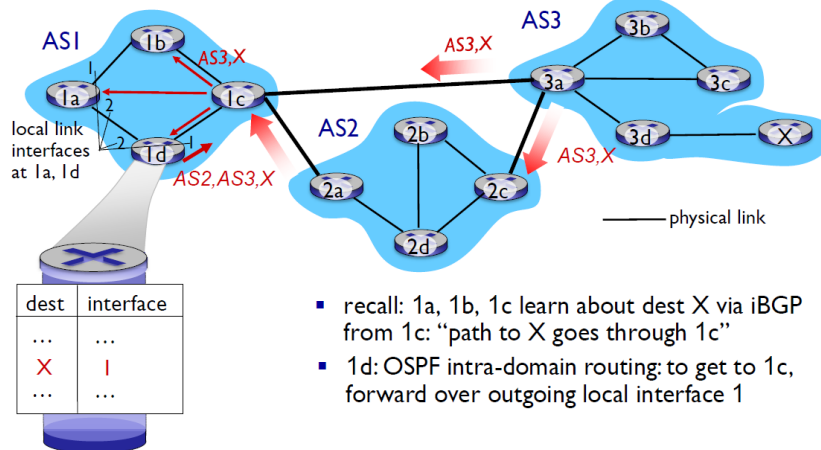
AS-PATH: 比如上面那个例子

NEXT-HOP: AS-PATH 起始路由器的 IP 地址

按上面的例子, AS1 知道的路由大概是 `xxx.xxx.xxx.xxx: AS2 AS3; x`
- 多条路由的选择策略 (按顺序)
 - Local policy decision
 - Shortest AS-PATH
 - Closet NEXT-HOP router (hot potato routing)
 - Additional criteria 如 BGP 标识符
- 转发表

BGP, OSPF, forwarding table entries

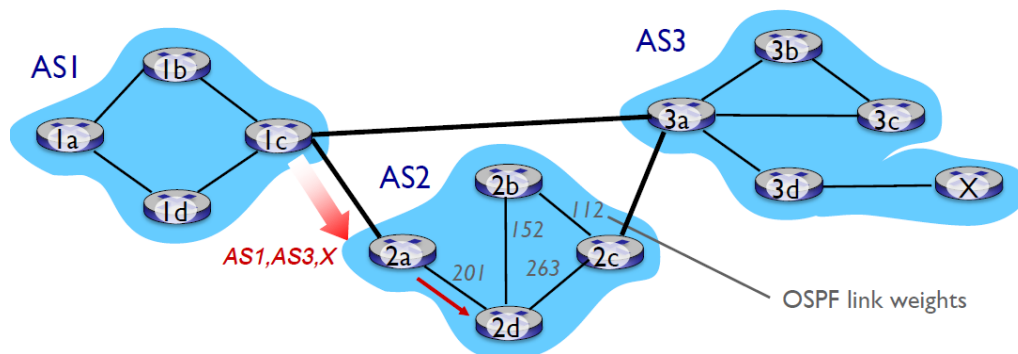
Q: how does router set forwarding table entry to distant prefix?



- recall: 1a, 1b, 1c learn about dest X via iBGP from 1c: "path to X goes through 1c"
- 1d: OSPF intra-domain routing: to get to 1c, forward over outgoing local interface 1

• 热土豆路由选择 Hot Potato Routing

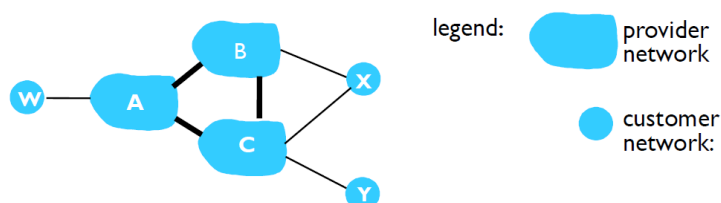
内部路由器会选择到网关路由器的最小开销路径



- 2d learns (via iBGP) it can route to X via 2a or 2c
- hot potato routing**: choose local gateway that has least intra-domain cost (e.g., 2d chooses 2a, even though more AS hops to X): don't worry about inter-domain cost!

5.4.4 BGP 通告路由选择

BGP: achieving policy via advertisements



Suppose an ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs)

- A advertises path Aw to B and to C
- B **chooses not to advertise** BAw to C:
 - B gets no "revenue" for routing CBAw, since none of C, A, w are B's customers
 - C does not learn about CBAw path
- C will route CAw (not using B) to get to w

- A,B,C are *provider networks*
- X,W,Y are customer (of provider networks)
- X is *dual-homed*: attached to two networks
- *policy to enforce*: X does not want to route from B to C via X
 - .. so X will not advertise to B a route to C

5.4.5 Intra-AS 与 Inter-AS Routing 的比较

policy:

- inter-AS: admin wants control over how its traffic routed, who routes through its net.
- intra-AS: single admin, so no policy decisions needed

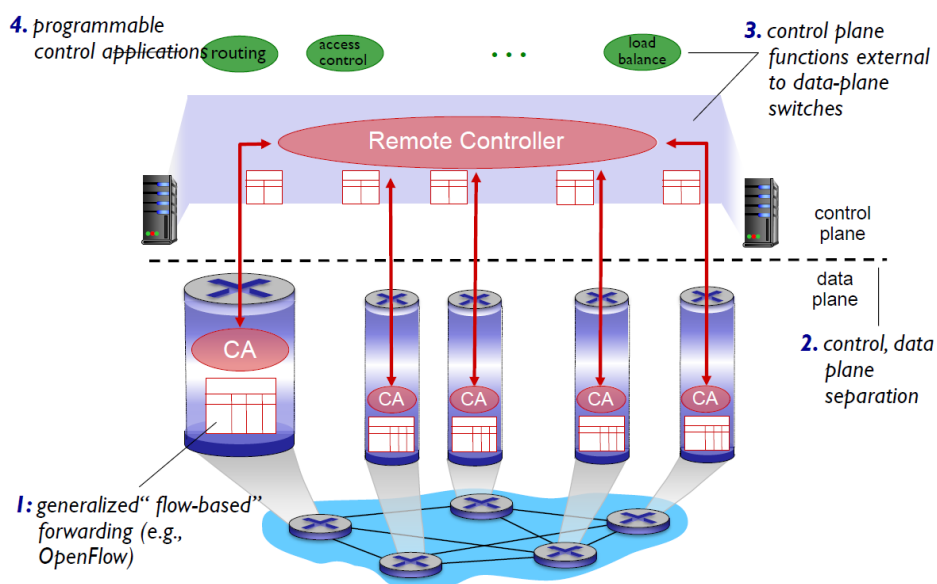
scale:

- hierarchical routing saves table size, reduced update traffic

performance:

- intra-AS: can focus on performance
- inter-AS: policy may dominate over performance

5.5 SDN 控制平面



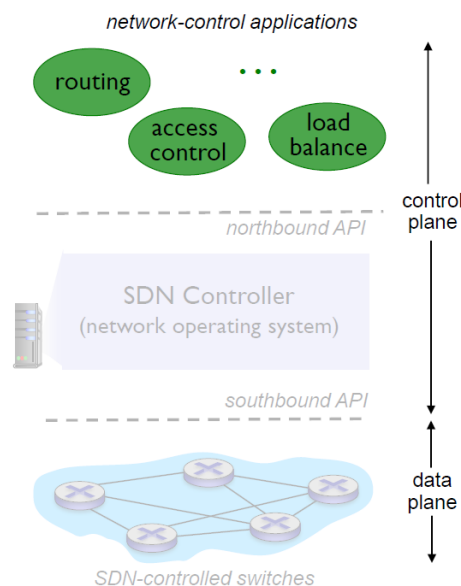
SDN (Software Defined Network) 体系的特征:

- 基于流的转发
- 数据平面与控制平面分离
- 网络控制功能：位于数据平面交换机外部

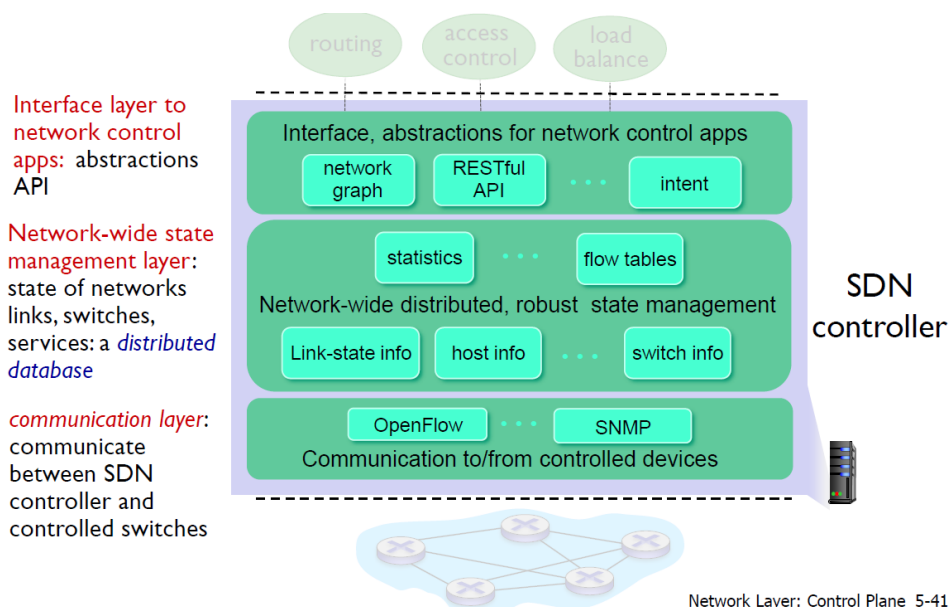
- 可编程的网络

5.5.1 SDN 结构

- Data plane switches
 - 快速、简单的 switch，运行数据平面的转发算法
 - SDN Controller 给它们计算并安装流表 (Flow Table)
 - API for table-based switch control (如 OpenFlow)
 - 与 Controller 之间的通信协议 (如 OpenFlow)
- SDN Controller (Network OS)
 - 维护网络状态
 - 通过 northbound API 与 SDN 网络控制应用程序通信
 - 通过 southbound API 与 network switches 通信
 - 分布式
- Network-control Apps



5.5.2 SDN 控制器



- 通信层
- 网络范围状态管理层
- 对于网络控制应用程序层的接口

5.5.3 OpenFlow 协议

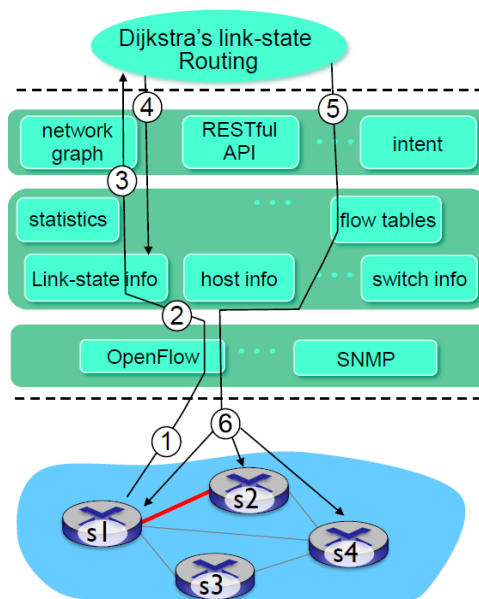
控制器 → 受控交换机

- 配置 Configure
- 修改状态 Modify State
- 读状态 Features
- 发送分组 Packet-out

受控交换机 → 控制器

- 流删除 Flow-removed
- 端口状态 Port status
- 分组入 Packet-in

5.5.4 数据平面与控制平面交互的例子



1. 交换机 s1 经历了自己与 s2 之间的链路故障，使用 OpenFlow “端口状态” 报文向 SDN 控制器通报该链路状态的更新
2. SDN 控制器接收指示链路状态更新的 OpenFlow 报文，并且通告链路状态管理器，由管理器更新链路状态库
3. 实现 Dijkstra 链路状态路由选择的网络控制应用程序先前进行了注册，当链路状态更新时将得到通告。应用程序接收该链路状态更新的通告
4. 链路状态路由选择应用程序与链路状态管理器相互作用，以得到更新的链路状态；它也会参考状态管理层中的其他组件，然后计算新的最低开销路径
5. 链路状态路由选择应用与流表管理器交互，流表管理器决定更新的流表
6. 流表管理器使用 OpenFlow 协议更新位于受影响的交换机 s1, s2 和 s4 的流表项，其中 s1 此时将经 s4 将分组的目的地指向 s2，s2 此时将经中间交换机 s4 开始接收来自 s1 的分组，s4 此时必须转发来自 s1 且目的地为 s2 的分组

5.6 ICMP: 因特网控制报文协议

- used by hosts & routers to communicate network-level information

- error reporting:
 - unreachable host, network, port, protocol
- echo request/reply (used by ping)

- network-layer “above” IP:
 - ICMP msgs carried in IP datagrams

- ICMP message:** type, code plus first 8 bytes of IP datagram causing error

Type	Code	description
0	0	echo reply (ping)
3	0	dest. network unreachable
3	1	dest host unreachable
3	2	dest protocol unreachable
3	3	dest port unreachable
3	6	dest network unknown
3	7	dest host unknown
4	0	source quench (congestion control - not used)
8	0	echo request (ping)
9	0	route advertisement
10	0	router discovery
11	0	TTL expired
12	0	bad IP header

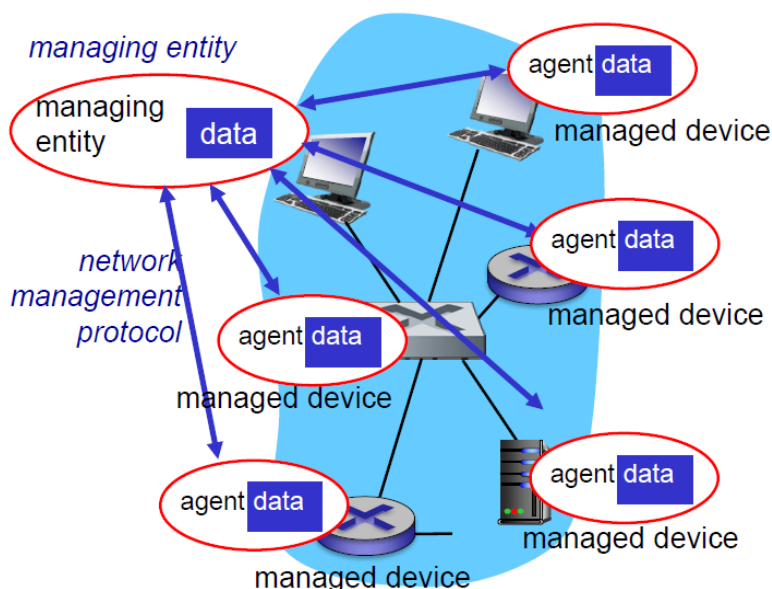
- 介于网络层和运输层之间，一般认为是网络层协议
- Traceroute

源主机向目的地发送 TTL=1, 2, 3 ... 的 ICMP 报文，当中间的某个路由器检测到了 TTL=1，他就把这个 ICMP 报文整个放到 IP 数据报的载荷里，然后加上 ICMP header 传回去。

5.7 网络管理和 SNMP

网络管理包括了硬件、软件和人类元素的设置、综合和协调，以监视、测试、轮询、配置、分析、评价和控制网络及网元资源，用合理的成本满足实时性、运营性能和服务质量的要求

5.7.1 网络管理框架



managed devices contain *managed objects* whose data is gathered into a **Management Information Base (MIB)**

- 管理服务器 Managing Server

是一个应用程序

- 被管设备 Managed Device

一个被管设备有几个被管对象 (Managed Object), 比如一个路由器和它的网卡, 路由器和它的路由选择协议都是这种关系

- 管理信息库 Management Information Base (MIB)

一个被管设备中的每个被管对象的关联信息收集在管理信息库中

- 网络管理代理 Network Management Agent

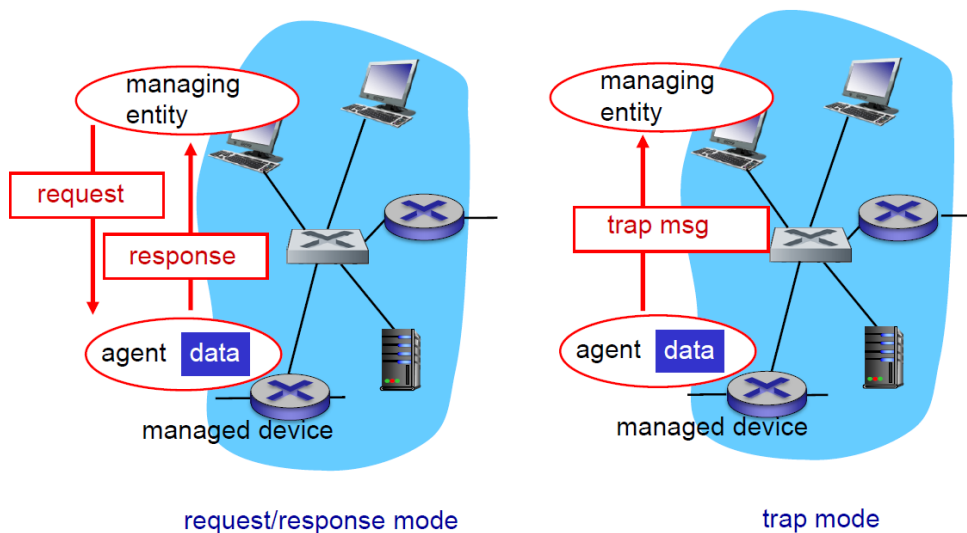
运行在被管设备中的一个进程, 与管理服务器通信

- 网络管理协议 Network Management Protocol

运行在管理服务器和被管设备之间, 如 SNMP

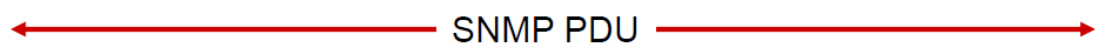
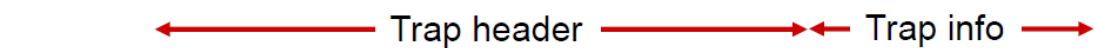
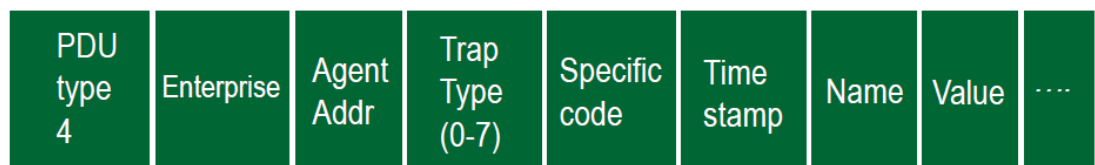
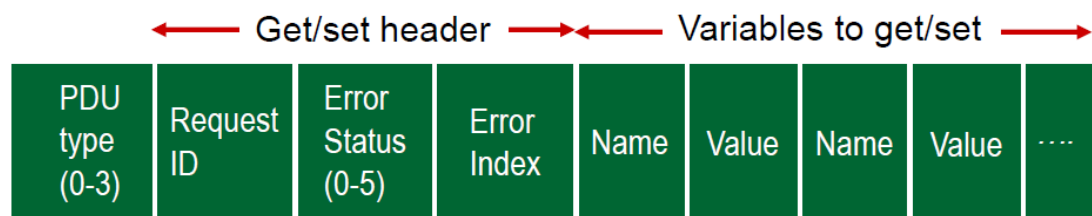
5.7.2 简单网络管理协议 SNMP

Two ways to convey MIB info, commands:



- 陷阱报文 (Trap Message) 是用于通知管理服务器一个异常情况已经导致了 MIB 改变
- 协议数据单元 PDU

<u>Message type</u>	<u>Function</u>
GetRequest GetNextRequest GetBulkRequest	manager-to-agent: "get me data" (data instance, next data in list, block of data)
InformRequest	manager-to-manager: here's MIB value
SetRequest	manager-to-agent: set MIB value
Response	Agent-to-manager: value, response to Request
Trap	Agent-to-manager: inform manager of exceptional event



第六章 链路层和局域网 Link Layer and LANs

6.1 链路层概述

- 节点 Node
 - 主机
 - 路由器
 - 交换机
 - WiFi 接入点

- 链路 Link

沿着通信路径连接相邻节点的通信信道

- 帧 Frame

链路层的数据形式

6.1.1 链路层提供的服务

- 成帧 Framing
- 链路接入

媒体访问控制 (Medium Access Control, MAC) 协议规定了帧在链路上传输的规则

- 可靠交付

然而有很多有线链路层协议不提供可靠交付服务，因为错误率太低

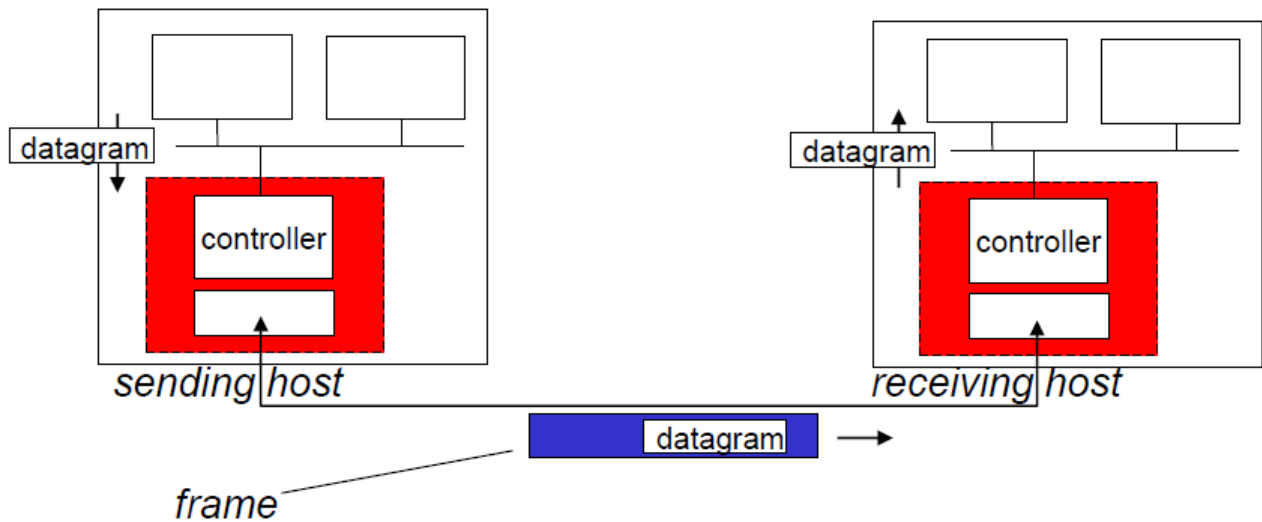
- 差错检测和纠正
- 流量控制
- 半双工和全双工 Half-duplex & Full-duplex

6.1.2 链路层在何处实现

- 网络适配器 Network Adapter

也叫**网络接口卡** (Network Interface Card, NIC, 网卡)

位于网络适配器核心的是链路层控制器



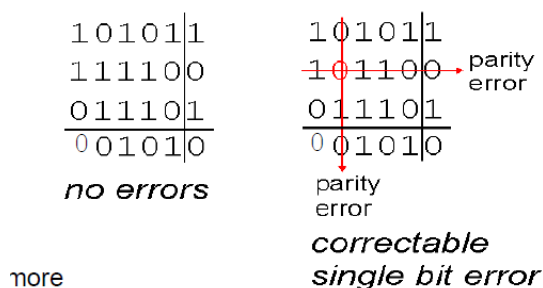
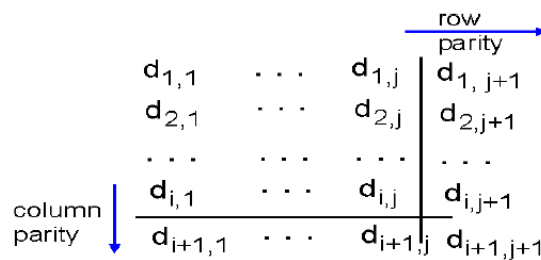
6.2 差错检测和纠正技术 Error Detection and Correction

- 差错检测和纠正比特 Error- Detection and- Correction (EDC)

6.2.1 奇偶校验

- 奇偶校验位 Parity Bit
- 二维奇偶校验 Two-dimensional Parity

可以检测并且纠正单个比特差错



6.2.2 循环冗余检测 Cyclic Redundancy Check

- CRC 编码又称多项式编码 (Polynomial Code)

CRC 的流程:

1. 发送方和接收方先协商一个 $r + 1$ 的比特模式 G , 称为生成多项式 (generator)

G 的最高有效位比特必须是 1

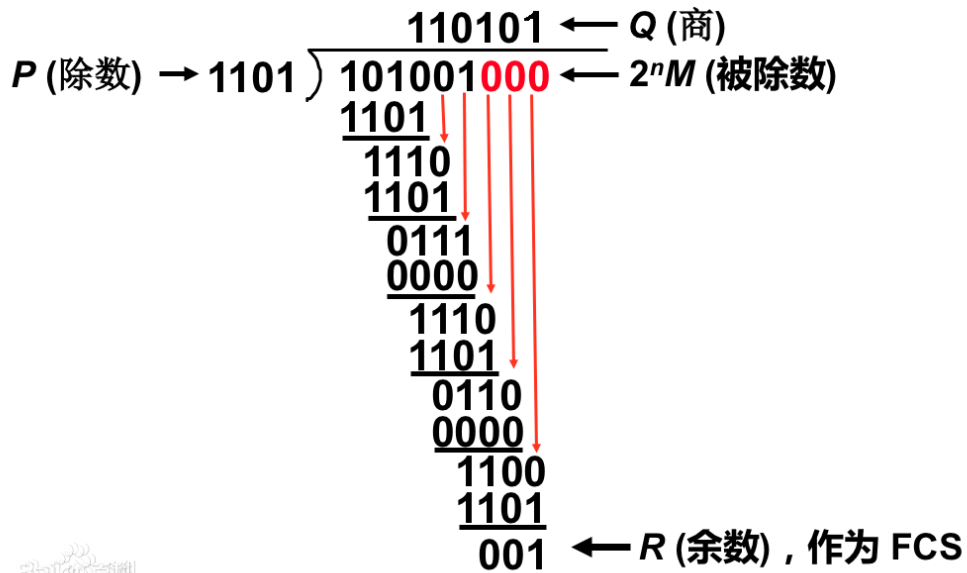
实际上就是两边先商量好一个二进制数

2. 对于一个数据段 D ，发送方从里面选 r 个附加比特 R ，把 R 附加到 D 后面得到的 $d + r$ 比特模式（实际就是个二进制数）模 2 算术能被 G 整除

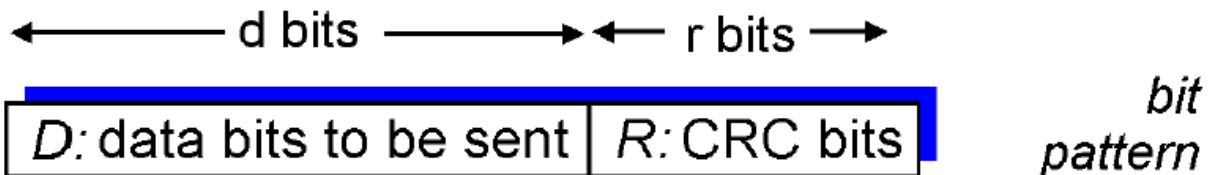
3. 接收方用 G 去除接收到的 $d + r$ 比特。如果余数非 0 证明出错

- 模 2 除法

没有借位进位，所以不论加减一律按异或 (XOR) 处理



- R 的计算



$$D * 2^r \text{ XOR } R \quad \text{mathematical formula}$$

- 你最后要把数据搞成这样
- 你要先把 D 左移 r 位: $D \times 2^r$
- 然后从后面补上 R
- 实际上是把后面的 r 位的 0 变成 R : $D \times 2^r \oplus R$
我怎么觉得用 or 也行，或者直接加
- 然后你要让这个东西能被 G 整除
- R 是你自己规定的
- 所以你可以先用 $D \times 2^r$ 除以 G ，然后把余数当做 R 不就能让整体整除了吗
- $R = \text{remainder}(\frac{D \times 2^r}{G})$
- 上面那个除法的图就是这个过程

6.3 多路访问链路和协议 Multiple Access

- 两种网络链路

1. 点对点链路 Point-to-point Link

- 点对点协议 Point-to-point Protocol (PPP)
- 高级链路数据控制 High-level Data Link Control (HDLC)

2. 广播链路 Broadcast Link

- Upstream HFC (混合光纤同轴网络)
- 802.11

- 多路访问协议 Multiple Access Protocol

规范在共享的广播信道上的传输行为

- 信道划分协议 Channel Partitioning Protocol
- 随机接入协议 Random Access Protocol
- 轮流协议 Taking-turns Protocol

- 碰撞 Collide

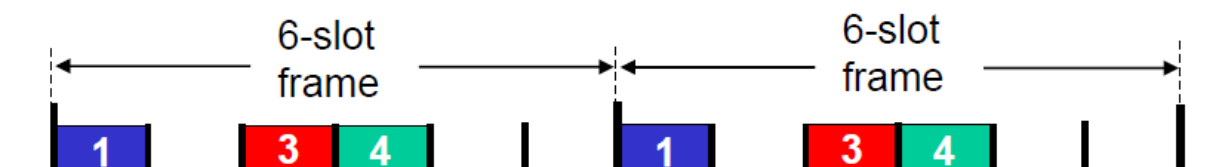
多个节点同时传输帧，所以接收方同时接收到多个帧，传输的帧在接收方处碰撞
所以混在一起全乱了，没有能正确传输的

6.3.1 信道划分协议 Channel Partitioning Protocol

- 时分多路复用 Time Division Multiple Access (TDMA)

将时间划分为时间帧 (Time Frame)

将每个时间帧划分为 N 个时隙 (Slot)

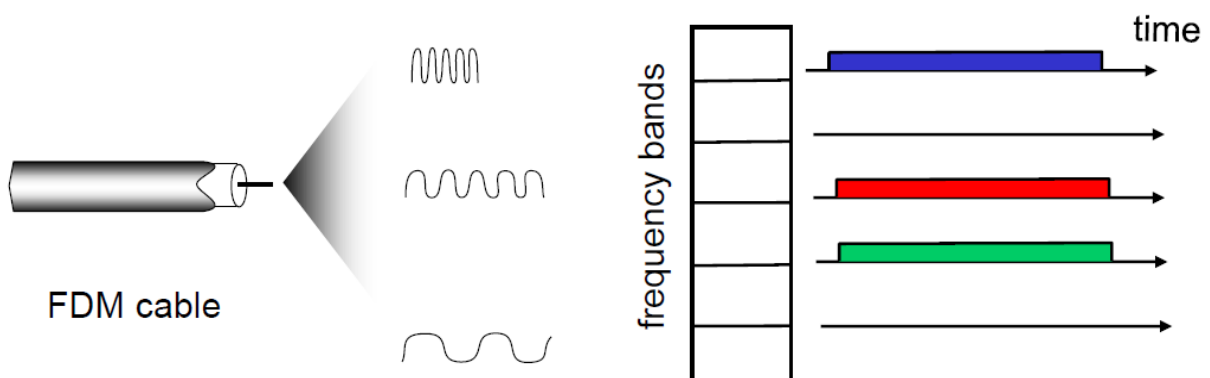


就是让各位按规定好的时间表传输

缺点是不传输的节点也给他预留了 slot

传输速率: $R/N \text{ bps}$

- 频分多路复用 Frequency Division Multiple Access (FDMA)



一人一个频率

传输速率: $R/N \text{ bps}$

- 码分多址 Code Division Multiple Access

每个节点分配一个独特的编码

不同节点能同时传输

抗干扰

6.3.2 随机接入协议 Random Access Protocol

- 每个节点都是全速发送 ($R \text{ bps}$)

6.3.2.1 时隙 ALOHA

- 假设

- 所有帧由 L 比特组成
- 时间被划分为长度为 L/R 秒的时隙 (一个时隙等于传输一帧的时间)
- 节点只能在时隙起点开始传输帧
- 节点是同步的, 每个节点都知道时隙何时开始
- 如果一个时隙中有两个或以上的帧碰撞, 则所有节点在该时隙结束之前能检测到碰撞

- 行为

令 p 为一个随机概率 `random(0, 1)`

- 当节点有一个新帧要发送时, 它等到下一个时隙开始并在该时隙传输整个帧
- 如果没有碰撞, 该节点成功传输它的帧, 可以接着准备下一个帧
- 如果有碰撞, 该节点在时隙结束之前检测到这次碰撞。该节点以概率 p 在后续的 **每个时隙**重传, 直到该帧被无碰撞地传输出去

- 效率 Efficiency

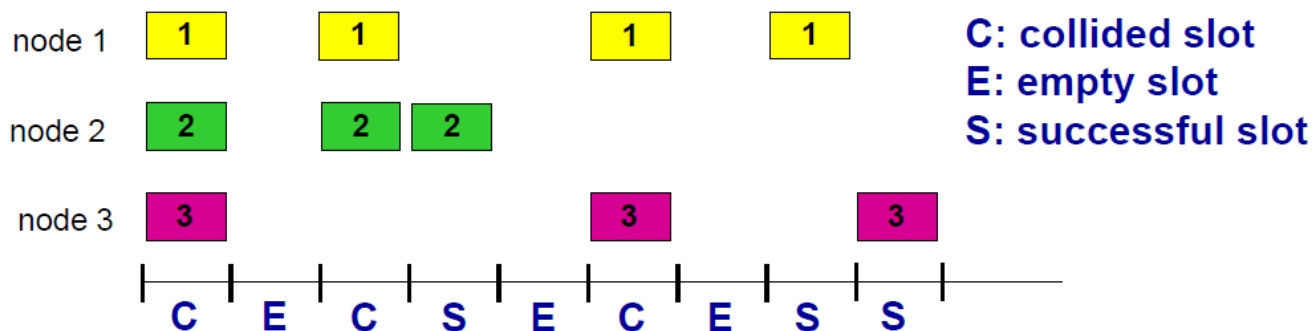
- 成功时隙 Successful Slot

刚好有一个节点传输的时隙

- 效率计算

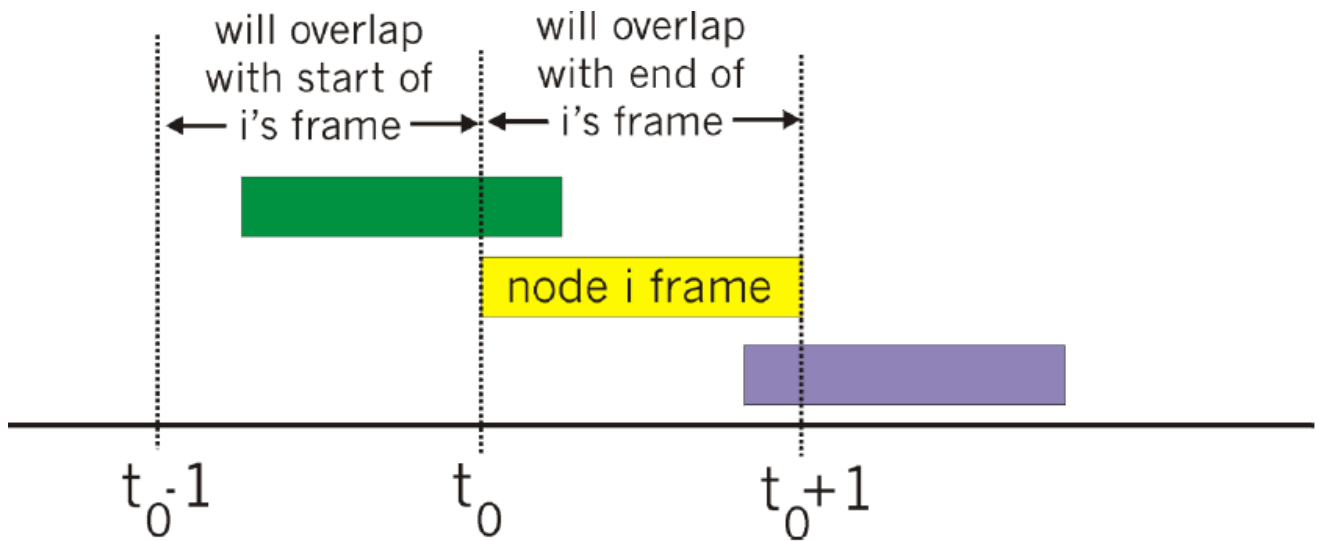
$$Np(1-p)^{N-1}$$

$N \rightarrow \infty$ 时最大效率只有 $\frac{1}{e} = 0.37$, 挺拉胯的



6.3.2.2 ALOHA

- 没有时隙，任意时刻都能发送



- 效率

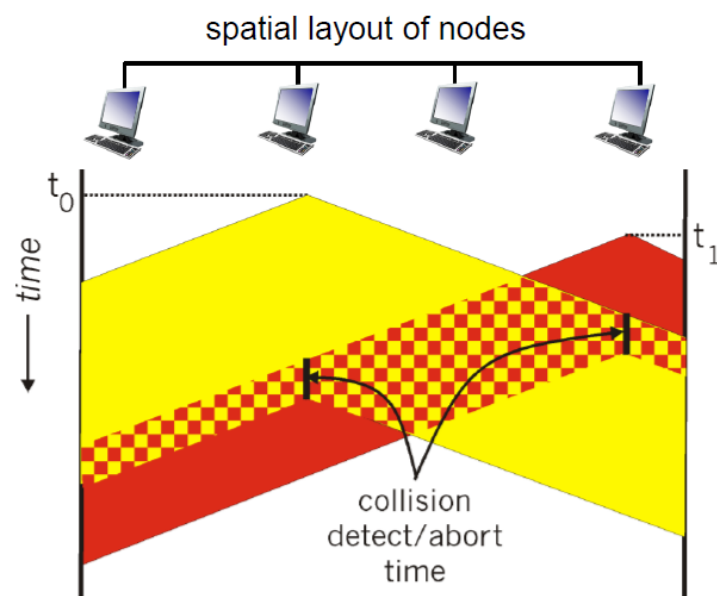
$$p(1-p)^{2(N-1)}$$

$N \rightarrow \infty$ 时只有 $\frac{1}{2e}$, 更低

6.3.2.3 载波侦听多路访问 CSMA

- 载波侦听 Carrier Sensing
说话之前先听。如果其他人正在说话，等到他们说完话为止
- 碰撞检测 Collision Detection
如果与他人同时开始说话，停止说话
- 信道传播时延 Channel Propagation Delay
越长，节点越可能监听不到其他节点的传输

6.3.2.4 具有碰撞检测的 CSMA (CSMA/CD)



- 流程

1. 适配器从网络层获得一条数据报，准备链路层帧，并将其放入帧适配器缓存中
2. 如果适配器侦听到信道空闲（即无信号能量从信道进入适配器），它开始传输帧。在另一方面，如果适配器侦听到信道正在忙，它将等待，直到侦听到没有信号能量时才开始传输帧
3. 传输过程中，适配器监视来自其他使用该广播信道的适配器信号能量的存在
4. 如果适配器传输整个帧而未检测到来自其他适配器的信号能量，该适配器就完成了该帧。在另一方面，如果适配器在传输时检测到来自其他适配器的信号能量，它终止传输
5. 终止传输后，适配器等待一个随机时间量，然后返回步骤 2

这个时间量常用二进制指数后退 (Binary Exponential Backoff) 算法决定

- 效率

$$\frac{1}{1+5d_{prop}/d_{trans}}$$

6.3.3 轮流协议 Taking-Turns Protocol

- 轮询协议 Polling Protocol

选一个节点作为主节点，主节点以循环的方式轮询每个节点，告诉每个节点最多能发多少帧

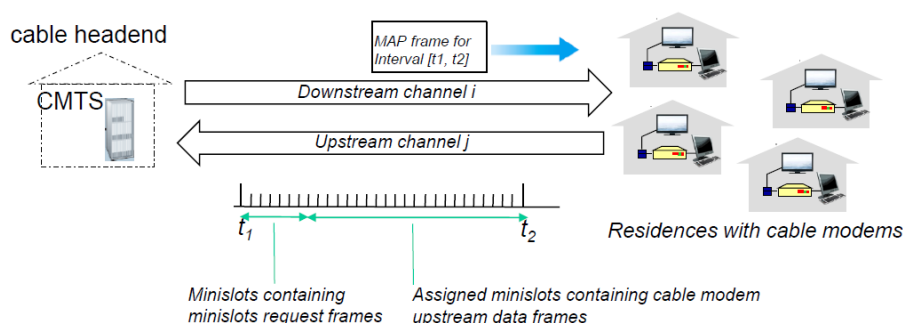
例：802.15 协议，蓝牙协议

- 令牌传递协议 Token-passing Protocol

一个称为令牌 (Token) 的小的特殊帧在节点之间以某种固定次序进行交换。当一个节点收到令牌时，仅当它有帧要发送时，它才持有令牌，然后发送最大数目的帧数，最后转发令牌给下一个节点；否则马上向下一个节点转发令牌。

例：光纤分布式数据接口 (FDDI) 协议，IEEE 802.5 令牌环协议

6.3.4 DOCSIS: 用于电缆因特网接入的链路层协议



DOCSIS: data over cable service interface spec

- FDM over upstream, downstream frequency channels
- TDM upstream: some slots assigned, some have contention
 - downstream MAP frame: assigns upstream slots
 - request for upstream slots (and data) transmitted random access (binary backoff) in selected slots

6.4 交换局域网 LANs

6.4.1 链路层寻址和 ARP

- MAC 地址

MAC 地址 = LAN 地址 = 物理地址

6 字节 48 比特

不可变（但是也有可能用软件改变）

每个适配器的 MAC 地址独一无二

例: 1A-2F-BB-76-09-AD

- 地址解析协议 Address Resolution Protocol

- ARP 表：每个主机或路由器的内存里都有

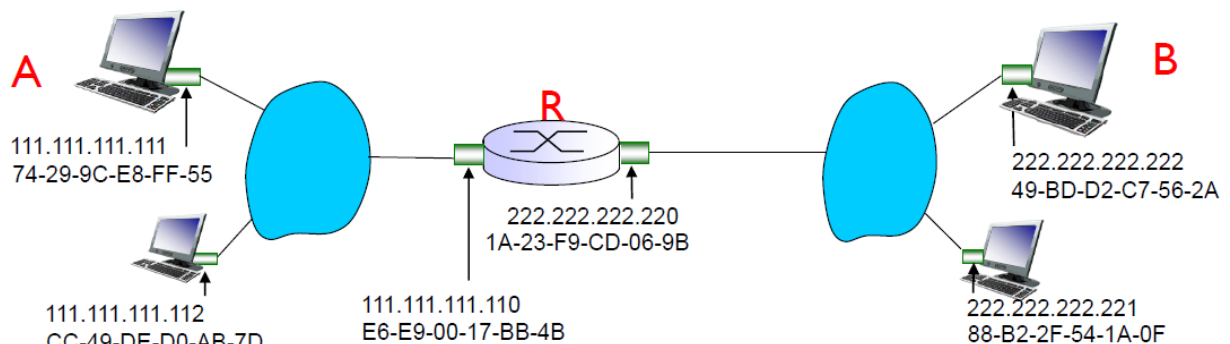
- IP 地址
 - MAC 地址
 - TTL

存的是 IP-MAC 的映射关系

- 流程

- 发送方想给接收方发消息，知道 IP 地址，但是不知道接收方的 MAC 地址
 - 发送方向广播 MAC 地址 FF-FF-FF-FF-FF-FF 发送 **ARP 查询分组**
 - 接收方收到，回复发送方告诉他 MAC 地址
 - 其他人也收到，直接丢弃，反正问的不是我
 - 发送方更新 ARP 表

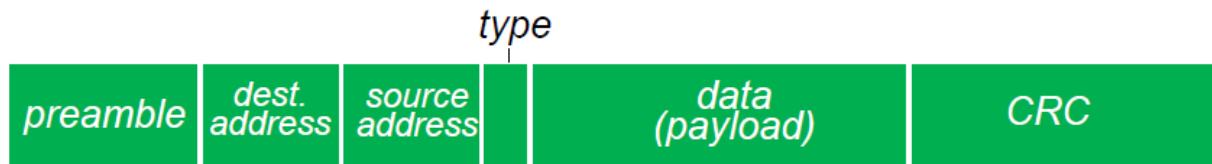
- 发送数据报到子网以外



- A 向 B 发 IP 数据报
 - 链路层帧的目的地址是 R 的 MAC 地址
 - R 收到之后转发给 B，帧的源地址是 R 的 MAC 地址，目的地是 B 的 MAC 地址
 - 注意如果 R 还不知道 B 的 MAC 地址，会先在右边的子网上跑一次 ARP
 - IP 数据报始终是 `src=A, dst=B`

6.4.2 以太网

- 有线
 - 局域网
 - 广播
 - 交换机
- 以太网帧结构



- 数据字段 46 ~ 1500 byte
 - 目的地址 6 byte
 - 源地址 6 byte
 - 类型字段 2 byte
 - CRC 4 byte
 - 前同步码 8 byte
 - 前 7 个 byte 都是 10101010
 - 最后一个 byte 是 10101011
 - 用来同步发送方和接收方的时钟
- 特点
 - connectionless
 - unreliable
 - CSMA/CD with binary backoff

6.4.3 链路层交换机

- 链路层设备
- 透明 (Transparent)
 - 发送方不知道自己发出去的帧会由交换机转发
- 即插即用
- 自学习
- 输出缓存

6.4.3.1 交换机转发和过滤

- 过滤 Filtering
 - 决定一个帧应该转发到某个接口还是应该丢弃
- 转发 Forwarding

决定一个帧应该被导向到哪个接口，并把该帧移动到那些接口

- 交换机表 Switch Table
 - MAC 地址
 - 通向该 MAC 地址的交换机接口
 - 表项放置在表中的时间

地址	接口	时间
62-FE-F7-11-89-A3	1	9:32
7C-BA-B2-B4-91-10	3	9:36
...	...	...

- 工作流程

假设目的地址为 DD-DD-DD-DD-DD-DD 的帧从接口 x 到达

- 表中没有对于 DD-DD-DD-DD-DD-DD 的表项，交换机向除了接口 x 之外的所有接口前面的输出缓存转发该帧的副本。其实就是广播该帧。flood
- 表中有个表项将 DD-DD-DD-DD-DD-DD 与 x 关联起来，丢弃（过滤功能）
- 表中有个表项将 DD-DD-DD-DD-DD-DD 与 $y \neq x$ 关联起来，交换机将该帧放到接口 y 前面的输出缓存

6.4.3.2 自学习 Self-Learning

- 交换机表初始为空
- 对于在每个接口接收到的每个入帧，交换机在其表中存储：
 1. 该帧的源地址
 2. 该帧到达的接口
 3. 当前时间
- 如果在一段时间 (老化期 Aging Time) 后，交换机没有收到以该地址作为源地址的帧，就在表中删除这个地址

6.4.3.3 链路层交换机的性质

- 消除碰撞

没有帧碰撞了

交换机的最大聚合带宽是所有接口速率之和
- 异质的链路

一个交换机可以连不同材料不同带宽的线
- 管理

6.4.3.4 交换机和路由器比较

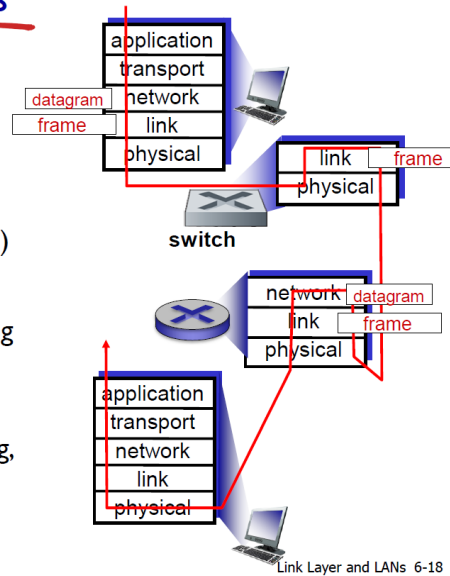
Switches vs. routers

both are store-and-forward:

- **routers**: network-layer devices (examine network-layer headers)
- **switches**: link-layer devices (examine link-layer headers)

both have forwarding tables:

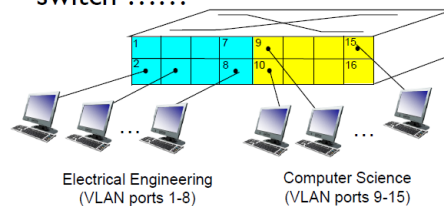
- **routers**: compute tables using routing algorithms, IP addresses
- **switches**: learn forwarding table using flooding, learning, MAC addresses



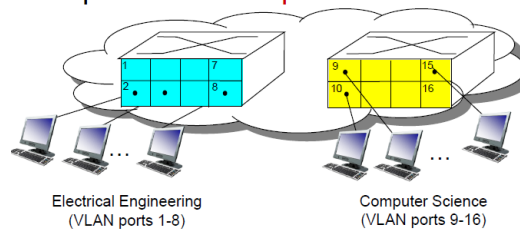
6.4.4 虚拟局域网

- 以端口划分

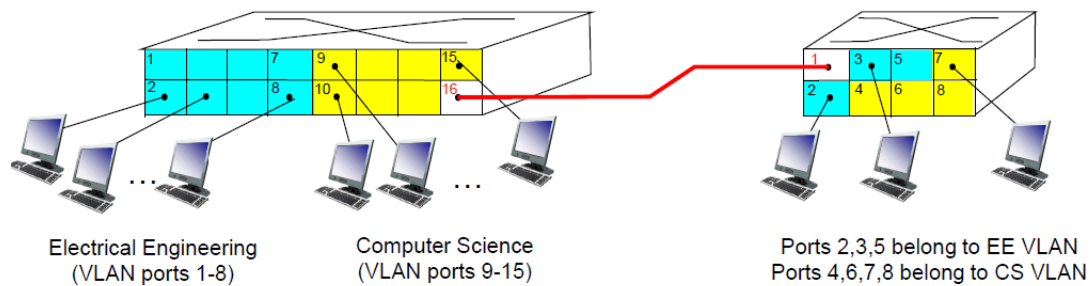
port-based VLAN: switch ports grouped (by switch management software) so that **single** physical switch



... operates as **multiple** virtual switches



- 流量隔离
一个虚拟局域网内的帧不会被发到其他虚拟局域网
- 动态用户管理
一个端口可以随时更换到不同的 VLAN 里
- VLAN 间转发
routing
就当现在是真的两台不一样的交换机
- 两台 VLAN 交换机的互联
VLAN 干线连接 (Trunking)

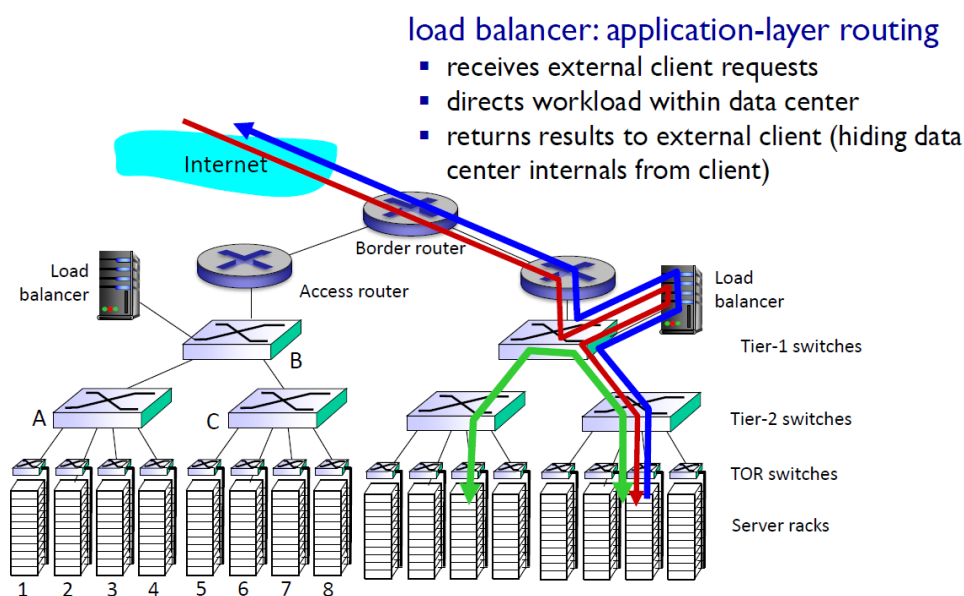


- **trunk port:** carries frames between VLANs defined over multiple physical switches
 - frames forwarded within VLAN between switches can't be vanilla 802.1 frames (must carry VLAN ID info)
 - 802.1q protocol adds/removed additional header fields for frames forwarded between trunk ports

6.5 链路虚拟化：网络作为链路层

这节没讲

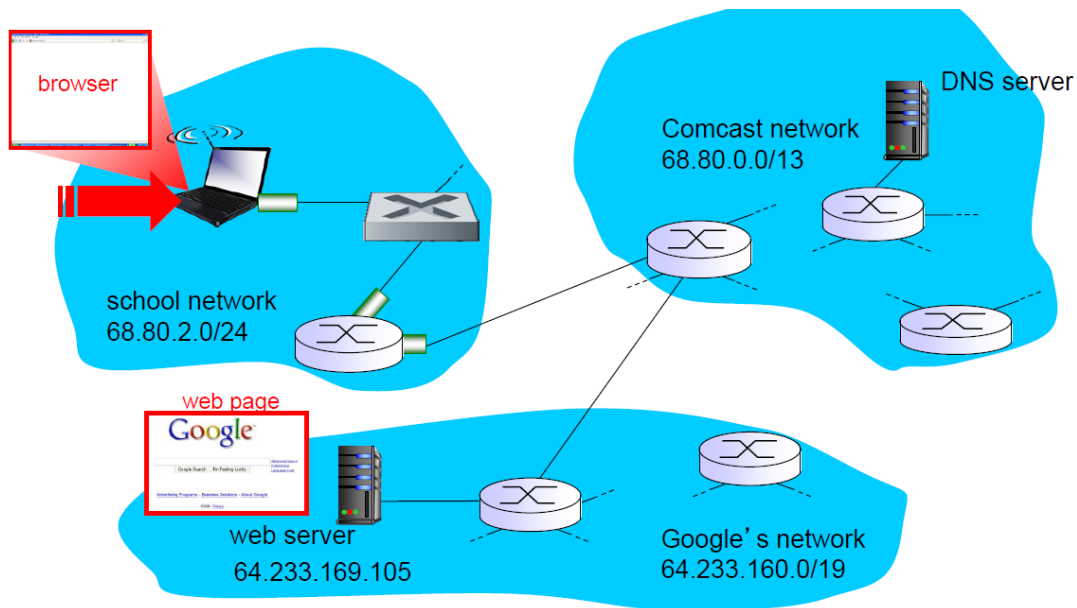
6.6 数据中心网络 Data Center Networking



- 机架顶部 (Top of Rack, TOR) 交换机
- 边界路由器 Border Router
- 负载均衡器 Load Balancer
- 路由器和交换机等级结构 Hierarchy of Router and Switch

上面那张图

6.7 回顾：Web 页面请求的历程



Bob 要用他的笔记本电脑访问 Google 主页

6.7.1 准备：DHCP、UDP、IP 和以太网

1. 操作系统生成一个 DHCP 请求报文 (4.3.3 节)，并将这个报文放入具有目的端口 67 (DHCP 服务器) 和源端口 68 (DHCP 客户) 的 UDP 报文段 (3.3 节)，该 UDP 报文段被放置在一个具有广播 IP 目的地址 255.255.255.255 和源 IP 地址 0.0.0.0 的 IP 数据报中 (4.3.1 节)
2. 包含 DHCP 请求报文的 IP 数据报则被放置在以太网帧中 (6.4.2 节)。帧的目的 MAC 地址是 FF-FF-FF-FF-FF-FF，广播到与交换机连接的所有设备
3. 该帧到达交换机，交换机在所有的出端口广播该帧
4. 路由器接收到该广播以太网帧，抽取出 IP 数据报。该数据报的广播 IP 目的地址指示了这个数据报应该由在该节点的高层协议处理，因此数据报的载荷被分解 (3.2 节) 向上到达 UDP，DHCP 请求报文被从 UDP 中抽取出来
5. DHCP 分配给 Bob 一个 IP 地址。DHCP 服务器生成包含这个地址以及 DNS 服务器的 IP 地址、默认网关路由器的 IP 地址和子网块的一个 DHCP ACK 报文，发给 Bob
6. 包含 DHCP ACK 的以太网帧由路由器发送给交换机。由于交换机是自学习的 (6.4.3 节)，并且先前从 Bob 那里收到了帧，所以匹配到了交换机表项。交换机向对应的输出端口转发
7. Bob 接收到包含 DHCP ACK 的以太网帧，一层一层抽取出 DHCP ACK 报文。Bob 的 DHCP 客户记录下 IP 地址和 DNS 服务器的 IP 地址。它还在其 IP 转发表中安装默认网关的地址 (4.1 节)

6.7.2 仍在准备：DNS 和 ARP

Bob 向浏览器地址栏输入 www.google.com。他的浏览器生成了一个 TCP 套接字 (2.7 节)，套接字用于发送 HTTP 请求 (2.2 节)

8. 操作系统生成一个 DNS 查询报文 (2.4.3 节)，将 www.google.com 放入问题段中。该 DNS 报文则放置在一个具有 53 号目的端口的 UDP 报文段中

9. Bob 把数据一层层封装进一个帧中，但是不知道默认网关的 MAC 地址，所以需要 ARP 协议 ([6.4.1 节](#))
10. Bob 生成一个 ARP 查询报文，放置在一个目的地是广播 MAC 地址的帧中，向交换机发送，然后交换机转发给其他所有设备
11. 网关路由器接收到该 ARP 查询报文，发现里面的 IP 地址匹配其接口的 IP 地址，所以它准备一个 ARP 回答报文，指示它的 MAC 地址，然后发给 Bob
12. Bob 收到 ARP 回答报文，抽取出默认网关的 MAC 地址
13. Bob 向默认网关发送 DNS 查询报文

6.7.3 还是在准备：域内路由选择到 DNS 服务器

14. 网关路由器接收到包含 DNS 查询的 IP 数据报，根据目的 IP 地址去转发表里匹配，发现应该发给 Comcast 最左边的路由器，将数据报封装到帧里经链路发送
15. 左边的路由器接收到该帧，抽取出 IP 数据报，检查数据报的目的 IP 地址，并根据转发表确定出接口，经该接口朝 DNS 服务器转发。转发表根据域内协议 (RIP, OSPF 等 [5.3 节](#)) 和因特网域间协议 BGP ([5.4 节](#)) 所填写
16. 包含 DNS 查询的 IP 数据报到达 DNS 服务器。服务器抽取出查询报文，在它的数据库中查找名字 `www.google.com` ([2.4 节](#))，找到包含对应 IP 地址的 DNS 源记录，然后生成一个 DNS 回答报文发给 Bob
17. Bob 从收到的 DNS 报文中获得 Google 的 IP 地址

6.7.4 Web 客户-服务器交互：TCP 和 HTTP

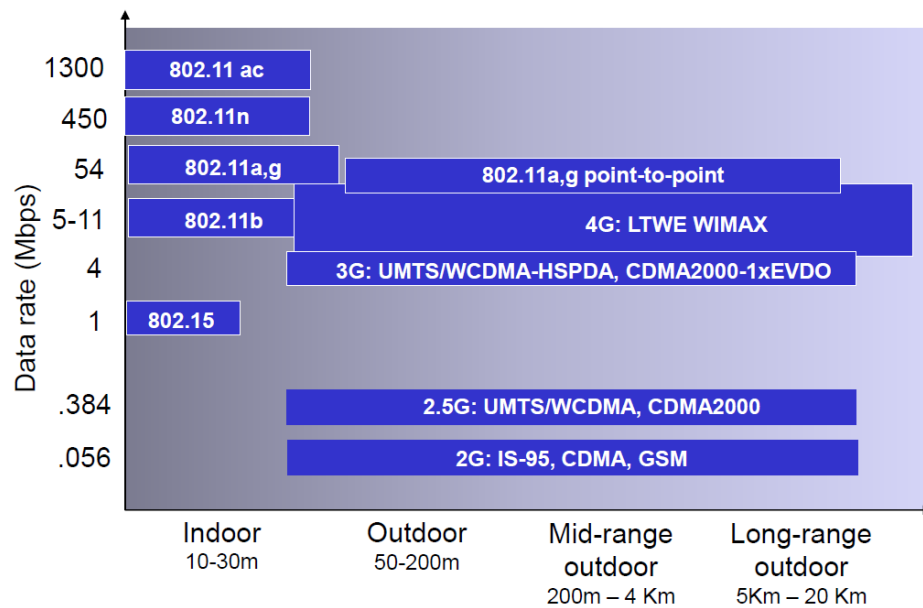
18. Bob 生成 TCP 套接字 ([2.7 节](#))，该套接字用于发送 HTTP GET 报文 ([2.2.3 节](#))。当 Bob 生成 TCP 套接字时，要先和 `www.google.com` 的 TCP 执行三次握手 ([3.5.7 节](#))。Bob 先生成一个具有目的端口 80 的 TCP SYN 报文段，向服务器发送
 19. 在学校网络、Comcast 网络和谷歌网络中的路由器朝着 `www.google.com` 转发 SYN 数据报
 20. TCP SYN 数据报到达 `www.google.com`。服务器抽取出 SYN 报文并分解到与端口 80 绑定的欢迎套接字 ([2.7 节](#))，产生一个 TCP SYNACK 报文段 ([3.5.7 节](#)) 发给 Bob
 21. SYNACK 数据报经过谷歌网络、Comcast 网络和学校网络到达 Bob。数据报被分解到第 18 步生成的套接字，进入连接状态
 22. Bob 的浏览器生成包含要获取的 URL 的 HTTP GET 报文，用 TCP 套接字发给谷歌服务器
 23. 服务器收到 GET 报文，生成 HTTP 响应报文 ([2.2 节](#))，将请求的 Web 页面内容放入 HTTP 响应体中，并将报文发送进 TCP 套接字中
 24. 数据报经转发到达 Bob，浏览器从套接字读取 HTTP 响应，抽取出 HTML，显示网页
-

第七章 无线网络和移动网络 Wireless and Mobile Networks

7.1 概述

7.1.1 无线网络的组成

- 无线主机 Wireless Host
- 无线链路 Wireless Communication Link



- 基站 Base Station
一台无线主机与某基站相关联：
 1. 该主机位于该基站的无线通信覆盖范围内
 2. 该主机使用该基站中继它和更大网络之间的数据例：蜂窝塔 Cell Tower，接入点 Access Point
- 网络基础设施
 - 与基站关联的主机通常被称为以**基础设施模式 (Infrastructure Mode)** 运行在**自组织网络 (Ad Hoc Network)** 中，无线主机没有这样的基础设施与之相连
 - 当一台移动主机的移动超出一个基站的覆盖范围而到另一个基站的覆盖范围后，它将改变关联的基站，这一过程称为**切换 (Handoff)**

7.1.2 无线网络的分类

- 标准：
 1. 在该无线网络中的分组是否跨越了一个无线跳或多个无线跳
 2. 网络中是否有基础设施，如基站

1. 单跳，基于基础设施

大部分日常使用的 WiFi 都是

2. 单跳，无基础设施

蓝牙和具有自组织模式的 802.11 网络

3. 多跳，基于基础设施

无线网状网络 Wireless Mesh Network

4. 多跳，无基础设施

移动自组织网络 Mobile Ad Hoc Network (MANET)

车载自组织网络 Vehicular Ad Hoc Network (VANET)

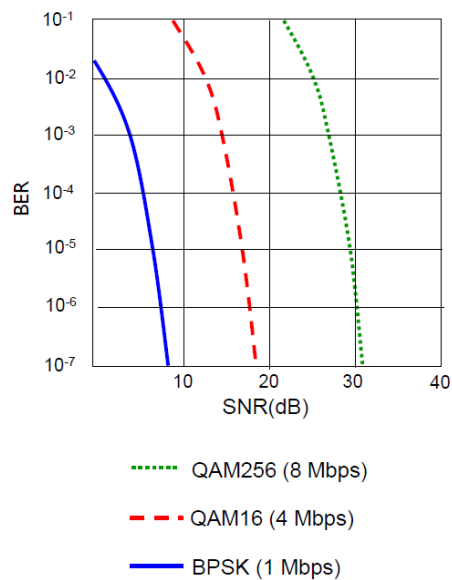
7.2 无线链路和网络特征

7.2.1 无线和有线的差别

- 递减的信号强度
路径损耗 Path Loss
- 来自其他源的干扰
- 多径传播 Multipath Propagation

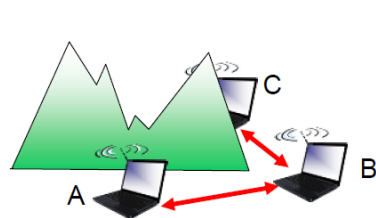
7.2.2 信噪比和比特差错率

- 信噪比 Signal-to-Noise Ratio (SNR)
所收到的信号和噪声强度的相对测量
- 比特差错率 BER
接收方收到的有错传输比特的概率
- SNR 和 BER 的关系
 - 对于给定的调制方案，SNR 越高，BER 越低
 - 对于给定的 SNR，具有较高比特传输率的调制技术（无论差错与否）将具有较高的 BER
 - 物理层调制技术的动态选择能用于适配对信道条件的调制技术



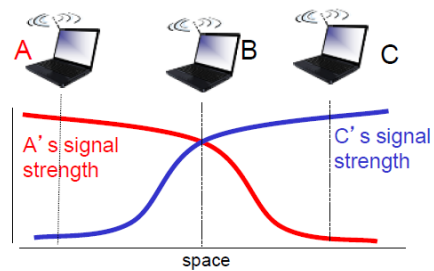
- 隐藏终端问题 Hidden Terminal Problem

Multiple wireless senders and receivers create additional problems (beyond multiple access):



Hidden terminal problem

- B, A hear each other
- B, C hear each other
- A, C can not hear each other means A, C unaware of their interference at B



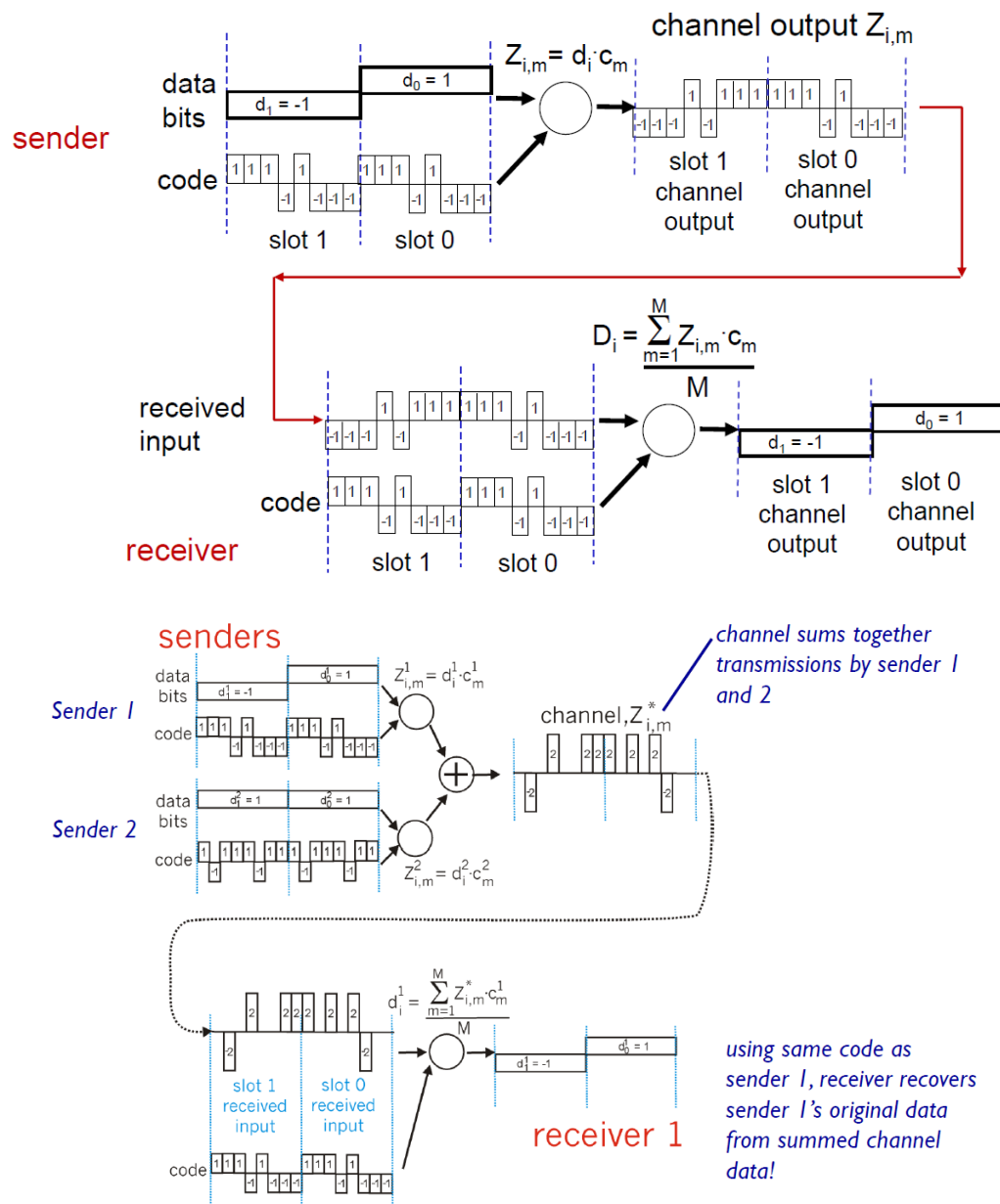
Signal attenuation:

- B, A hear each other
- B, C hear each other
- A, C can not hear each other interfering at B

- 衰减 Fading

7.2.3 码分多址 CDMA

- 发送的每个 bit 都通过乘以一个信号（编码）的比特来进行编码，这个信号的变化速率通常称为码片速率 (Chipping Rate)，比初始数据比特序列的变化速率快得多

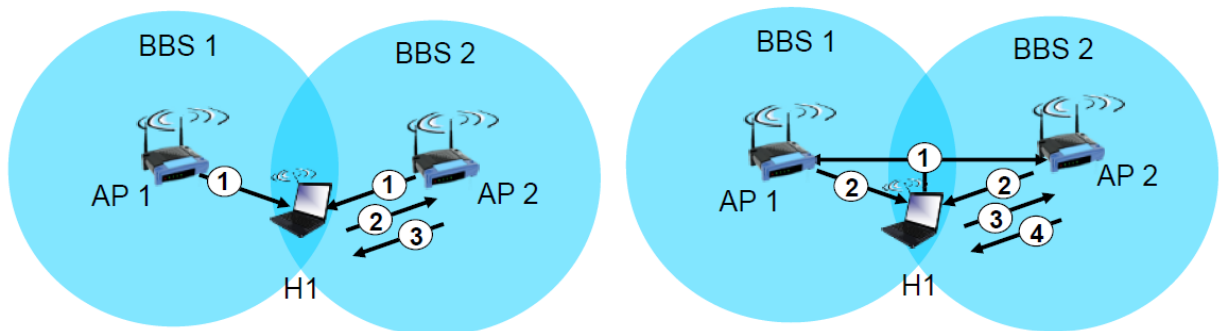


7.3 WiFi: 802.11 无线 LAN

IEEE 802.11 standard	Year	Max data rate	Range	Frequency
802.11b	1999	11 Mbps	30 m	2.4 Ghz
802.11g	2003	54 Mbps	30m	2.4 Ghz
802.11n (WiFi 4)	2009	600	70m	2.4, 5 Ghz
802.11ac (WiFi 5)	2013	3.47Gpbs	70m	5 Ghz
802.11ax (WiFi 6)	2020 (exp.)	14 Gbps	70m	2.4, 5 Ghz
802.11af	2014	35 – 560 Mbps	1 Km	unused TV bands (54-790 MHz)
802.11ah	2017	347Mbps	1 Km	900 Mhz

7.3.1 802.11 体系结构

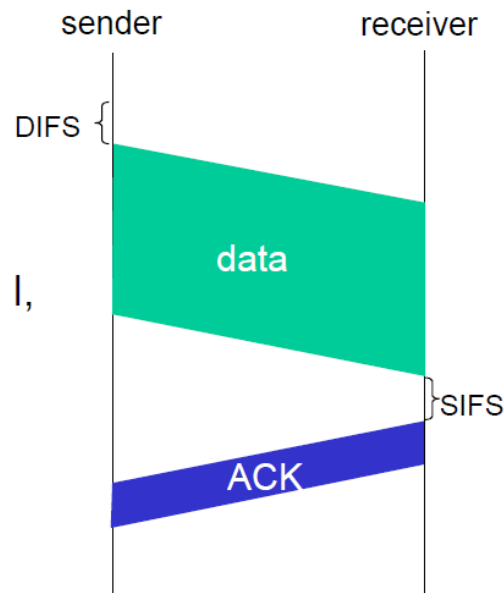
- 基本服务集 Basic Service Set (BSS)
包含一个或多个无线站点和一个接入点 (Access Point, AP)
- 基础设施无线 LAN Infrastructure Wireless LAN
配置了 AP 的无线 LAN
- 服务集标识符 Service Set Identifier (SSID)
用来标识一个 AP
- WiFi 丛林 (WiFi Jungle)
一个无线站点能从两个或以上 AP 收到很强的信号
- 关联 Associate
与一个 AP 连接
- 信标帧 Beacon Frame
每个 AP 周期性发送信标帧，包含 SSID 和 MAC 地址
- 被动扫描 Passive Scanning
扫描信道和监听信标帧
- 主动扫描 Active Scanning
向位于无线主机范围内的所有 AP 广播探测帧



7.3.2 802.11 MAC 协议

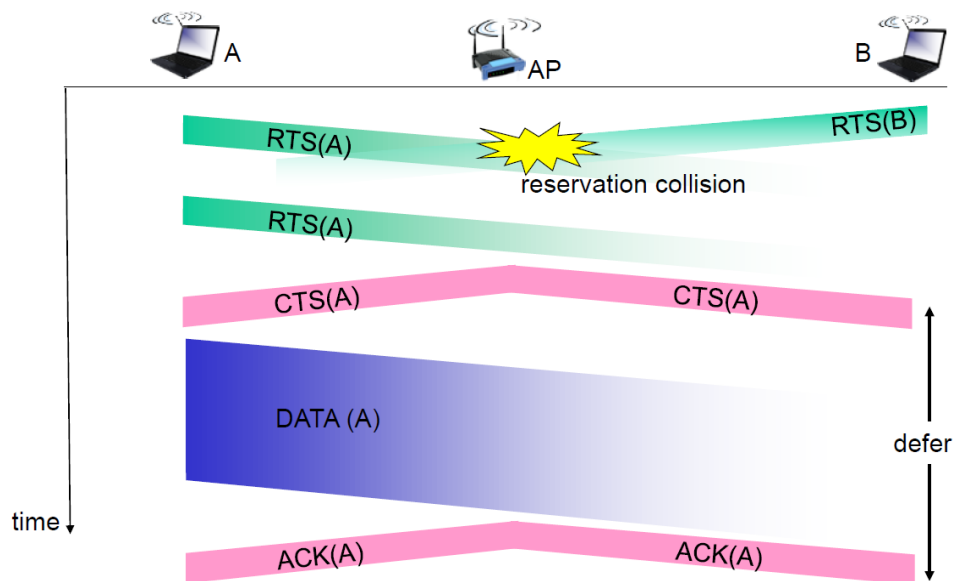
- CSMA/CA: 带碰撞避免的 CSMA (CSMA with collision avoidance)
- 没有碰撞检测
 - 检测碰撞要求站点具有同时发送和接收的能力，这个硬件开销太大
 - 还有隐藏终端问题和衰减问题
- 链路层确认 Link-Layer Acknowledgement
目的站点收到一个通过 CRC 校验的帧后，它等待一个被称为短帧间间隔 (Short Inter-Frame Spacing, SIFS) 的一小段时间，然后发回一个确认帧。如果发送站点在一定时间内未收到确认帧，它重传。如果在若干固定次重传后仍未收到确认，发送站点放弃发送并丢弃该帧
- CSMA/CA 工作流程

1. 如果某站点最初监听到信道空闲，它将在一个被称作分布式帧间间隔 (Distributed Inter-Frame Space, DIFS) 的短时间段后发送该帧
2. 否则，该站点选择一个随机回退值，并且在侦听信道空闲时递减该值。信道忙时保持不变
3. 当计数值减为 0 时，该站点发送帧并等待确认
4. 如果收到确认，发送站点知道帧已经被正确接收。如果要发送另一帧，从第二步开始；如果未收到确认，发送站点进入第二步的回退阶段，并从一个更大的范围内选取随机值

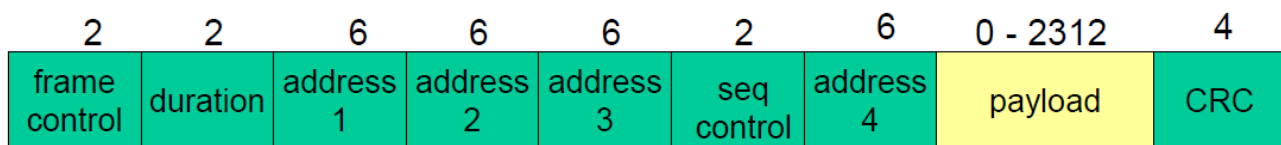


- 处理隐藏终端: RTS 和 CTS

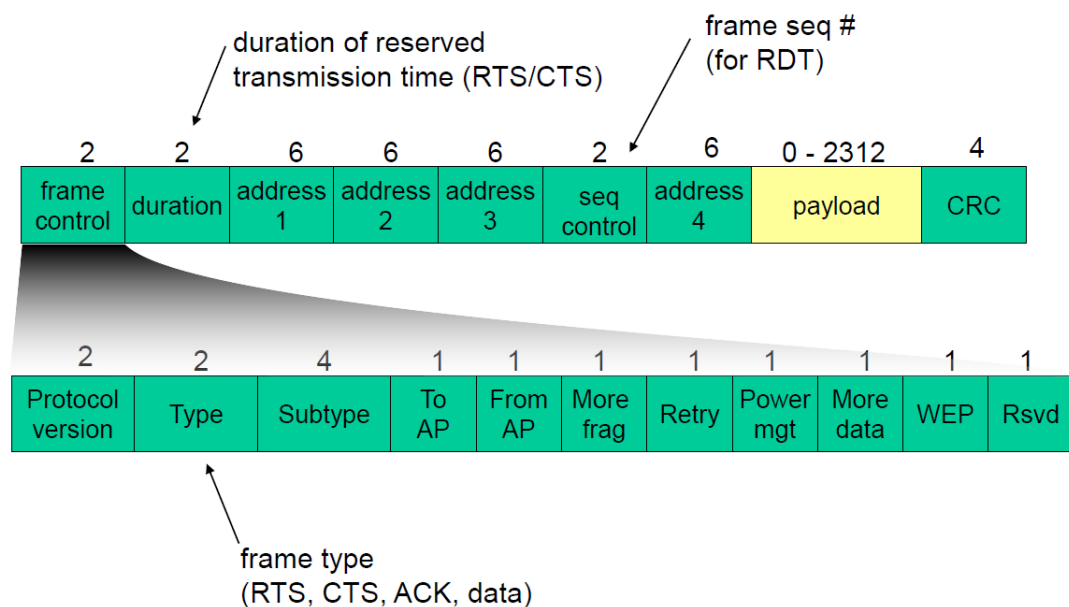
- 请求发送 (Request to Send, RTS) 控制帧
- 允许发送 (Clear to Send, CTS) 控制帧
- 相当于预约
- RTS 和 CTS 都是广播发送



7.3.3 IEEE 802.11 帧

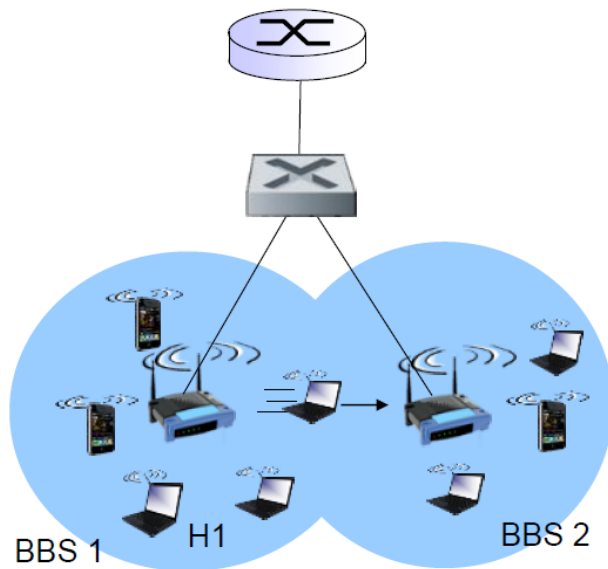


- 有效载荷与 CRC 字段
- 地址字段
 - 地址 2 是传输该帧的站点的 MAC 地址 (源地址)
 - 地址 1 是要接收该帧的无线站点的 MAC 地址 (目的地址)
 - BSS 是一个子网的一部分, 这个子网经一些路由器与其他子网相连。地址 3 包含这个路由器的 MAC 地址 (网关地址?)
 - 地址 4 在 ad hoc 模式里用
- 序号、持续期和帧控制字段



7.3.4 在相同的 IP 子网中的移动性

- 无线主机在相同子网内的不同 BSS 之间移动
- 因为子网不变, IP 地址也不变
- TCP 连接也能保持不变
- 交换机可以通过自学习修改交换机表



为什么 PPT 总是打成 BBS

7.3.5 802.11 中的高级特色

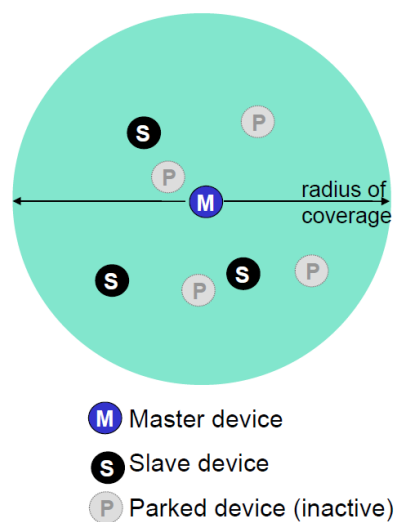
- 802.11 速率适应
根据当前和近期的信道特点选择物理层调制技术
- 功率管理
休眠与唤醒

7.3.6 个人域网络：蓝牙和 ZigBee

- 跳频扩展频谱 Frequency-Hopping Spread Spectrum (FHSS)

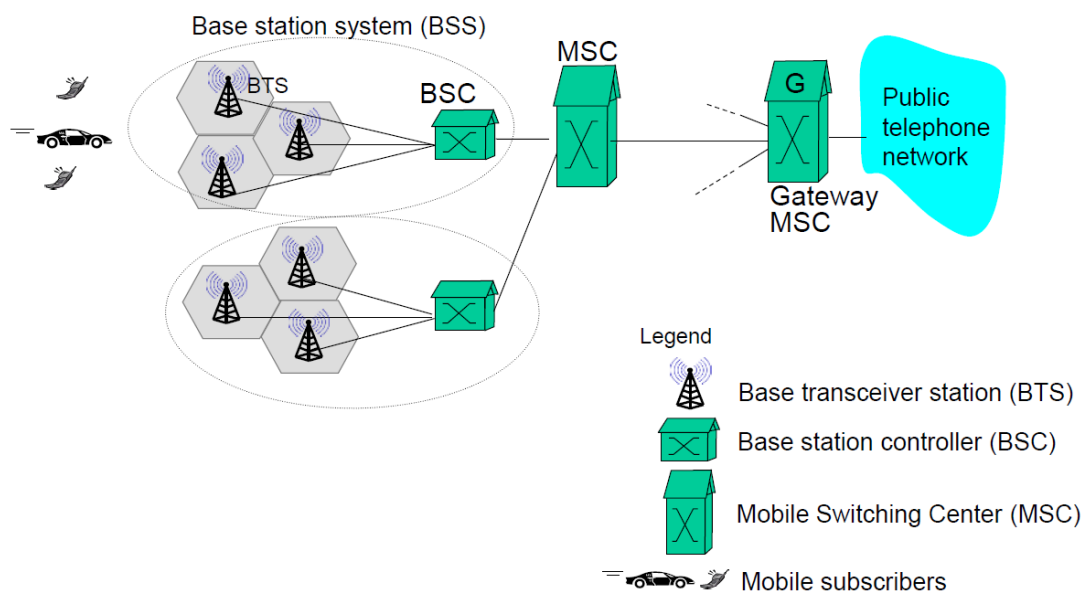
802.15: personal area network

- less than 10 m diameter
- replacement for cables (mouse, keyboard, headphones)
- ad hoc: no infrastructure
- master/slaves:
 - slaves request permission to send (to master)
 - master grants requests
- 802.15: evolved from Bluetooth specification
 - 2.4-2.5 GHz radio band
 - up to 721 kbps



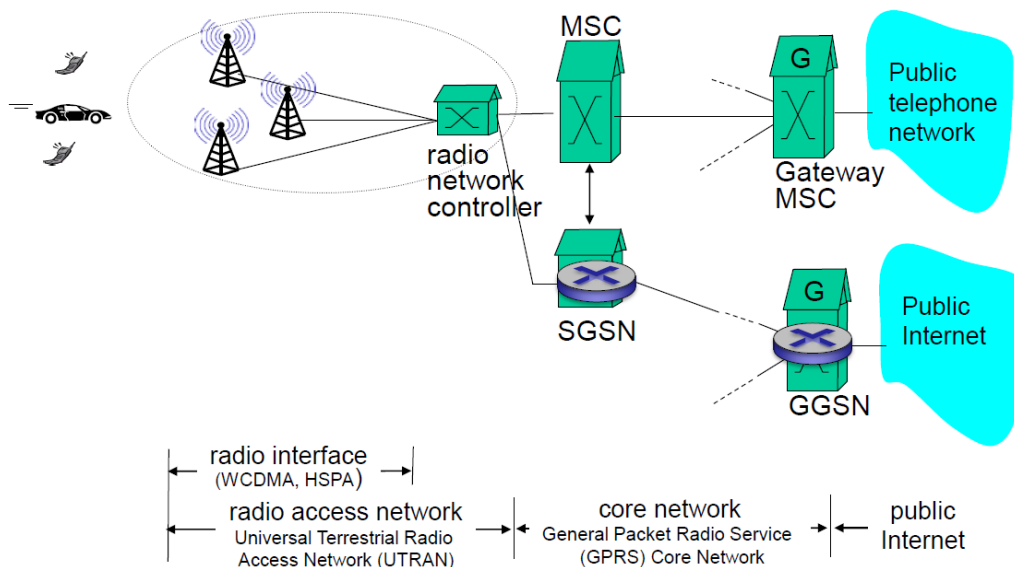
7.4 蜂窝因特网接入 Cellular Internet Access

7.4.1 2G 蜂窝网体系结构：语音与电话网连接



- 小区 Cell
- 收发基站 Base Transceiver Station (BTS)
负责向小区内的站点发送或接收信号
- 基站控制器 Base Station Controller (BSC)
为移动用户分配 BTS 无线信道
寻呼 Paging: 找出某移动用户所在的小区
执行移动用户的切换
- GSM 基站系统
- 移动交换中心 Mobile Switching Center (MSC)
用户鉴别和账户管理
呼叫建立和切换

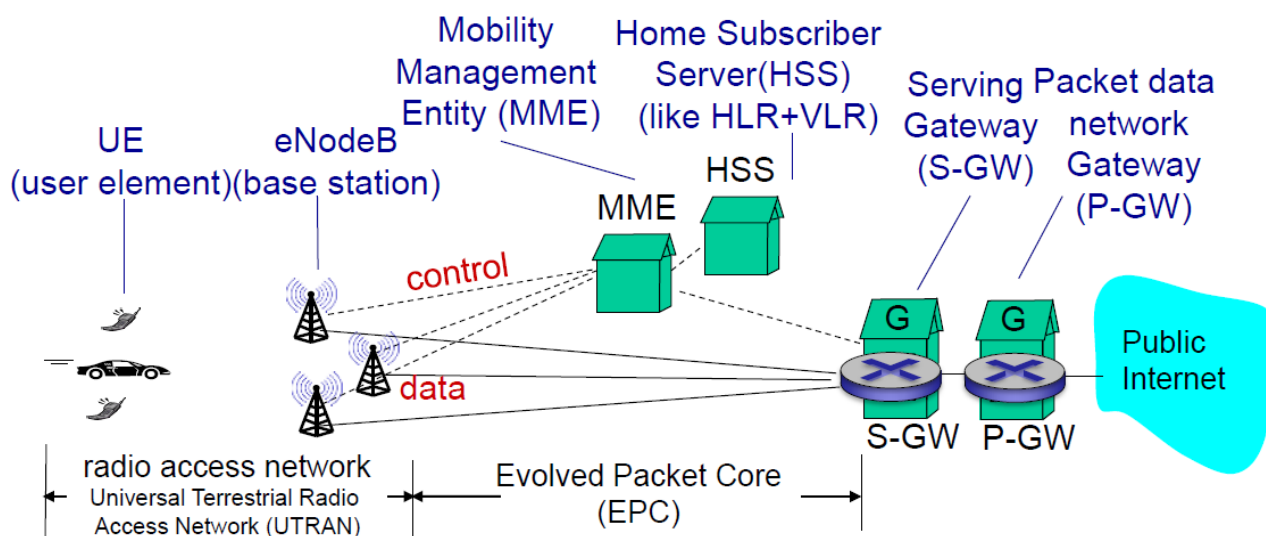
7.4.2 3G 蜂窝数据网：将因特网扩展到蜂窝用户



- 服务通用分组无线服务支持节点 Serving Generalized Packet Radio Service Support Node (SGSN)
- 网关 GPRS 支持节点 Gateway GPRS Support Node (GGSN)
- 无线电网路控制器 Radio Network Controller (RNC)

7.4.3 走向 4G: LTE

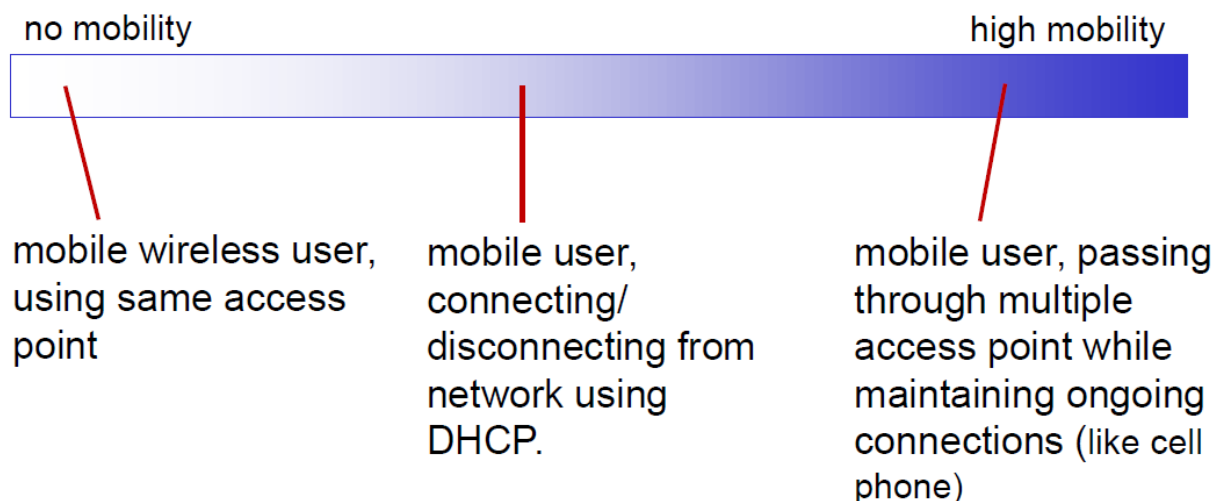
LTE: Long Term Evolution



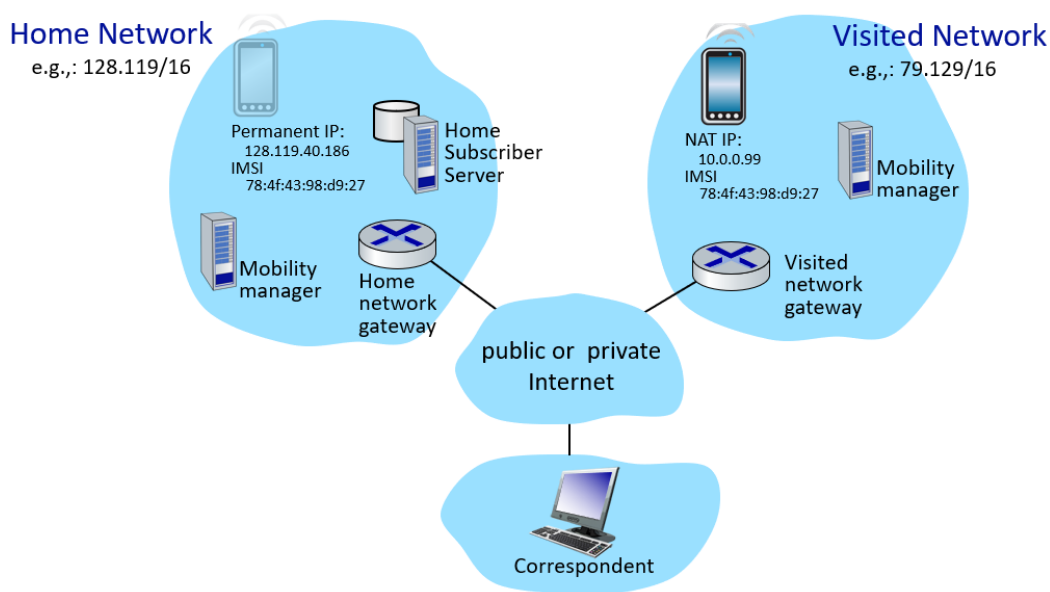
- 一种统一的、全 IP 网络体系结构
- 4G 数据平面与控制平面的清晰分离
- 无线电接入网与全 IP 核心网之间的清晰分离
- eNodeB
- 分组数据网络网关 Packet Data Network Gateway (P-GW)
- 服务网关 (S-GW)
- 移动性管理实体 Mobility Management Entity (MME)
- 归属用户服务 Home Subscriber Server (HSS)
- cnm 再多来点专业名词 Fuck Technical Nouns (FTN)

7.5 移动管理原理

- 移动性 Mobility



- 归属网络 Home Network
一个移动节点的永久居所
- 归属代理 Home Agent
在归属网络中代表节点执行移动管理功能的实体
- 外部网络 Foreign Network = 被访网络 Visited Network
移动节点当前所在网络
- 外部代理 Foreign Agent
在外部网络中帮助移动节点做移动管理功能的实体
- 通信者 Correspondent
希望与该移动节点通信的实体

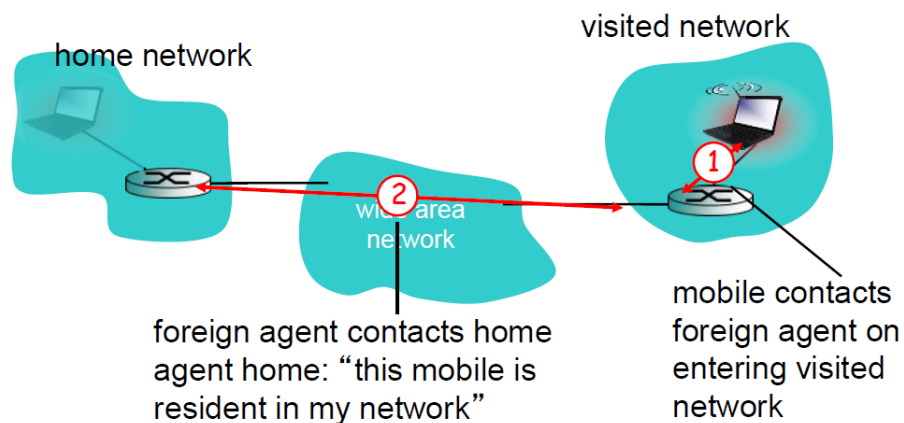


7.5.1 寻址

- 转交地址 Care-Of Address (COA) = 外部地址 Foreign Address
外部代理为移动节点创建的地址，COA 的网络部分与外部网络的网络部分匹配
- 永久地址 Permanent Address

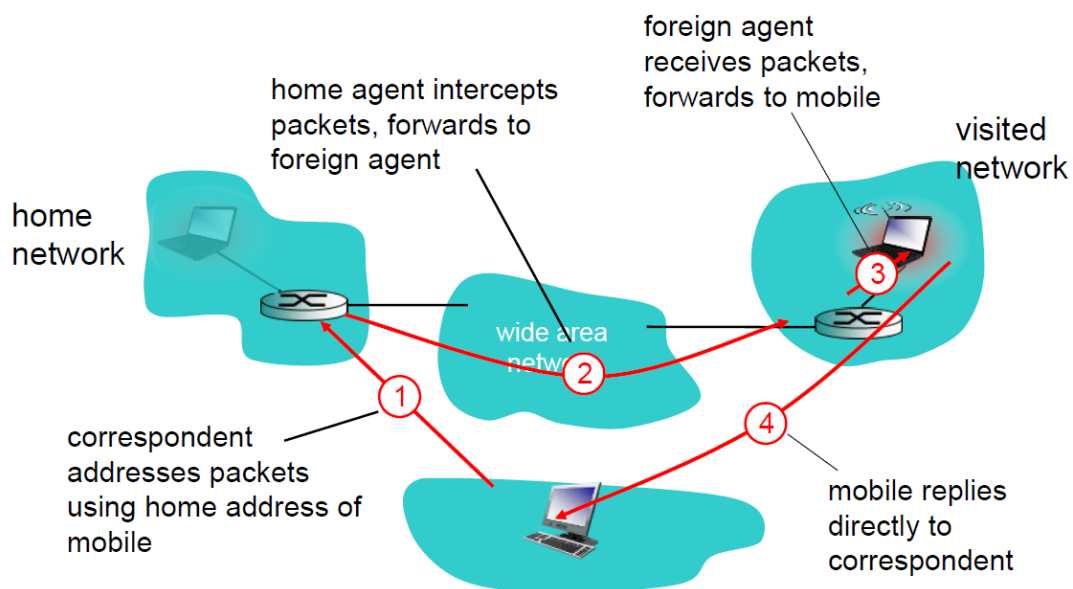
7.5.2 路由选择到移动节点

7.5.2.1 移动节点的间接路由选择 Indirect Routing



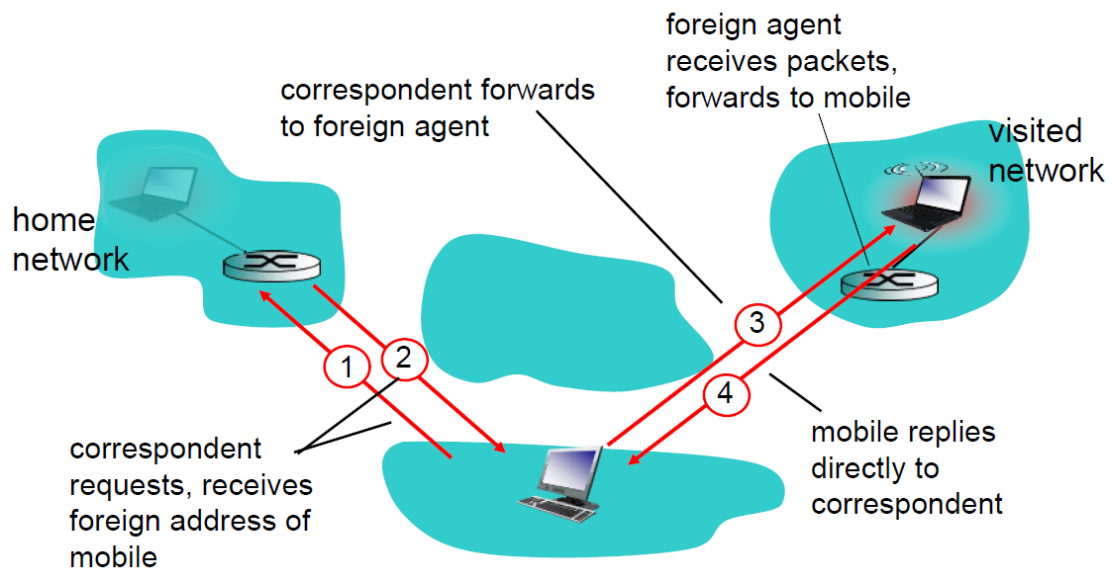
end result:

- foreign agent knows about mobile
- home agent knows location of mobile



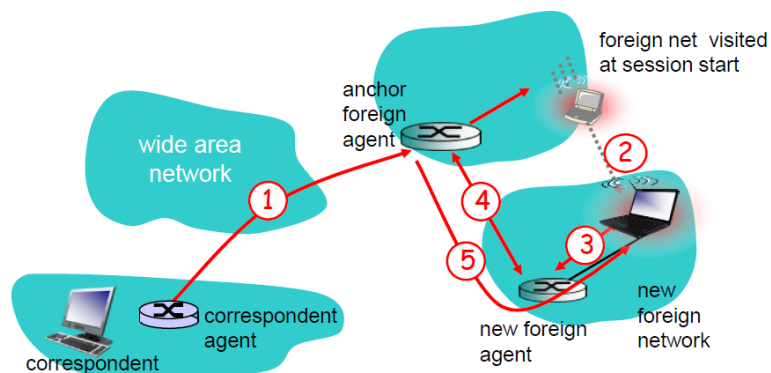
- 三角路由选择 Triangle Routing
- 移动到新子网的时候可以保持连接

7.5.2.2 移动节点的直接路由选择 Direct Routing



- 处理移动问题

- anchor foreign agent: FA in first visited network
- data always routed first to anchor FA
- when mobile moves: new FA arranges to have data forwarded from old FA (chaining)



- 锚外部代理 Anchor Foreign Agent
通信者给它发就完事了

END



